

NEUROSymbolic AUTHORING AND TEST-TIME COMPUTE ALLOCATION FOR
GENERATIVE EDUCATIONAL SIMULATIONS

By

AMOGH MANNEKOTE

A DISSERTATION PRESENTED TO THE GRADUATE SCHOOL
OF THE UNIVERSITY OF FLORIDA IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

UNIVERSITY OF FLORIDA

2026

© 2026 Amogh Mannekote

To Rainesville, in whose
summer-afternoon showers
I'm grateful to have spent
thinking, fast and slow.

ACKNOWLEDGEMENTS

When I began, I was half-convinced that needing help was a kind of failing. For years I had wished for mentors who could show me how this was done; the people in these pages became exactly that. They steadied me, challenged me, kept me fed and looked after, and slowly got me comfortable accepting it. I owe them a great deal.

My deepest thanks go to my advisor, Kristy Elizabeth Boyer, for taking me on when I had no research experience and teaching me the value of precision in language and argument. I am grateful for our shared journey through a fast-changing AI landscape and for your unwavering integrity, all while encouraging me to have a well-rounded PhD experience.

I am equally grateful to the rest of my committee. Bonnie Dorr, your work ethic and attention to detail are an inspiration; your high standards always pushed me to do my best. Benjamin Lok, for bringing a practical, human-computer interaction perspective to the work, and for your sharp questions and steady encouragement. Anthony Botelho, for your cheerful presence and for the long, enjoyable brainstorming sessions that helped me shape my early research questions.

I was fortunate to have wonderful labmates. Xiaoyi Tian, thank you for being a close friend throughout my PhD and keeping me company in the lab during the quiet post-lockdown days; our camping trip and the shared Chinese snacks made lab life much brighter. Mehmet Celepkolu, gelek spas for being my first research mentor and introducing me to the lab. Amanda Griffith, for sharing your ADHD lifehacks; and Toni Earle-Randell, for being a familiar face in unfamiliar cities. Thank you to all other LearnDialogue members, especially Joseph B. Wiggins, Yingbo Ma, Timothy Brown, and Lincoln Lu—who also has my thanks for the beautiful red bike.

At INVITE, I was lucky to find such a collaborative group. Adam Davies, some of my best brainstorming sessions were with you, and they turned into pivotal papers. Your openness to learning and sharing is something I strive to emulate. Francesco Pinto, thank

you for taking me under your wing.

Thank you to my internship mentors: Jinseok Nam and Ritesh Sarkhel at Amazon Alexa, and David McGee at Openstream. A special thanks to Delip Rao, who helped me re-ground my understanding of research when I needed it most, taught me how to bridge the gap between theory and building, and encouraged me to trust my own judgment.

Some friendships predate this journey and kept home close through the years away. Vamsi Priyatham Reddy, my ಬ್ಯಾಟರಾಯನಪುರ brother; and my friends from Bengaluru who shared this academic journey with me, Nihal Nayak and Shrikanth Yadav.

I owe the most to the people closest to me. ಅಮ್ಮ and ಅಪ್ಪ, for your constant support and love; ಅಜ್ಜಿ and ತಾತ, for showing me how to appreciate both the artistic and the rigorously mathematical aspects of life. Sridhar ಚಿಕ್ಕಪ್ಪ, for your steadfast guidance through the PhD journey. Roopa ಅತ್ತೆ, Shrirang uncle, and Ayush, for being my second family and keeping me grounded amid the chaos of the research world.

Daniel (장현), my time at your house in Bellevue meant a great deal—thank you for backing me when I doubted myself, always ready with a movie or Family Guy reference for whatever life threw at me. Ishan Agrawal, for a friendship that matured alongside our modes of transit—from bicycles to mopeds to cars—fueled by our explorations of Gainesville, deep discussions spanning physics to robotics, and the co-invention of our surely patentable "Chess-TT." Aditya Subramanian, for surviving the Arctic blizzard night with me. And to my friends in Gainesville—Palash Sethi, Yash Ranjan, Arindam Sarkar, and Shlok Gilda—for making the city feel like home.

Thanks also to the NSF for supporting my work through Grant DUE-1935111, and to the Institute of Education Sciences under Grant 2229612.

To the countless others who helped along the way, more than I could hope to name here—thank you.

Finally, Sakshee: my partner in work and in life. Your unwavering kindness and support made all of this possible. Thank you.

TABLE OF CONTENTS

	<u>page</u>
ACKNOWLEDGEMENTS	4
LIST OF TABLES	11
LIST OF FIGURES	12
ABSTRACT.....	13
CHAPTER	
1 INTRODUCTION.....	15
1.1 Motivation.....	15
1.2 Challenges	17
1.3 Learning Context: Pair Programming	21
1.4 Research Questions	22
1.5 Research Methodology.....	23
1.6 Research Contributions.....	24
1.7 Chapter Roadmap	25
2 RELATED WORK	26
2.1 Foundational Work and Learning Context.....	26
2.1.1 Simulated Learners for Educational Technology Authoring	27
2.1.2 Learning Dialogue and Pair Programming Studies	28
2.2 Agent-Based Modeling and Generative Agent-Based Models	29
2.2.1 Foundations of Agent-Based Modeling	29
2.2.2 Emergence in Agent-Based Modeling	29
2.2.3 Traditional Agent-Based Modeling Approaches	30
2.2.4 Generative Agent-Based Models	31
2.2.5 Calibration Challenges in Agent-Based Modeling	32
2.3 Large Language Models for Behavioral Simulation	32
2.3.1 LLM Roleplaying Agents and Social Simulation	32
2.3.2 Applications of LLMs for Behavioral Simulation	34
2.4 Calibration and Validation Approaches.....	35
2.4.1 Prompt Engineering and Calibration Limitations	35
2.4.2 Validation and Evaluation of Agent Simulations	36
2.5 Planning Abilities and Limitations of Large Language Models	37
2.6 Gaps and Open Questions	40
3 UNDERSTANDING MANUAL AUTHORING: A USABILITY STUDY OF TASK-ORIENTED DIALOGUE AGENT SPECIFICATION INTERFACES	42
3.1 Background.....	43
3.1.1 Accessible Dialogue Agent Authoring	43
3.1.2 Data-Efficient Approaches: Schemas and Instructions.....	44
3.1.3 Data Requirements in Dialogue Agent Development	46

3.1.4	Schema-Based Dialogue	47
3.1.5	Instruction-Based Prompts.....	47
3.1.6	Usability Considerations	48
3.1.7	Research Question and Contribution.....	48
3.2	Study Design and Methodology.....	49
3.2.1	Participants and Recruitment	49
3.2.2	Task Scenarios	49
3.2.3	Authoring Approaches	50
3.2.4	Study Procedure	51
3.2.5	Data Collection Instruments	53
3.2.6	Analysis Approach	53
3.3	Results.....	53
3.3.1	Cognitive Scaffolding: Thinking Through the Details	54
3.3.2	Effort: Time and Cognitive Load	55
3.3.3	Precision and Expressivity: Control vs. Ambiguity	55
3.3.4	Quantitative Results: System Usability Scale	56
3.4	Discussion	56
3.5	Contributions.....	58
3.6	Conclusion.....	59
4	SYSTEMATIC CALIBRATION OF LLM-BASED LEARNER SIMULATIONS: THE HYP-MIX FRAMEWORK	60
4.1	Motivation.....	60
4.1.1	LLM-Based Learner Simulations: Opportunities and Challenges.....	61
4.1.2	Current Calibration Approaches and Limitations	62
4.1.3	Research Objectives and Contributions	64
4.2	The Case for Hypothesis-Based Calibration	64
4.2.1	From Expert Knowledge to Formal Hypotheses.....	64
4.2.2	Compositional Generalization Properties.....	66
4.2.3	Evaluating Calibration: A Statistical Framework	66
4.3	The HYP-MIX Framework	67
4.3.1	Core Abstractions.....	67
4.3.2	Authoring Workflow.....	69
4.3.3	Design Principles.....	72
4.4	Case Study: HoloOrbits Learning Environment	74
4.4.1	Experimental Setup	74
4.4.2	The Learner Model Edit Graph	76
4.4.3	Testing Calibration Under Model Edits	80
4.5	Results.....	81
4.5.1	Calibration Stability Across Edit Operations.....	81
4.5.2	Analysis of LLM Behavioral Modeling Capabilities	84
4.5.3	Limitations of the Current Approach	85
4.6	Discussion and Future Directions	86
4.6.1	Framework Contributions and Achievements.....	87

4.6.2	Limitations and Outstanding Challenges.....	87
4.6.3	Implications for Automated Calibration Methods.....	88
4.7	Contributions.....	89
4.8	Conclusion.....	90
5	EVALUATING BELIEF-BEHAVIOR CONSISTENCY IN LLM-BASED SIMULA- TIONS.....	92
5.1	Motivation.....	93
5.1.1	From Instantaneous Actions to Population-Level Patterns.....	94
5.1.2	Language Models and Population-Level Reasoning.....	94
5.1.3	Existing Validation Methods.....	95
5.1.4	Research Questions and Contributions.....	95
5.2	Belief-Behavior Consistency Framework.....	96
5.2.1	Methodology for Belief Elicitation.....	97
5.2.2	Implications for Planning-Based Calibration.....	97
5.2.3	Population-Level Belief-Behavior Consistency.....	98
5.2.4	Individual-Level Belief-Behavior Consistency.....	99
5.3	Experimental Testbed.....	100
5.3.1	The Trust Game as Testbed.....	100
5.3.2	Game Structure and Simulation Setup.....	101
5.3.3	Trustee Archetypes for Individual-Level Evaluation.....	102
5.3.4	Models Evaluated.....	103
5.4	Results.....	104
5.4.1	Population-Level Consistency.....	104
5.4.2	Belief Elicitation Strategies.....	104
5.4.3	Empirical Results on Consistency.....	105
5.4.4	Magnitude and Direction of Effects.....	107
5.4.5	Implications for Trajectory-Level Constraint Specification.....	108
5.4.6	Controllability Analysis.....	109
5.4.7	Self-Conditioning Approach.....	109
5.4.8	Imposed Prior Approach.....	110
5.4.9	Implications for Planning-Based Calibration.....	110
5.4.10	Individual-Level Consistency.....	112
5.4.11	Multi-Round Forecasting Methodology.....	112
5.4.12	Forecasting Accuracy Over Time Horizons.....	113
5.4.13	Implications for Sequential Decision-Making.....	115
5.4.14	Connection to Planning-Based Calibration.....	116
5.5	Discussion.....	117
5.5.1	Framework Contributions and Methodological Innovations.....	117
5.5.2	Limitations and Outstanding Challenges.....	118
5.5.3	Implications for Automated Calibration Methods.....	119
5.6	Contributions.....	120
5.7	Conclusion.....	121

6	CALIBRATING SYNTHETIC LOGS TO BEHAVIORAL TARGETS.....	123
6.1	Problem Formulation and Evaluation.....	126
6.1.1	Behavioral Targets from Published Aggregates.....	127
6.1.2	Pair Programming Testbed and Evaluation Metrics.....	129
6.1.3	Research Questions.....	135
6.2	Benchmark Construction	136
6.2.1	Target Selection	137
6.2.2	Target Families.....	137
6.2.3	Validation Protocol	140
6.2.4	Validation Results.....	141
6.3	Inference-Time Allocation Methods.....	143
6.4	Experimental Results	145
6.4.1	Learner Pairing Changes Target-Satisfaction Difficulty	146
6.4.2	Locality Predicts Target-Satisfaction Difficulty	149
6.4.3	Locality Predicts Where Planning Helps	151
6.5	Discussion	156
6.5.1	Compute Should Match the Missing Information	157
6.5.2	Validate Targets Before Comparing Methods	157
6.5.3	Implications Beyond Pair Programming.....	158
6.6	Limitations	158
6.7	What This Means in Practice.....	160
6.8	Future Work.....	161
6.9	Conclusion.....	161
	Chapter Appendix.....	163
6.10	Appendix: The Target Pool.....	163
6.10.1	Full Target Pool by Family.....	163
6.10.2	Scoring Rule Groups and Replication Materials.....	169
6.11	Per-Target Satisfaction-Difficulty Records	169
6.12	Persona-Pool Composition Sensitivity	169
6.13	Learner-Pairing Stress Test: Targets and Constructed Pairings.....	171
6.14	Cost Accounting Details	172
6.15	Additional Baselines and Stress Tests.....	172
7	CONCLUSION	174
7.1	Revisiting the Problem	175
7.2	What the Earlier Chapters Established	175
7.3	Answers to the Research Questions.....	176
7.3.1	RQ1: Which test-time compute allocation choices improve target-satisfaction under a finite model-call budget, and at what cost in persona fidelity and model calls?.....	176
7.3.2	RQ2: What makes some targets harder to satisfy than others?	177
7.4	Contributions.....	177
7.5	Limitations	178
7.6	Generalization and Transfer	179

7.7 Scope and Threads Not Covered.....	180
7.8 Future Directions.....	182
7.9 Closing	183
APPENDIX	
A PERSONA POOL.....	184
LIST OF REFERENCES	189
BIOGRAPHICAL SKETCH	207

LIST OF TABLES

<u>Tables</u>	<u>page</u>
4-1 Hypotheses used in learner modeling experiments	78
5-1 Population-level belief-behavior consistency across models and elicitation strategies	106
5-2 Belief conditioning effects on consistency	111
6-1 How the dissertation’s two research questions map onto the analyses in this chapter	136
6-2 The five target families and one representative target from each	138
6-3 Structured load study of the three allocation methods	154
6-4 Planner ablation: persona-aware versus persona-blind planning	154
6-5 Full target statements in the temporal patterns family (10 targets).	163
6-6 Full target statements in the behavior distributions family (10 targets).	164
6-7 Full target statements in the variable relationships family (10 targets).	165
6-8 Full target statements in the inter-session variation family (10 targets).....	166
6-9 Full target statements in the code construct patterns family (10 targets).....	168
6-10 Scoring-rule groups used by the target pool	170
6-11 Released materials for inspecting the benchmark, validation pipeline, and experi- ments.....	170
6-12 Full target-set results.....	171
6-13 Learner-pairing stress test	172
A-1 The 28-learner persona pool used in DUET-Bench	184

LIST OF FIGURES

<u>Figures</u>	<u>page</u>
1-1 Overview of the dissertation and the calibration problem in pair programming.....	18
3-1 A sample snippet of the custom block-based interface for the Procedural schema format	52
3-2 A sample snippet of the custom block-based interface for the Declarative schema format	52
4-1 A screenshot of the HoloOrbits environment	75
4-2 Learner Model Edit Graph used to evaluate LLM robustness across five distinct edit operations	77
4-3 Results of statistical tests evaluating calibration stability across five learner model edit operations	82
5-1 Trust Game: Model consistency across rounds	114
6-1 From a published aggregate to a scored synthetic batch.....	127
6-2 Per-session realism and batch-level fit are different objectives	129
6-3 From a log to a target-satisfaction score.....	134
6-4 The five target families, grouped by where the requested behavior lives.....	139
6-5 Prompting helps many targets but not all.....	142
6-6 A generated session for a joint-distribution target.....	142
6-7 Two levers for spending test-time compute on a batch, and the four generation methods that use them	144
6-8 Persona choice can change how easily a behavioral target is satisfied.....	147
6-9 Compatible versus adversarial learner pairings on eight targets drawn from the pool.....	148
6-10 A generated session under an adversarial pairing	149
6-11 Locality separates easier from harder targets	151
6-12 Allocation-method comparison as the number of requested behaviors increases ...	152
6-13 Planning closes most of the easy-hard gap, where it is needed	155
6-14 Overall simulation quality under fixed versus model-selected pairing at near-matched call budgets	156

Abstract of Dissertation Presented to the Graduate School
of the University of Florida in Partial Fulfillment of the
Requirements for the Degree of Doctor of Philosophy

NEUROSymbolic Authoring and Test-Time Compute Allocation for Generative Educational Simulations

By

Amogh Mannekote

August 2026

Chair: Kristy Elizabeth Boyer

Cochair: Bonnie J. Dorr

Major: Computer Science

Artificial intelligence and educational technology hold great potential for delivering personalized learning support at scale, yet the development and evaluation of these systems face a persistent data-scarcity crisis. Human studies of student interactions are costly and limited, while privacy concerns and ethical restrictions prevent raw behavioral data from being widely shared. This scarcity constrains researchers developing intelligent tutoring systems, collaborative learning platforms, and adaptive learning environments by limiting access to the detailed behavioral data needed to inform design decisions.

Agent-based modeling offers a promising path forward for generating synthetic behavioral data, and recent advances in large language models make it possible to build agents that plan, reason, and roleplay arbitrary personas. Using such agents to stand in for a population requires *calibration*: adjusting a simulation’s controllable inputs so that its aggregate outputs match desired behavioral patterns. Those patterns are expressed as *behavioral targets*, aggregate properties measured across a collection of generated sessions, such as the share of utterances that are help-seeking or the distribution of session outcomes across a batch. Calibration is difficult because large language model outputs are sensitive to how they are specified, and because a model cannot reliably reason on its own about how a specification will translate into population-level behavior.

This dissertation reframes calibration as *explicit target satisfaction* and asks how to allocate test-time compute so that a batch of generated sessions satisfies its behavioral targets under a finite model-call budget. Pair programming serves as the testbed, since classroom studies report well-documented aggregate patterns that supply realistic targets, such as “30% of utterances are help-seeking” or “balanced participation between partners.” The work builds on three earlier studies: a usability study of manual authoring interfaces, the HYP-MIX framework for composable behavioral specifications, and a belief-to-behavior study of how reliably models translate specifications into population-level behavior. The final study introduces DUET-BENCH, an evaluation framework that turns published aggregate findings into auditable behavioral targets, validates them before use, and compares test-time allocation methods that differ in how they spend a model-call budget.

Two research questions organize the final study: which test-time compute allocation choices improve target-satisfaction, and at what cost in the believability of the simulated learners and in model calls; and what makes some behavioral targets harder to satisfy than others. The dissertation makes four contributions: a conceptual framing of calibration as explicit target satisfaction, with the behavioral target as the unit of specification; empirical evidence about when and how large language models can be steered to satisfy behavioral targets; methods and tooling, including the HYP-MIX framework, a belief-to-behavior consistency framework, and the DUET-BENCH benchmark with its target pool, feature-based judges, validation protocol, and allocation methods; and practical guidance for generating trustworthy synthetic educational behavioral logs. Together, these aim to widen access to calibrated synthetic data for educational technology and to support faster, safer, and more rigorous development of adaptive learning systems.

CHAPTER 1

INTRODUCTION

1.1 Motivation

The impracticality of providing individualized human tutoring for each learner [24] has inspired decades of research on how to support learners using new technologies and artificial intelligence [179]. In response to this challenge, artificial intelligence and educational technology have emerged as powerful tools for transforming how we provide personalized learning support to students [153]. These advances have taken many forms: intelligent tutoring systems (Intelligent Tutoring System (ITS)s) that adapt to individual learners [96, 97], collaborative learning platforms that facilitate peer interaction [193], and adaptive learning environments that tailor content to student needs. Together, these systems leverage computational methods to provide tailored educational experiences that would be impossible to deliver at scale through human tutors alone [52].

However, the development and evaluation of these educational technologies are constrained by a persistent data scarcity crisis. The need for detailed behavioral data that captures how learners actually interact with learning environments is difficult to satisfy because human studies of student interactions in real-world classroom settings are both costly and scarce [124, 73, 23, 47]. Privacy concerns and ethical restrictions surrounding underage participants mean that raw behavioral data collected from these studies are rarely open-sourced, creating siloed research projects within participating institutions that limit the broader research community’s ability to develop effective educational technologies [76].

This dissertation approaches this challenge through a sequence of studies that examine facets of the calibration problem, the earlier ones in other domains and the final and most extensive one in pair programming, a collaborative learning activity where two students work together at a single computer to solve coding problems. In pair programming, students alternate between the roles of driver (operating the keyboard and mouse) and navigator (guiding the work, suggesting ideas, and reviewing code). The continuous verbal exchange between learners generates fine-grained learning dialogue that

can be categorized into dialogue acts such as guide, inquiry, lecture, and seek_help [37, 121, 176, 156]. Pair programming serves as an appropriate testbed for this investigation because it provides well-documented behavioral patterns from classroom studies, enabling the extraction of realistic calibration targets such as “30% of utterances are seek_help dialogue acts” or “balanced participation between partners,” and because the fine-grained learning dialogue produced maps directly to collaborative regulation processes that predict educational outcomes. However, the calibration methods developed here are intended to generalize beyond this specific domain to address the broader data scarcity crisis across educational technology research, though it is the method that transfers, with each new domain requiring its own targets, features, and scoring rules.

Agent-Based Modeling (ABM) (computational approaches that simulate complex systems by modeling individual agents and their interactions) offers a promising path forward for generating synthetic behavioral data [172]. In agent-based modeling, autonomous agents follow behavioral rules and interact with each other and their environment, producing emergent population-level patterns that can match observed real-world phenomena [11]. Datasets of synthetic behavioral logs generated through agent-based simulations that distributionally match real data would enable researchers and developers to supplement limited field data, evaluate interventions through rigorous A/B testing, and stress-test learning tasks without repeated expensive deployments [4, 158, 45, 16].

Recent advances in large language models offer a particularly promising solution for agent-based modeling. LLMs-based agents that can plan, reason, and act provide an unprecedented opportunity to generate synthetic behavioral logs, given their ability to understand natural language, roleplay arbitrary personas [143, 145], and reason in novel situations [147, 170]. The LLM-based roleplaying paradigm has shown promise across diverse domains including social simulacra, market research, and experimental economics [104, 59, 98, 78]. Here, *calibration* refers to adjusting the simulation’s controllable inputs so

that aggregate outputs match desired behavioral patterns. These desired patterns can be expressed as *behavioral targets*: aggregate behavioral patterns measured over entire simulation trajectories (e.g., “30% of utterances are help-seeking” or “one partner dominates the interaction”). However, specifying the behavior of LLM-based agents to produce desired emergent outcomes remains fundamentally challenging. I situate LLM-based agents within the broader paradigm of agent-based modeling [172, 143], where calibration represents a central challenge [18, 207, 191, 21]. This challenge is particularly acute because LLM outputs are notoriously sensitive to specification variations [163, 118, 133], making reliable calibration difficult. A critical gap remains in how practitioners can reliably specify LLM-based agent behavior to ensure that simulated outputs match behavioral priors in a principled, verifiable way.

1.2 Challenges

Despite the promise of LLM-based agents for synthetic data generation, calibrating these agents to reliably satisfy desired behavioral priors remains difficult. Current approaches face three core limitations. First, there is no principled, verifiable way to ensure that natural-language specifications produce trajectories aligned with intended behavioral priors; second, calibration is brittle and typically requires costly trial-and-error iteration; and third, manual calibration practices do not scale with task complexity or the number of constraints. For example, in a pair programming simulation, a researcher might specify that one partner should take a strongly dominant role (driving most turns, initiating most decisions, and rarely asking for help), while the other should mostly respond. Yet the realized trajectory can drift toward near-equal participation and a more symmetric dialogue pattern (e.g., both partners alternating turns and asking questions at similar rates), even when the instruction is explicit. The difficulty is that these are batch-level properties, emerging from many local, turn-by-turn choices across many sessions and hard to enforce through prompting alone without explicit tracking and correction during generation.

These challenges make calibration a central bottleneck for using LLM-based agents to

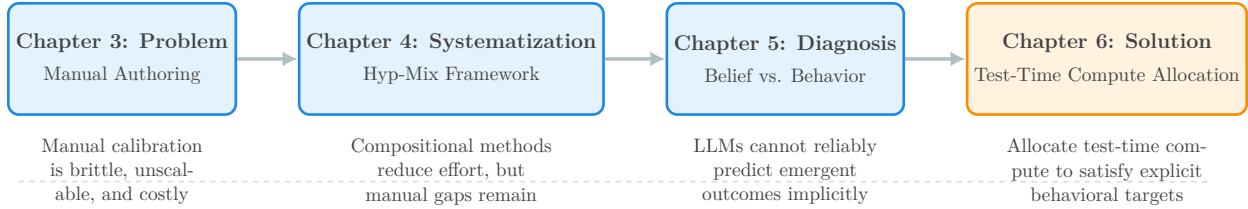


Figure 1-1. Overview of the dissertation and the calibration problem in pair programming.

generate trustworthy synthetic behavioral data. This dissertation addresses the bottleneck through a progression of empirical investigations that examine manual authoring limitations, develop systematic calibration frameworks, validate underlying assumptions, and ultimately motivate the allocation of test-time compute to satisfy explicit behavioral targets. The three earlier chapters present previously published studies, each conducted in its own domain rather than in pair programming. They are gathered here not because they share a domain but because each examines a different facet of a single question, how to calibrate an LLM-based behavioral simulation: the cost of authoring a specification, whether a specification composes, and whether a model can anticipate the behavior a specification produces. Because each study was carried out on its own terms, their domains differ, and this dissertation does not treat the differences across them as a controlled cross-domain comparison; the final study consolidates the thread in pair programming, whose published aggregate patterns are rich enough to supply realistic behavioral targets. As presented in Chapter 3, a controlled usability study of three approaches to authoring task-oriented dialogue agents (instruction-based prompts, procedural schemas, and declarative schemas) established that manual authoring faces severe scalability challenges regardless of interface choice. The empirical findings revealed that cognitive load, effort, and precision all degrade with task complexity, with researchers spending 40 hours iteratively adjusting specifications to achieve target behavioral patterns. The authoring bottleneck in educational technology is well-documented, with traditional approaches requiring substantial manual effort and some systems requiring over 100 hours of expert time per instructional hour [137, 95, 5, 88].

Before taking up the test-time compute allocation studied in Chapter 6, it is helpful to answer a simpler, more foundational question, namely whether LLMs can maintain behavioral relationships across learner model edits when given appropriate specifications. I address this question in Chapter 4 by developing a systematic framework for compositional calibration that enables “calibrate once, use many” authoring practices. The HYP-MIX framework treats expert behavioral priors as modular, testable units, demonstrating, in the HoloOrbits learning environment, that Generative Pre-trained Transformer (GPT)-4 Turbo maintains calibration across 89% of learner-model modifications. This foundational result establishes that compositional generalization is achievable with current LLM technology, providing necessary evidence that automated calibration systems can leverage reusable behavioral specifications. However, this framework still requires substantial manual effort for template calibration, with no guarantee of success. The persistence of manual calibration overhead reveals that systematization alone is insufficient for truly scalable simulation authoring, motivating the need for fully automated approaches.

In Chapter 5, using an economic Trust Game as the testbed, I investigate a fundamental assumption underlying automated approaches, namely that LLMs can reliably reason about the relationship between agent specifications and emergent population-level outcomes. The systematic inconsistencies discovered regarding effect sizes, controllability, and forecasting accuracy reveal fundamental limitations in LLMs’ ability to predict emergent outcomes through implicit reasoning alone. These findings validate the need for explicit target specifications and formal target-checking mechanisms rather than relying on implicit LLM reasoning.

This progression of investigations converges on a solution architecture in which, rather than relying on manual calibration or implicit LLM reasoning about emergent patterns, we frame calibration as explicit target satisfaction. The simplest version of such a target is a *trajectory-level constraint*, an aggregate property of a single simulated rollout (for example, “the student attempts at least three distinct fixes before asking for help in

this session”). The dissertation’s primary focus is the corpus-wide form of the same idea, where **behavioral targets** are aggregate behavioral patterns measured across a collection of generated sessions (for example, “30% of utterances across the batch are help-seeking” or “the batch distribution over outcome categories matches the published cohort distribution”). Experts specify these behavioral targets declaratively, and the simulator allocates test-time compute to satisfy them through some combination of target-compatible selection, persona-aware allocation, sampling and filtering, and planner-guided generation. This approach uses LLMs for their strengths in generation, world knowledge, and multi-step reasoning while compensating for their weaknesses through formal target-checking mechanisms.

Current calibration approaches face several critical limitations that illustrate the urgency of developing automated calibration methods. A researcher might carefully calibrate an agent specification to produce a “high persistence learner,” but when adding a “low geometry proficiency” characteristic, the specification produces unexpected help-seeking patterns. An LLM asked to predict how increasing agent conscientiousness will affect explanation episodes might predict 30%, but the actual simulation shows only 15% due to complex interactions between attributes that the model failed to account for. A researcher spending 40 hours iteratively adjusting specifications to achieve a target participation balance finds that adding a new constraint necessitates complete recalibration, invalidating previous work.

Addressing this challenge is urgent because reliable agent specification enables critical capabilities that are currently blocked by data scarcity and manual calibration overhead. Synthetic behavioral data generation would allow researchers to supplement limited field data, evaluate interventions through rigorous A/B testing, and stress-test learning tasks without repeated expensive deployments. Current manual approaches do not scale to the complexity of specifying and verifying multidimensional behavioral patterns across diverse populations, blocking the development of educational technologies. The rapid advancement

of LLM capabilities, combined with increasing demand for personalized educational technologies, makes this an opportune moment to develop systematic, automated calibration methods that can keep pace with the complexity of modern learning environments.

1.3 Learning Context: Pair Programming

The dissertation’s final study investigates agent calibration within the domain of pair programming, a collaborative learning activity where two students work together at a single computer to solve coding problems. In pair programming, students alternate between the roles of driver (operating the keyboard and mouse) and navigator (guiding the work, suggesting ideas, and reviewing code). The continuous verbal exchange between learners generates fine-grained learning dialogue that can be categorized into dialogue acts such as guide, inquiry, lecture, and seek_help [37, 121, 176, 156].

Pair programming serves as an appropriate testbed for several reasons. First, it provides well-documented behavioral patterns from classroom studies, enabling the extraction of realistic calibration targets such as “30% of utterances are seek_help dialogue acts” or “balanced participation between partners.” These measurable behavioral patterns provide clear aggregate constraints that can be used to test whether multi-agent systems can be steered toward reproducing specified emergent statistical properties. Second, pair programming represents broader collaborative learning challenges, including role coordination, knowledge co-construction, and collaborative regulation of learning processes [37]. Third, the fine-grained learning dialogue produced by dyads maps directly to collaborative regulation processes that predict educational outcomes, making the calibration problem both practically relevant and methodologically tractable.

Empirical pair programming literature supplies dialogue-act taxonomies, conditional frequencies, and aggregate correlations with learning outcomes that can serve as retrospective calibration targets when raw transcripts are unavailable [37, 121, 176]. Learning dialogue demonstrates strong links to educational outcomes, in that collaborative

regulation behaviors such as negotiating strategies, offering tutoring, and maintaining balanced participation correlate with higher self-efficacy, deeper conceptual understanding, and improved task performance [156]. This context provides the testbed for investigating whether LLM-based agents can be calibrated in-context to match specified aggregates, enabling synthetic logs to become a reusable substitute for limited field data while reducing the time and expert effort required by manual ABM calibration.

1.4 Research Questions

This work addresses two research questions about how to calibrate LLM-based simulations to published aggregate statistics under a finite model-call budget. Each is addressed through controlled empirical investigations in pair-programming simulations, where even explicit specifications can fail to produce the intended aggregate behavior.

RQ1: Which test-time compute allocation choices improve target-satisfaction under a finite model-call budget, and at what cost in persona fidelity and model calls?

Once a target set is fixed and the generator is held constant, model calls can be spent in several places, by selecting target-compatible personas before generation, allocating personas with target information, sampling and filtering candidate sessions during generation, or running planner-guided generation. Higher target-satisfaction can come at the expense of persona consistency or a substantially larger model-call budget, so RQ1 compares these options on the same target sets and reports not only which choices help and by how much, but how each behaves across target-satisfaction, persona fidelity, and model-call cost, so that an apparent winner on one metric is not confounded with a loss on another. Chapter 6 reports modular studies that together populate this comparison and shows that the right allocation depends on which trade-off the practitioner is willing to make.

RQ2: What makes some targets harder to satisfy than others?

Target difficulty is not uniform, since some targets are easier than others and

combinations of targets within the same batch can interfere in ways that are not predictable from the individual targets alone. RQ2 asks where that difficulty comes from, across target identity, target combinations, and the number of simultaneous targets per batch. Chapter 6 addresses this through a five-category target taxonomy (temporal patterns, behavior distributions, variable relationships, inter-session variation, and code-construct patterns) and structured multi-target stress tests at $k = 1$ through $k = 4$.

1.5 Research Methodology

This dissertation addresses the calibration challenge as a *test-time compute allocation* problem. The generator (an LLM) and the target specification are held fixed, and only the placement of model calls in producing each batch varies. As introduced in Section 1.1, ABM offers a promising path forward for generating synthetic behavioral data, and recent advances in LLMs make it possible to build agents that can plan, reason, and roleplay arbitrary personas. The core insight is that experts can specify behavioral targets declaratively (e.g., “30% of utterances are SEEK_HELP dialogue acts”), and the simulator can then choose among several test-time compute allocation strategies (selecting target-compatible personas, allocating personas with target information, sampling and filtering candidate sessions, or running planner-guided generation) to satisfy them under a finite call budget.

My preliminary investigations provide the empirical foundation for this approach. The controlled usability study in Chapter 3 established that manual authoring faces severe scalability challenges, with cognitive load and effort degrading as task complexity increases. The HYP-MIX framework in Chapter 4 demonstrated that compositional generalization is achievable (GPT-4 Turbo maintains calibration across 89% of learner model modifications), providing evidence that automated systems can leverage reusable behavioral specifications. The belief-behavior consistency study in Chapter 5 revealed systematic limitations in LLMs’ ability to predict emergent outcomes through implicit reasoning, validating the need for explicit target specifications and formal target-checking mechanisms.

Building on these foundations, the final empirical study in Chapter 6 conducts controlled experiments in pair programming simulation that systematically vary target identity, target-set composition, target load (singleton through four-target sets), generation method (target-compatible selection, persona-aware allocation, sampling and filtering, planner-generation), and model-call budget. Behavioral targets are extracted from published pair-programming studies to serve as realistic calibration targets. For each experimental condition, the study generates batches of simulated sessions and evaluates target-satisfaction score, persona fidelity, and model-call cost.

This methodology enables systematic investigation of how a finite model-call budget should be allocated to satisfy explicit behavioral targets, bridging the gap between the theoretical promise of LLM-based simulation and the practical requirements of reliable synthetic data generation for educational technology research.

Answering these two questions requires an environment in which calibration can be measured and compared: a fixed pool of behavioral targets, judges that score target-satisfaction from the logs, and an accounting of persona fidelity and the model calls each method spends. The final study builds that environment, **DUET-Bench**, and uses it as the setting in which the dissertation’s two research questions are investigated.

1.6 Research Contributions

This dissertation makes four contributions:

1. **Theoretical framing:** a theoretical framing of behavioral targets as a specification language for LLM-based simulations.
2. **Empirical evidence:** empirical evidence about when and how LLMs can satisfy behavioral targets.
3. **Methods and tooling:** methods and tooling that reduce calibration effort from weeks to hours or days.

4. **Practical guidance:** guidance for generating trustworthy synthetic educational behavioral logs.

1.7 Chapter Roadmap

The remainder of this dissertation includes a literature review (Chapter 2), my previously published works that inspire and inform the empirical study (Chapters 3, 4, and 5), and the final empirical study and benchmark (Chapter 6). In Chapter 2, I review simulated learners, LLM roleplaying agents and social simulation, calibration in agent-based modeling, planning abilities of large language models, pair programming studies, and validation of agent simulations; in Chapter 3, I present my controlled usability study of three agent authoring approaches (instruction-based prompts, procedural schemas, and declarative schemas), establishing baseline manual authoring challenges that motivate automation; in Chapter 4, I present HYP-MIX, my systematic framework for compositional calibration that enables “calibrate once, use many” authoring practices, while highlighting remaining manual calibration overhead; in Chapter 5, I introduce my belief-behavior consistency framework, showing systematic limitations in LLMs’ reasoning about emergent outcomes and motivating explicit constraint checking; and in Chapter 6, I present **DUET-Bench**, the evaluation framework through which the dissertation’s two research questions are investigated, comprising a pool of behavioral targets, feature-based judges, and three modular experiment suites that compare test-time allocation methods for satisfying explicit behavioral targets in pair-programming simulation. For committee members short on time, you may read the introduction (Chapter 1) first, then proceed directly to the proposed work (Chapter 6); the literature review (Chapter 2) can be skimmed if you are already familiar with simulated learners, LLM-based behavioral modeling, and agent-based model calibration, and Chapters 3, 4, and 5 describe already-published studies (you may wish to read the “Contributions” and “Conclusion” sections at the end of each chapter).

CHAPTER 2 RELATED WORK

This chapter reviews work relevant to LLM-based behavioral simulation with behavioral target calibration. Section 2.1 covers simulated learners and pair programming, identifying the authoring bottleneck that motivates synthetic data generation and the use of published aggregate statistics as calibration targets when raw transcripts are unavailable. Pair programming involves two interacting learners and emergent group behaviors, making agent-based modeling a natural framework.

Section 2.2 reviews agent-based modeling and LLM-powered generative agent-based models, covering bottom-up emergence and the calibration challenge of aligning micro-level behaviors to produce desired macro-level patterns. Section 2.3 examines LLM use in behavioral simulation, including roleplaying agents and social simulation applications. Section 2.4 reviews calibration and validation methods, showing that manual calibration and instance-level evaluation are insufficient and motivating automated, distributional approaches.

Section 2.5 examines LLM planning abilities and limitations, particularly for long-horizon tasks and behavioral target satisfaction, identifying fundamental challenges that motivate this dissertation’s approach. Section 2.6 synthesizes these gaps, showing how they converge on the need for automated, target-based calibration that addresses the authoring bottleneck while ensuring systematic validation.

2.1 Foundational Work and Learning Context

This section reviews foundational work that establishes the problem space and learning context for this dissertation’s approach. We begin with simulated learners for educational technology, which reveals the authoring bottleneck that motivates synthetic data generation, then examine pair programming as the learning context that provides calibration targets for that approach.

2.1.1 Simulated Learners for Educational Technology Authoring

Educational technology development faces an authoring bottleneck requiring high manual effort to formalize domain knowledge and create tutoring rules [95, 5, 22]. Alevan et al. (2015) [5] reported that some intelligent tutoring systems require over 100 hours of expert time per instructional hour. This effort involves iterative testing cycles requiring diverse learner interactions, motivating simulated learners that automatically generate behavior for testing and refinement.

Matsuda et al. (2016) [127] developed SimStudent, learning cognitive models by demonstration [97] to reduce authoring effort. SimStudent models memory, knowledge representation, and skill acquisition through production rule learning grounded in Adaptive Control of Thought-Rational (ACT-R) architectures [155]. Matsuda et al. (2015) [126] demonstrated its use for interactive tutoring, allowing developers to test systems before deployment. However, Käser & Alexandron (2023) [102] note that SimStudent and similar approaches focus primarily on cognitive aspects, leaving affective and social dimensions unaddressed.

Traditional simulated learners face fundamental limitations. They emphasize cognitive and knowledge aspects while learning sciences emphasize broader dimensions: sociocultural perspectives viewing learning as social participation [181], affective dimensions including emotions and motivation [91], situative perspectives emphasizing context and identity [194], and self-regulation involving metacognition and persistence [162]. Christensen et al. (2011) [42] attempted to simulate psychological aspects, but their handcrafted method is context-specific and does not scale. Kaeser-Favero et al. (2025) [88] note that formalizing knowledge, designing production rules, and handling psychological aspects requires months of expert work per domain. Käser & Alexandron (2023) [102] identified widespread validation gaps, with almost half of studies providing no evidence that models address fundamental validity questions.

These limitations motivate whole learner modeling [125] that captures social,

affective, and motivational characteristics beyond academic proficiency, including non-cognitive skills like persistence and growth mindset [33]. Traditional simulations cannot achieve both low authoring cost and rich behavioral realism simultaneously, creating an opportunity for LLM-based approaches that leverage world knowledge while reducing manual specification overhead. Mannekote et al. (2025) [123] note that open-ended interactive learning environments present particular challenges due to large state spaces and emphasis on affective scaffolding, making them ideal testbeds for LLM-based simulation approaches.

2.1.2 Learning Dialogue and Pair Programming Studies

Pair programming involves two students working together at a single computer, alternating between driver (operating keyboard and mouse) and navigator (guiding work, suggesting ideas, reviewing code) roles. Continuous verbal exchange reveals collaborative regulation of learning: how students plan, monitor, and adapt their work together. Empirical pair programming literature supplies dialogue-act taxonomies, conditional frequencies, and aggregate correlations with learning outcomes that serve as retrospective calibration targets when raw transcripts are unavailable [76].

Learning dialogue links strongly to educational outcomes. Collaborative regulation behaviors (negotiating strategies, offering tutoring, maintaining balanced participation) correlate with higher self-efficacy, deeper conceptual understanding, and improved task performance [154, 70, 106].

Classroom research on pair programming faces two limitations. First, datasets are small due to logistical constraints and privacy considerations. Second, studies report qualitative descriptions or aggregate statistics rather than full interaction logs. Institutional review board restrictions prevent publishing complete datasets, leaving only macro-level statistics and selected examples. As a result, calibration targets must be specified using aggregate behavioral targets rather than matching individual trajectories. Fine-grained behavioral logs recording each utterance, turn, or code edit are critical for

evaluating interventions, yet remain inaccessible [90, 71]. This data scarcity motivates target-based calibration: since only published aggregate statistics are available as calibration targets (e.g., “30% of utterances are seek_help dialogue acts”), the system must satisfy aggregate-level targets rather than matching individual trajectories.

2.2 Agent-Based Modeling and Generative Agent-Based Models

Agent-based modeling (ABM) offers a path forward for generating synthetic behavioral data to address data scarcity in educational technology [172, 4, 158, 45, 16]. In ABM, autonomous agents follow behavioral rules and interact with each other and their environment, producing emergent population-level patterns matching observed real-world phenomena [11].

2.2.1 Foundations of Agent-Based Modeling

Agent-based modeling simulates complex systems by modeling individual agents and their interactions [172, 11]. Unlike equation-based models describing systems at the aggregate level, ABMs operate at the micro-level, where agents with simple behavioral rules interact locally, producing macro-level patterns that emerge from these interactions [53]. This bottom-up approach suits modeling social systems, where individual decisions and interactions lead to collective outcomes difficult to predict from aggregate equations alone.

ABM captures heterogeneity, local interactions, and adaptive behavior characterizing real-world systems [27]. Agents can have diverse characteristics, follow different rules, and adapt behavior based on local conditions and interactions with other agents. This flexibility enables modeling complex phenomena such as opinion dynamics, market behavior, disease spread, and social learning processes [45].

2.2.2 Emergence in Agent-Based Modeling

A central concept in ABM is *emergence*: macro-level patterns arising from micro-level agent interactions without being explicitly programmed at the aggregate level [53, 38]. Emergent properties are system-level outcomes not directly intended or controlled by individual agents but resulting from complex interactions between agents and their

environment. Cenani et al. (2021) [38] demonstrated this in simulations of workers’ decisions to work from home during COVID-19, where individual decisions led to higher survival rates due to reduced hospital overcrowding, an outcome emerging from aggregate individual choices rather than being directly specified.

In behavioral simulation for educational technology, this study repurposes emergence to refer to aggregate outcomes reported in research publications. These aggregate-level outcomes (such as “30% of utterances are seek_help dialogue acts” or “balanced participation between partners”) emerge from sequences of individual actions and interactions within simulation trajectories, positioning calibration as ensuring that agent behaviors produce desired emergent patterns at the aggregate level.

2.2.3 Traditional Agent-Based Modeling Approaches

Traditional ABM approaches rely on rule-based or equation-based specifications of agent behavior [27, 46]. Agents follow predefined rules determining actions based on internal state, environmental conditions, and interactions with other agents. These rules are handcrafted by domain experts, requiring substantial manual effort to specify and tune [207, 191, 21].

Traditional ABM calibration involves manually tuning micro-level parameters until simulated outputs match observed macro-level aggregates [18, 207, 191, 21]. This process is labor intensive, brittle to environment changes, and becomes intractable as parameter spaces grow. As the number of agent characteristics, behavioral rules, and interaction patterns increases, combinatorial explosion makes exhaustive search infeasible, forcing practitioners to rely on heuristic search or manual tuning. This limitation motivates reformulating calibration as an automated target-satisfaction problem where behavioral targets are specified declaratively and the simulator spends test-time compute (through independent sampling, sampling and selection among candidate outputs, and planner-guided generation with or without persona information) to satisfy them.

Traditional ABM approaches face challenges modeling complex human behavior

involving reasoning, social interaction, and context-dependent decision-making. Specifying behavioral rules capturing human cognition and social dynamics requires extensive domain knowledge and often results in oversimplified models failing to capture important nuances [183]. This limitation becomes acute in educational contexts, where learner behavior involves cognitive, affective, social, and motivational dimensions difficult to formalize through simple rules [125].

2.2.4 Generative Agent-Based Models

Recent advances in large language models have enabled Generative Agent Based Models (Generative Agent-Based Models (GABM)s) [180]. GABMs use LLMs to generate agent behavior, leveraging extensive training on human-generated text to simulate human-like reasoning, decision-making, and social interaction [143, 59]. This approach situates LLM-based agents within the broader ABM paradigm [172, 143], where agents understand natural language, roleplay arbitrary personas, and reason in novel situations [147, 170].

GABMs offer advantages over traditional ABM approaches. They leverage LLMs’ world knowledge about human behavior, reducing manual specification of behavioral rules [143, 3]. They generate contextually appropriate responses to novel situations, enabling flexible and realistic simulations [147]. They capture complex behavioral patterns difficult to formalize through rules, including social dynamics, affective responses, and nuanced decision-making processes [123].

However, GABMs inherit challenges from both ABM and LLM research. The calibration challenge remains central: how can practitioners reliably specify LLM-based agent behavior to ensure simulated outputs match behavioral priors in a principled, verifiable way? This challenge is acute because LLM outputs are sensitive to specification variations [163, 118, 133], making reliable calibration difficult. Additionally, LLMs encode accurate knowledge internally but often apply it inconsistently across contexts [141], raising questions about reliability for generating synthetic behavioral data. This

inconsistency between encoded knowledge and applied behavior motivates explicit target specifications rather than relying on implicit LLM reasoning about emergent patterns, as demonstrated by belief-behavior inconsistencies in Chapter 5.

2.2.5 Calibration Challenges in Agent-Based Modeling

Calibration refers to adjusting simulation or agent parameters to satisfy desired behavioral specifications. In traditional ABM workflows, this process requires manual parameter tuning until simulated outputs match aggregate behavioral patterns—a labor-intensive approach that is brittle to environment changes and becomes intractable as parameter spaces grow. ABM calibration is closely related to verification and validation (Verification and Validation (V&V)) [207, 191, 21]: verification ensures correct implementation, while validation ensures the model accurately represents the real-world system [14]. The relationship between the number and nature of behavioral targets and LLMs’ ability to reproduce them is of interest for practitioners, since calibration has traditionally involved manual, error-prone processes expensive to implement and maintain. Section 2.4 examines calibration approaches for GABMs in detail, including the challenges of reformulating calibration as an in-context steering problem and addressing behavioral target satisfaction.

2.3 Large Language Models for Behavioral Simulation

This section reviews methodological approaches using LLMs for behavioral simulation, examining their capabilities and limitations that inform the target-based calibration approach. We review roleplaying agents and social simulation applications.

2.3.1 LLM Roleplaying Agents and Social Simulation

Large language models have emerged as tools for behavioral simulation. Foundational models like T5 [150] and GPT-3 [31] significantly streamlined dialogue system development, enabling new approaches to conversational agent authoring. These advances have supported both structured dialogue schemas [26, 152, 136, 39, 138, 113, 110, 128] and instruction-based prompting [132, 115] for zero-shot and few-shot dialogue tasks

[189, 148, 35, 157]. Park et al. (2023) [143] demonstrated their ability to roleplay diverse personas and generate human-like social interactions in a small-scale virtual town. Park et al. (2024) [146] extended this to simulations of 1,000 people, demonstrating scalability. LLM-based roleplaying agents use large language models to simulate human-like behavior, memory, and social interactions across domains including social media platform design [145], market research [28, 98, 78], and experimental economics [66].

Prior studies instantiate LLMs as autonomous conversational agents to simulate social interactions and test dialogue plausibility at small scales. Existing work emphasizes single-trajectory realism and agent believability rather than explicit calibration to published aggregate statistics [68], creating an opportunity to recast prompting and orchestration as calibration variables. Argyle et al. (2023) [9] assessed whether survey response patterns generated by LLMs match real-world surveys. Huang et al. (2024) [81] created TRUSTSIM, which asks agents to self-report beliefs and checks alignment with free-form responses. Wang et al. (2023) and Bhandari et al. (2025) [188, 20] focus on whether personality cues in dialogue match expected attributes, but these evaluations are post-hoc and do not address calibration to aggregate statistics. This gap between single-trajectory realism and aggregate-level calibration motivates the research questions in Chapter 6: which test-time compute allocation choices improve target-satisfaction under a finite model-call budget, and at what cost in persona fidelity and model calls (**RQ1**); and how target-set satisfiability varies with target composition and target load (**RQ2**).

Research has examined LLM-based roleplaying agents across social simulation scenarios [184, 166, 117, 187, 200], yet their ability to satisfy expected behavioral targets under controlled allocation of test-time compute remains underexplored. Prior LLM-based agent-based modeling techniques underutilize LLMs’ capacity for in-context reasoning and planning during the calibration phase. While LLMs can plan effectively in individual tasks, using them to satisfy behavioral targets within multi-agent interactions represents an underexplored area that could automate ABM calibration. Section 2.5 examines LLM

planning capabilities and limitations in detail.

2.3.2 Applications of LLMs for Behavioral Simulation

LLM-based behavioral simulation spans diverse domains beyond education, including social simulacra, market research [98, 78], experimental economics [66], computational social science, and policy research [161, 15]. These applications share a premise: LLMs’ extensive training on human-generated text may enable simulating human-like reasoning, decision-making, and social interaction when prompted to roleplay specific personas or situations. Aher et al. (2023) [3] demonstrated that LLMs accurately predict human responses to natural language descriptions of hypothetical situations and actual experimental setups from psychology and behavioral economics.

Dasgupta et al. (2022) and Ozeki et al. (2024) [49, 142] showed that LLMs reproduce known cognitive biases in reasoning tasks. Frisch & Giulianelli (2024) and Jiang et al. (2024) [57, 85] demonstrated that LLMs maintain consistency with specified personality traits across extended interactions. Xiao et al. (2023) [196] found they generate plausible responses to scenarios from psychology and economics experiments. These capabilities suggest LLMs could serve as “silicon participants” for rapid prototyping, testing hypotheses before expensive human data collection, or exploring counterfactuals impossible to study with real humans. Large-scale applications like VACSIM model emergent social phenomena such as opinion dynamics in social networks [79], demonstrating potential impact of LLM-based role-play in influencing real-life policy decisions.

However, enthusiasm must be tempered by concerns. Loya et al. (2023) and Sclar et al. (2023) [118, 163] demonstrated that LLMs are sensitive to minor prompt variations: synonyms, reordering, even punctuation changes substantially alter outputs. Mohammadi (2024) [133] highlighted that this sensitivity makes prompt design critical. This sensitivity raises questions about whether apparent simulation success reflects genuine behavioral understanding or fortuitous alignment with specific phrasing choices. Second, evaluation practices often rely on subjective judgments creating opportunities for Clever Hans effects

where expected outcomes are unconsciously cued through prompt design [86]. Third, LLMs trained on vast web data may rely on memorization rather than genuine reasoning, limiting generalization [160]. Critical gaps remain in generalization, validation, and systematic evaluation [7, 158, 2].

2.4 Calibration and Validation Approaches

This section examines calibration and validation approaches for agent-based models and LLM-based simulations, revealing limitations motivating automated, distributional approaches.

2.4.1 Prompt Engineering and Calibration Limitations

Current approaches to LLM-based behavioral simulation rely heavily on manual prompt engineering, where practitioners iteratively refine natural language instructions until desired behaviors emerge. However, LLMs’ sensitivity to prompt variations [118, 163] poses fundamental challenges for reliable behavioral modeling. Changing the order of in-context examples, swapping synonymous words, or adjusting whitespace produces dramatically different output distributions. Zamfirescu-Pereira et al. (2023) [205] documented how non-AI experts struggle to design effective LLM prompts, highlighting the expertise barrier.

The calibration challenge in GABMs synthesizes challenges from both traditional ABM and LLM research. As discussed in Section 2.2, GABMs face the labor-intensive manual tuning required by traditional ABM calibration while also contending with LLMs’ sensitivity to prompt variations and inconsistent application of knowledge across contexts. Reformulating calibration as an in-context steering problem for GABMs positions prompt structure, target encoding, and orchestration as calibration parameters to be empirically compared against traditional methods. However, this approach requires addressing fundamental challenges in how LLMs can be directed to satisfy behavioral targets that emerge from sequences of actions rather than being directly specified at each step. A related challenge is controllability: whether practitioners can reliably override an LLM’s

default beliefs or behaviors to achieve desired simulation outcomes [169], which becomes critical when simulations must align with specific theoretical expectations or empirical findings.

Prompt calibration approaches have emerged focusing on reducing LLMs’ sensitivity to the order of in-context examples [120, 208]. However, these methods reduce variance between templates rather than ensuring consistency across simulation contexts. Chain-of-thought reasoning [192] and systematic prompt optimization [203] can improve consistency but still require extensive manual effort and domain expertise. The compositionality challenge in prompting [89] becomes acute when specifying complex behavioral patterns emerging from interactions of multiple characteristics. This challenge is compounded by the need for compositional generalization [122, 204], where modular elements must be added, edited, removed, swapped, and combined to shape LLM outputs.

These limitations motivate systematic approaches that reduce dependence on manual prompt crafting while providing stronger guarantees about behavioral consistency and target satisfaction. Recent work has explored knowledge editing [185, 141] and inference-time steering [109, 105, 131] as alternatives to prompt engineering, but their applicability to behavioral simulation remains underexplored.

2.4.2 Validation and Evaluation of Agent Simulations

Evaluating simulated behavioral data quality presents methodological challenges. Traditional approaches rely on crowdsourced human evaluation [177], but these methods face limitations including cost, required domain expertise, and demographic mismatch between evaluators and target populations [100]. Park et al. (2023) and Jiang et al. (2024) [144, 85] used crowdworkers to rate realism of generated behaviors, but Huff et al. (2015) [82] noted that typical crowdworkers lack deep involvement in education, making them ill-equipped to assess behaviors of young learners with specific characteristics.

Turing-like tests for simulated learners [171, 102] assess whether human experts can distinguish synthetic from real behavioral data, but such tests are expensive and may miss

systematic biases apparent only through statistical analysis. Statistical validation methods [65] offer more scalable approaches by comparing distributional properties of synthetic and real data.

Falsifiable hypotheses in evaluation [60] are important for behavioral simulation, where the goal is systematic reproduction of known patterns rather than just plausibility. This motivates shifting from instance-level evaluation (“does this individual trajectory look realistic?”) to distributional evaluation (“do aggregate patterns match empirical findings?”) [135]. Mannekote et al. (2025) [123] addressed this by centering on falsifiable hypotheses for both authoring and evaluation, using marginal distributional hypotheses specifying statistical relationships between learner characteristics and behaviors. However, Orgad et al. (2024) [141] demonstrate that LLMs encode accurate knowledge internally but often apply it inconsistently across contexts, making validation challenging. The limitations of instance-level evaluation (susceptibility to Clever Hans effects and inability to detect systematic biases) motivate this dissertation’s use of distributional evaluation, where target satisfaction is assessed by comparing aggregate patterns (e.g., percentage of dialogue acts, participation balance) against published empirical findings rather than relying on subjective judgments of individual trajectory realism.

2.5 Planning Abilities and Limitations of Large Language Models

Planning involves selecting a sequence of actions to achieve a goal state, requiring reasoning about future consequences of current decisions and maintaining coherence across multiple steps. In classical planning, agents operate in well-defined state spaces with explicit preconditions and effects, enabling systematic search for action sequences that transform initial states into goal states [69]. However, behavioral simulation requires planning in more open-ended environments where goals may be behavioral targets rather than discrete end states, and where action effects are probabilistic and context-dependent.

Research demonstrates that LLMs can perform multi-step planning and generate intermediate reasoning traces [129, 51, 198]. Mechanistic interpretability studies reveal that

LLMs encode planning representations beyond immediate next-token prediction. Men et al. (2024) [129] identified look-ahead planning mechanisms through linear probes, showing that LLMs internally represent future action sequences before generating outputs. Dong et al. (2025) [51] documented emergent response planning detectable via mechanistic analysis, providing evidence that LLMs engage in multi-step reasoning processes during generation. These findings suggest LLMs can reason about action sequences and their consequences, not merely generate text token-by-token.

Studies have examined LLM performance on constraint-based planning tasks like BlocksWorld [69] and TravelPlanner [198], where success means achieving desired end states. These studies focus on instance-level goals: achieving particular end states for individual problem instances. However, Valmeekam et al. (2023) [178] conducted a systematic investigation revealing fundamental weaknesses in LLM planning abilities. Their analysis across multiple planning domains showed that LLMs struggle with maintaining plan consistency over long horizons, often generating sequences that violate preconditions or fail to achieve stated goals. Momennejad et al. (2023) [134] identified specific failure modes: LLMs generate sequences with logical inconsistencies, fail to maintain coherent state representations across steps, and struggle with flexible reasoning when problem structure differs from training examples.

Multi-step reasoning capabilities are central to planning. Laat et al. (2025) [149] surveyed multi-step reasoning in LLMs, finding that while models can perform logical reasoning, solve word problems, and engage in planning tasks, their capabilities degrade significantly with sequence length. Chain-of-thought prompting [192] and related techniques improve reasoning by encouraging step-by-step decomposition, but fundamental limitations remain for long-horizon tasks requiring maintaining coherence across many steps.

Cao et al. (2025) [36] surveyed the planning literature and identified persistent challenges: error accumulation in long sequences, poor generalization across domains, and

difficulty handling uncertainty and partial observability. Arbuzov et al. (2025) [8] demonstrated that error accumulation follows complex patterns beyond simple exponential decay, with failures compounding non-linearly as sequence length increases. As planning horizons extend, LLMs struggle to maintain consistent state representations, leading to logical inconsistencies and plan failures. Valmeekam et al. (2023) [178] found that LLMs often generate plans that violate preconditions or fail to achieve stated goals, particularly when plans require more than a few steps. These findings suggest that while LLMs show promise for short-horizon planning, robust performance on long-horizon tasks remains an unsolved challenge.

Given these limitations, understanding how LLM planning performance varies with problem characteristics (target types, planning horizon length, number of simultaneous targets) becomes critical for determining when automated target-based calibration is feasible, motivating empirical investigation of target satisfaction ability across different problem configurations.

These limitations become particularly acute for behavioral target satisfaction, where agents must reason about aggregate properties that emerge over sequences of actions rather than achieving discrete end states. Traditional constraint satisfaction problems focus on finding variable assignments that satisfy constraints [99]. In behavioral simulation, targets specify aggregate properties that emerge from individual agent actions rather than direct variable assignments. Baier & McIlraith (2009) [13] explored planning with preferences and soft constraints, but their work addressed classical planning where constraints can be checked at each step.

The challenge of applying target satisfaction to LLM-based agents lies in the gap between declarative target specifications and the procedural nature of trajectory generation. Unlike classical planning where constraints can be checked incrementally, behavioral targets can only be evaluated over complete or partial trajectories, requiring agents to reason about long-term consequences of immediate actions [72]. This fundamental

challenge motivates formal target-checking mechanisms that verify whether generated batches satisfy specified aggregate patterns, directly motivating the research questions in Chapter 6: which test-time compute allocation choices improve target-satisfaction under a finite model-call budget, and at what cost in persona fidelity and model calls (**RQ1**); and how target-set satisfiability varies with target composition and target load (**RQ2**).

2.6 Gaps and Open Questions

This section synthesizes the limitations identified in Sections 2.2 through 2.5 and highlights three converging gaps motivating this dissertation’s approach. First, while LLM-based roleplaying agents show promise for behavioral simulation [143, 9, 81, 188], their ability to satisfy expected behavioral targets under controlled allocation of test-time compute remains underexplored. Prior work emphasizes single-trajectory realism and agent believability rather than explicit calibration to published aggregate statistics [68]. Prior LLM-based agent-based modeling techniques underutilize LLMs’ capacity for in-context reasoning and planning during the calibration phase. Existing frameworks like HYP-MIX address calibration for individual learner characteristics but do not address behavioral target satisfaction in multi-agent interactions [123].

Second, while LLMs demonstrate planning capabilities for instance-level goals [129, 51, 69, 198], using them to satisfy behavioral constraints within multi-agent interactions represents an underexplored area. The challenge lies in the gap between declarative target specifications and the procedural nature of trajectory generation. Unlike classical planning where constraints can be checked at each step [99, 13], emergent behavioral targets can only be evaluated over complete or partial trajectories, requiring agents to reason about long-term consequences of immediate actions [72].

Third, as discussed in Section 2.4, current approaches rely heavily on manual prompt engineering, which is labor-intensive and brittle. While techniques like chain-of-thought reasoning and systematic prompt optimization can improve consistency, they still require extensive manual effort and domain expertise, and face compositionality challenges when

specifying complex behavioral patterns.

Fourth, evaluation practices often rely on subjective judgments creating opportunities for Clever Hans effects [86], and critical gaps remain in generalization, validation, and systematic evaluation [7, 158]. There is a need for shifting from instance-level evaluation to distributional evaluation comparing aggregate patterns against empirical findings [60, 135, 123]. However, Orgad et al. (2024) demonstrate that LLMs encode accurate knowledge internally but often apply it inconsistently across contexts, making validation challenging [141].

Finally, as discussed in Sections 2.2 and 2.4, traditional ABM calibration methods are labor-intensive and brittle, motivating reformulation as an in-context steering problem. However, the relationship between the number and nature of behavioral targets and LLMs’ ability to reproduce them remains underexplored, creating uncertainty about which calibration scenarios are tractable for LLM-based planning.

This review reveals a convergence of challenges motivating this dissertation’s approach: the authoring bottleneck in educational technology, limitations of traditional ABM calibration methods, gaps in LLM-based roleplaying agents’ ability to satisfy behavioral targets, and insufficient calibration and validation approaches. These threads converge on two research questions: **RQ1** asks which test-time compute allocation choices improve target-satisfaction under a finite model-call budget, and at what cost in persona fidelity and model calls; **RQ2** asks how target-set satisfiability varies with target composition and target load (target identity, target combinations, and the number of simultaneous targets). Chapter 6 addresses these questions through controlled experiments in pair-programming simulations, where the generator spends test-time compute (through independent sampling, sampling and selection among candidate outputs, and planner-guided generation with or without persona information) to satisfy behavioral targets matching published aggregate statistics.

CHAPTER 3

UNDERSTANDING MANUAL AUTHORING: A USABILITY STUDY OF TASK-ORIENTED DIALOGUE AGENT SPECIFICATION INTERFACES

This dissertation develops a progression from manual to automated calibration of LLM-based agents for behavioral simulation. Before proposing systematic frameworks or automated planning approaches, establishing a baseline understanding is essential: how do practitioners currently author agents using natural language interfaces, and what challenges limit the scalability of manual approaches, motivating the need for automated calibration methods developed in Chapters 4 and 5?

The first stage of that progression examines the usability of different agent authoring approaches through a controlled human-computer interaction study. Three interfaces representing two authoring approaches are investigated: one instruction-based interface and two schema-based interfaces (procedural and declarative). The comparison reveals fundamental trade-offs in cognitive scaffolding, effort, and precision that scale poorly with task complexity. These findings establish empirical grounding for the authoring challenges that motivate the systematic and automated calibration methods developed in subsequent chapters.

This study¹ focuses on task-oriented dialogue agents, but the findings apply broadly to LLM-based agent authoring. Whether building dialogue agents that respond to user requests or role-playing agents that simulate learner behavior, practitioners face the same core challenge: specifying agent behavior through natural language while ensuring agents behave as intended, maintain consistency across contexts, and avoid costly re-authoring when agent behavior fails to match expectations. These shared challenges establish the empirical foundation that motivates the systematic calibration frameworks and automated planning approaches developed in Chapters 4 and 5.

¹ This chapter is based on work published in collaboration with Mehmet Celepkolu, Joseph B. Wiggins, and Kristy Elizabeth Boyer.

3.1 Background

This chapter presents a usability study that examines three authoring approaches to identify their usability trade-offs, addressing a critical gap in understanding how practitioners experience these authoring methods. LLMs have enabled more accessible development of task-oriented dialogue systems, and two main data-efficient authoring approaches have emerged: schema-based approaches and instruction-based prompts. Within the schema-based approach, multiple formats exist. This study examines three distinct authoring approaches: instruction-based prompts, procedural schemas, and declarative schemas.

3.1.1 Accessible Dialogue Agent Authoring

To understand why alternative interfaces are needed, the limitations of current commercial platforms are first examined. Large language models such as T5 [150] and GPT-3 [31] have significantly streamlined the development of task-oriented dialogue systems, yet two main challenges persist: (1) the requirement of machine learning expertise for training or fine-tuning language models on specific dialogue tasks, and (2) the limited availability of labeled in-domain training data [34]. Commercial platforms such as Google DialogFlow² and Amazon Lex³ have attempted to address the first challenge by providing accessible interfaces that cater to machine learning non-experts. However, these platforms generally rely on fragmented pipeline architectures, and their performance remains heavily dependent on substantial in-domain training data [159], recreating the cold-start bottleneck that motivates this chapter’s investigation of alternative authoring approaches.

The standard approach to developing dialogue agents has traditionally required collecting and annotating substantial in-domain data for training the various components of the dialogue pipeline, including natural language understanding, dialogue management, and response generation. In particular, inferring a dialogue manager policy from examples is challenging since a latent policy must be inferred from the examples. However, obtaining

² <https://cloud.google.com/dialogflow>

³ <https://aws.amazon.com/lex/>

in-domain data for training machine learning models remains a bottleneck; the process is labor-intensive, time-consuming, and requires expertise. Although commercial tools simplify development, they remain data-hungry. In response to this cold-start problem, data-efficient machine learning approaches, such as few-shot learning [189], have emerged. Until recently, few-shot learning methods primarily focused on learning models for *individual* components within a dialogue system’s pipeline, such as natural language understanding [148], dialogue state tracking [35], and response generation [148]. The special case of few-shot learning where the model requires zero demonstrations is called zero-shot learning [157]. In contrast to these piecemeal approaches, direct specification methods—schemas and instructions—offer alternatives that allow practitioners to specify the *overall* behavior of dialogue agents while reducing dependence on training data. This context motivates investigation of alternative interfaces that allow specifying agent behavior more directly.

3.1.2 Data-Efficient Approaches: Schemas and Instructions

In response to the cold-start problem, two complementary authoring approaches (methods for specifying agent behavior) have emerged in recent research.

A dialogue schema in the context of task-oriented dialogue is a structured representation of conversation flow that captures key information pieces (called “slots” or “entities”), their relationships, and the paths users can take to complete a task. In dialogue systems, *intents* represent user goals or purposes (e.g., “book a hotel”, “cancel a ride”), while *slots* are specific pieces of information needed to fulfill those intents (e.g., check-in date, number of guests). For example, in a hotel booking dialogue, slots might include number of guests, check-in date, and room preferences. The schema specifies how the agent should collect this information and what actions to take. Schema-based approaches facilitate knowledge transfer across previously unseen domains using a minimal number of training examples [26, 152, 136].

The task-oriented dialogue research community has released several public benchmark

datasets for evaluating progress on key dialogue tasks. Historically, these datasets consisted primarily of example dialogues between users and virtual assistants. However, the community has recently moved toward schema-based datasets that include structured schema representations alongside dialogue examples. Two prominent schema-based benchmark datasets have emerged: the Schema-Guided Dialogue (Schema-Guided Dialogue (SGD)) dataset [152], which uses a declarative schema format that links intents and slots to natural language descriptions, and the Situated Task-oriented Agent Reasoning (STAR) dataset [136], which uses a procedural schema format that represents dialogue as a directed graph similar to a flowchart. Various schema formats have been proposed in prior research. This study examines two prominent formats grounded in these benchmark datasets: procedural schemas, which represent dialogue as sequential flowcharts, and declarative schemas, which organize dialogue around intents and slots with natural language descriptions. Procedural schemas (see Figure 3-1) capture sequential flow with nodes for user/system actions and service calls, helping practitioners play out interactions. Declarative schemas (see Figure 3-2) link intents and slots to concise descriptions, supporting modular, higher-level specification. Both aim to enable knowledge transfer with minimal examples. The primary focus of current research in schema-based dialogue has been model development [39, 138, 113, 110, 128], but extending these methods to practical development environments “in the wild” [93] requires addressing usability challenges—the focus of this investigation.

This study investigates three authoring approaches:

1. **Instruction-based prompts:** Natural language text instructions
2. **Procedural schemas:** Flowchart-like sequential dialogue structures (see Figure 3-1)
3. **Declarative schemas:** Intent- and slot-based modular specifications (see Figure 3-2)

Structured dialogue schemas encode flows, entities, and relationships so that practitioners can specify conversations explicitly and transfer knowledge across domains

with minimal training examples [26, 152, 136]. Instruction-based prompts, in contrast, leverage the rapid advancements in large language models’ capacity to interpret natural language instructions [132, 115], allowing practitioners to specify behavior in natural language that LLMs can interpret in a zero-shot manner, mirroring how humans are trained to communicate in new domains—for example, when a customer service representative learns to handle support requests for a new product line. However, instructions written in the form of “unconstrained prose” can be ambiguous, especially for scenarios requiring ordering constraints and conditional logic, making it easy to omit critical details. These three interfaces seek to minimize dependence on labeled training examples (utterances annotated with their intended meanings). This study examines these three interfaces to understand their usability trade-offs for specifying the *overall* behavior of dialogue agents.

3.1.3 Data Requirements in Dialogue Agent Development

The standard approach to developing dialogue agents has traditionally required collecting and annotating substantial in-domain data for training the various components of the dialogue pipeline, including natural language understanding, dialogue management, and response generation. In particular, the dialogue manager from examples is tricky since a latent policy has to be inferred from the examples. However, obtaining in-domain data for training machine learning models remains a bottleneck; the process is labor-intensive, time-consuming, and requires expertise. Although commercial tools simplify development, they remain data-hungry. In response to this cold-start problem, data-efficient machine learning approaches, such as few-shot learning [189], have emerged. Until recently, few-shot learning methods primarily focused on learning models for *individual* components within a dialogue system’s pipeline, such as natural language understanding [148], dialogue state tracking [35], and response generation [148]. The special case of few-shot learning where the model requires zero demonstrations is called zero-shot learning [157]. In contrast to these piecemeal approaches, direct specification methods—schemas and instructions—offer alternatives that allow practitioners to specify the *overall* behavior of dialogue agents while

reducing dependence on training data.

3.1.4 Schema-Based Dialogue

A dialogue schema in the context of task-oriented dialogue is a structured representation of the conversation flow that can capture the key entities (also known as “slots”), their relationships, and the potential paths a user can take to accomplish a specific task or goal. Schema-based approaches facilitate knowledge transfer across previously unseen domains using a minimal number of training examples [26, 152, 136]. Various schema formats have been proposed in prior research. Procedural schemas (see Figure 3-1) capture sequential flow with nodes for user/system actions and service calls, helping practitioners play out interactions. Declarative schemas (see Figure 3-2) link intents and slots to concise descriptions, supporting modular, higher-level specification. Both aim to enable knowledge transfer with minimal examples. The primary focus of current research in schema-based dialogue has been model development [39, 138, 113, 110, 128], but extending these methods to practical development environments “in the wild” [93] requires addressing usability challenges—the focus of this investigation.

3.1.5 Instruction-Based Prompts

The rapid advancements in large language models’ capacity to interpret natural language instructions have sparked a significant increase in the use of instruction-based prompts [132] for executing various Natural Language Processing (NLP) tasks in a zero-shot manner [115]. While prompts can adopt diverse forms, including in-context examples, employing natural language instructions is particularly fitting for developing dialogue agents, as this approach mirrors how humans learn and are instructed to communicate in new domains. Instruction-based prompts offer an accessible, low-friction interface that leverages LLM’ capacity to interpret natural language. However, instructions written in the form of “unconstrained prose” can be ambiguous, especially for scenarios requiring ordering constraints and conditional logic, making it easy to omit critical details.

3.1.6 Usability Considerations

While the primary focus of current research in schema-based and instruction-based approaches has been model development and performance metrics [39, 138, 113, 110, 128], the human experience of authoring with these interfaces remains underexplored. To truly extend the applicability of these authoring approaches beyond research settings and into broader development environments “in the wild” [93], it is crucial to address the challenges of transforming these formats into practical and user-friendly interfaces.

From a human-computer interaction perspective, dialogue agent authoring is fundamentally a collaborative problem between human practitioners and computational tools. Different interface approaches distribute cognitive work in distinct ways: procedural schemas provide scaffolding for stepwise reasoning through conversation flows; declarative schemas support higher-level conceptual thinking about intents and slots; and instruction-based prompts maximize flexibility but risk ambiguity in specification. Understanding these usability trade-offs is essential to designing workflows that appropriately match interface choice to task complexity and iteration needs.

3.1.7 Research Question and Contribution

To address this gap in understanding, an exploratory human-computer interaction study was conducted to answer the following research question:

What are the usability trade-offs and issues involved in authoring a task-oriented dialogue agent using three distinct authoring approaches: instruction-based prompts, procedural schemas, and declarative schemas?

To answer this question, two hypothetical dialogue scenarios in a taxi ride-booking domain are introduced, and 23 participants who had completed a graduate-level course on spoken dialogue systems are asked to author dialogue agents for each scenario. For each scenario, participants first authored using the instruction-based interface, then used one of the two schema interfaces (procedural or declarative) of their choosing. This study design resulted in four authoring activities per participant: two using the instruction-based

interface (one per scenario) and two using a schema interface (one per scenario, with their choice of procedural or declarative format). Through inductive coding and subsequent thematic analysis of survey data, users’ authoring experiences across these three interfaces were explored. The findings reveal distinct trade-offs in cognitive scaffolding, effort, and precision/expressivity dimensions, establishing empirical evidence of manual authoring bottlenecks, which directly motivate the systematic and automated calibration approaches evaluated in Chapters 4, 5, and 6.

3.2 Study Design and Methodology

The human-computer interaction study design is detailed below, including the participant population, the task scenarios, and the three authoring approaches participants evaluated.

3.2.1 Participants and Recruitment

The study recruited 23 undergraduate and graduate students (18 male, 5 female) who had completed a spoken dialogue systems course. The course covered concepts related to the linguistics of conversation, dialogue interface design, components of a standard dialogue system, and fundamental machine learning and NLP concepts. As part of their final evaluation, the course required students to create their own dialogue agents throughout the semester and live-test them with their peers. Participants were therefore familiar with dialogue system concepts, specifically intents, slots, and dialogue flows, and had prior experience creating dialogue agents. Participants were recruited through a course discussion board announcement and earned 2% course credit for their involvement in the study.

3.2.2 Task Scenarios

Participants were asked to author dialogue agents for two scenarios in the taxi ride-booking domain. This domain was chosen because it represents a familiar, well-understood task-oriented dialogue scenario that participants could readily comprehend, while still requiring structured information gathering and conditional logic that would reveal differences across authoring approaches. The scenarios were designed to

represent two common authoring tasks: editing an existing dialogue flow and creating a new dialogue flow from scratch.

Scenario 1: Adding Service Tiers (Editing an Existing Flow). Participants were asked to edit an existing ride-booking dialogue flow to incorporate a new feature: the taxi service provider wanted to offer three different service tiers (*XL*, *Share*, and *Regular*) to customers. Participants needed to modify the existing flow to ask customers which tier they preferred. This represented a relatively straightforward editing task: adding a single new piece of information (the service tier) to be collected during the conversation.

Scenario 2: Integrating Ride Cancellation (Adding a New Flow). Participants were asked to create a new dialogue flow from scratch for a ride cancellation feature. The taxi service wanted to allow customers to cancel previously booked rides, with a \$10 cancellation fee if not cancelled within the first 5 minutes. Participants needed to design a complete dialogue flow that would: collect the customer’s name and phone number, retrieve their booking information and any applicable fees from a backend system, inform the customer of the cancellation charge, obtain confirmation, and finalize the cancellation. This represented a more complex task requiring a multi-step sequential process with conditional logic and multiple backend service calls.

3.2.3 Authoring Approaches

To clarify the terminology used throughout this section: an *authoring interface* refers to the tool or user interface that participants interact with to specify agent behavior (e.g., a text field, a visual block-based interface). A *schema format* refers to the underlying data structure or representation used to encode dialogue specifications (e.g., procedural vs. declarative formats, as introduced in the Data-Efficient Approaches subsection). A *block-based interface* is a custom visual programming interface created for this study that allows participants to construct schemas by dragging and connecting visual blocks. Finally, a *schema* is the actual structured representation of a dialogue flow that results from using an authoring interface.

The instruction-based prompt interface⁴ provides participants with a simple text field for writing natural language instructions. The block-based visual programming interfaces are developed to make schema-based authoring more accessible. As discussed in Section 3.1.2, procedural and declarative schema formats are typically represented as raw JavaScript Object Notation (JSON) objects in benchmark datasets, which can be difficult and error-prone to edit directly. To address this issue and provide participants with a more accessible authoring experience, two custom block-based programming interfaces are developed using Google Blockly⁵, each corresponding to one of the schema formats introduced earlier. These visual interfaces reduce syntax errors and support structured composition through drag-and-drop interaction.

The block-based interface for the Procedural schema format (see Figure 3-1) enables participants to construct dialogue flows as directed graphs similar to flowcharts, with nodes representing user utterances, system responses, or backend service calls. Nodes linked to user utterances or system responses can be connected to example utterances. The block-based interface for the Declarative schema format (see Figure 3-2) enables participants to define intents and slots with natural language descriptions at a higher level of abstraction, supporting modular specification.

3.2.4 Study Procedure

Sessions are remote and capped at 90 minutes. For both scenarios listed in Section 3.2.2, participants first author using the instruction-based interface, then use one of the two schema interfaces (procedural or declarative) of their choosing. This results in four authoring activities per participant: two using the instruction-based interface (one per scenario) and two using a schema interface (one per scenario, with their choice of

4 To clarify the terminology used throughout this section: an *authoring interface* refers to the tool or user interface that participants interact with to specify agent behavior (e.g., a text field, a visual block-based interface). A *schema format* refers to the underlying data structure or representation used to encode dialogue specifications (e.g., procedural vs. declarative formats, as introduced in Section 3.1.2). A *block-based interface* is a custom visual programming interface created for this study that allows participants to construct schemas by dragging and connecting visual blocks. Finally, a *schema* is the actual structured representation of a dialogue flow that results from using an authoring interface.

5 <https://developers.google.com/blockly>

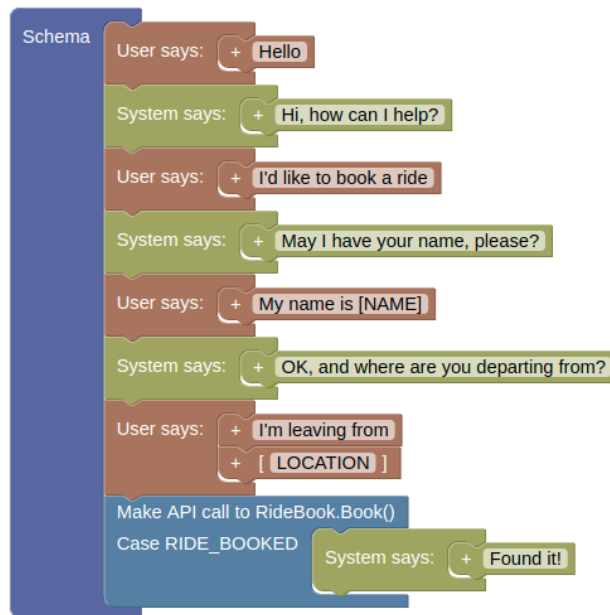


Figure 3-1. A sample snippet of the custom block-based interface for the Procedural schema format, which is based on the STAR dataset [136]. The flowchart-like structure allows practitioners to visually construct conversation flows with nodes representing user utterances, system responses, and backend service calls.

Figure 3-2. A sample snippet of the custom block-based interface for the Declarative schema format, which is based on the Schema-Guided Dialogue (SGD) dataset from [152]. This interface enables practitioners to define intents and slots with natural language descriptions at a higher level of abstraction.

procedural or declarative format).

3.2.5 Data Collection Instruments

At the outset of the study, participants complete a pre-survey in which they report basic demographic information and self-assess their proficiency in three areas: (1) creating virtual assistants using platforms such as Google DialogFlow, (2) general programming abilities, and (3) understanding of machine learning concepts. Subsequently, participants watch two brief video tutorials that demonstrate how to use the two custom block-based interfaces to define Declarative and Procedural schemas respectively. Upon completion of the authoring tasks, participants fill out a post-survey that consists of: (1) a questionnaire designed to assess the usability of the block-based interfaces for schema definition, utilizing the System Usability Scale [29], and (2) a survey asking for participants’ open-ended feedback on their experiences of creating schemas and crafting instruction-based prompts.

3.2.6 Analysis Approach

A mixed-methods approach is employed to analyze the data. For quantitative analysis, System Usability Scale (SUS) scores between the two scenarios are compared using a paired t-test. For qualitative analysis, an inductive coding process [55] is used to analyze the open-ended responses from the post-survey. Two researchers conduct open coding on all of the transcripts, and then the primary coder, in consultation with another researcher, iteratively derives themes from participants’ open-ended responses. Three main themes emerge from this analysis: cognitive scaffolding, effort, and precision/expressivity.

3.3 Results

Out of 23 participant-defined block-based programs, 18 were successfully compiled into schemas, while 5 had minor issues like orphaned blocks or improper text formatting within the blocks. Based on the System Usability Scale results, participants were generally satisfied with the block-based interfaces for both scenarios, with no statistical difference between the reported SUS scores for the two scenarios ($p = 0.20$).

Through inductive analysis of participant responses, three main themes emerged that

characterize the usability landscape of dialogue agent authoring approaches. To analyze the responses to the open-ended survey items, the response-units were first grouped based on which interface they pertained to (instruction-based, procedural schema, or declarative schema), and then individual themes were identified under each. Each dimension reveals distinct trade-offs that affect how effectively practitioners can specify agent behavior across varying task complexities.

3.3.1 Cognitive Scaffolding: Thinking Through the Details

When developing a dialogue agent for a real-world task, the key decisions that need to be made include the types of information that needs to be collected from the end-user, the set of supported intents, and other factors such as phrasing of the utterances. A dialogue system authoring interface can offer the practitioner various kinds of scaffolding to reduce the cognitive effort in translating their understanding of the dialogue into a working agent.

Seven participants reported that the instruction-based prompt’s blank text field fell short in helping them think through the specific pieces of information that needed to go into their instructions. Of particular note was the concern around the lack of scaffolding provided by a blank text-field in thinking through fine-grained details of the dialogue flow. The majority of participants (13 out of 23) appreciated the Procedural schema’s resemblance to an actual conversation, stating that it allowed them to play out a conversation in their head and think through details that emerge in live interactions. Several participants also found the visual representation appealing. Reactions to the declarative schema were mixed: three participants attributed their aversion to the abstract nature of thinking about the dialogue flow in terms of intents and slots, while two participants who did prefer the Declarative schema said that thinking about the dialogue flow in a declarative way freed them up from having to worry about the specific order in which the slots had to be requested as well as the natural language phrasing of the utterances.

3.3.2 Effort: Time and Cognitive Load

This theme encompasses findings related to both the duration of time as well as the cognitive effort involved in authoring an agent. Perceptions about the effort involved varied depending on task complexity and the interface used.

For instruction-based prompts, perceptions about effort differed depending on which of the two scenarios was in question. Four participants reported that writing instructions for Scenario 1 (adding service tiers) was quite straightforward since they only had to mention the three tiers. However, Scenario 2 (ride cancellation) was seen as significantly more complex, and participants felt that unambiguously writing instructions required putting significant thought into it. Even then, many felt the result was “long-winded.”

For the Procedural schema, although the amount of manual effort was not reported as an issue when it came to Scenario 1, five participants stated that defining a Procedural schema for Scenario 2 (in which they had to define a complete dialogue flow from the ground up) was tedious and time-consuming. In contrast, three participants reported the Declarative schema to be less labor-intensive than the Procedural schema, describing it as “less cumbersome” and “simpler and higher-level.”

3.3.3 Precision and Expressivity: Control vs. Ambiguity

Participants reported fine-grained control over minute details of the dialogue flow as an important factor in dialogue authoring formats. This theme presents their responses on the level of precision and expressivity that each authoring format provided them.

Six participants felt that natural language instructions inherently left room for ambiguity and misinterpretation, particularly in Scenario 2. This was especially problematic for structured, sequential tasks where certain steps must occur in a specific order (“must do A before B” relationships). Participants made no direct comments about the expressivity (or the lack thereof) of the Procedural schema, suggesting it provided adequate fine-grained control.

For the Declarative schema, two contrasting perspectives emerged concerning

expressivity. The first viewpoint (4 participants) considered the inability to create complex dialogue flow patterns, such as loops and conditional branches, as a limitation on the schema’s expressivity. On the other hand, the second perspective (4 participants) argued that delegating control logic to the underlying model was a more sensible approach, as manually crafting dialogue flow could potentially lead to brittleness.

3.3.4 Quantitative Results: System Usability Scale

Based on the System Usability Scale results, participants were generally satisfied with the block-based interfaces for both scenarios. There was no statistically significant difference between the reported SUS scores for the two scenarios: Scenario 1 (modifying an existing schema to include a service tier) had a mean SUS score of 74.18 (SD 11.67), while Scenario 2 (creating a ride cancellation feature from scratch) had a mean SUS score of 69.70 (SD 17.99), with $p = 0.20$ from a paired t-test.

3.4 Discussion

These findings reveal fundamental challenges in agent authoring that extend beyond interface design to questions of cognitive load distribution, effort scalability, and model transparency, and motivate design considerations for authoring frameworks introduced in subsequent chapters.

Trade-offs Vary by Task Complexity Interfaces offer different trade-offs. For simple edits, instructions may suffice; for complex flows, schemas provide valuable scaffolding albeit with higher manual effort. Choosing an interface should consider task complexity and iteration needs.

Cognitive Load in Agent Authoring Authoring is cognitively demanding regardless of interface. Procedural schemas distribute load across steps, declarative schemas require up-front conceptualization of intents and slots, and instructions demand maintaining coherence across long descriptions.

Effort Scalability Considerations Manual effort grows with complexity across interfaces; ground-up procedural design is particularly time-consuming. These observations motivate systematic frameworks that improve modularity and reuse and, ultimately, automation to reduce calibration burden.

Audience Design and Model Transparency Participants differed in how specific they believed instructions or schemas must be, reflecting uncertainty about model interpretive capabilities. People adapt their language based on their audience’s interpretative capability [43], and this was evident in participants’ mixed interpretations of required specificity. Interface choice should be informed by model behavior; transparency about what the model can reliably infer helps avoid under- and over-specification.

Design Recommendations These findings suggest several general design implications for the development of authoring tools for defining task-oriented dialogue agents. First, combining the strengths of different interfaces may be beneficial: while the instruction-based interface often fell short in helping participants think through specific pieces of information needed, the procedural schema interface excelled at it. At the same time, schema-based interfaces (particularly the declarative schema) ran the risk of being seen as complex and *code-like* compared to writing instructions in English. For example, the procedural schema interface could be used when authoring an agent for a brand new scenario from scratch, while the declarative schema interface or instruction-based interface can be used to make edits to an existing agent.

Second, participants’ reported effort for writing instruction-based prompts and creating schemas varied depending on the complexity of the task. In order to create a more user-friendly authoring experience, it is important to design interfaces that minimize cognitive and manual effort, particularly for complex tasks. This could include features such as drag-and-drop components, reusable templates, or visual aids. Supporting context-aware interface switching may further improve authoring efficiency.

3.5 Contributions

This study provides empirical grounding for understanding the usability challenges inherent in manual agent authoring. The findings establish key insights about interface design trade-offs and motivate the need for improved authoring approaches.

1. **Empirical Characterization of Interface Trade-offs.** Manual authoring is challenging across interfaces; cognitive and effort costs scale with complexity. Each interface makes distinct trade-offs, and no single approach fully addresses all challenges. The three dimensions identified (cognitive scaffolding, effort, and precision/expressivity) provide a systematic framework for evaluating agent authoring approaches.
2. **Implications for Framework Design.** Current interfaces lack modularity and make it difficult to reuse and adapt specifications. These findings motivate frameworks that improve modularity and reuse. Modular approaches that support reusable specifications can reduce re-authoring overhead while maintaining evaluability.
3. **Motivation for Automated Approaches.** Even with improved interfaces, calibration remains effortful. The baseline authoring challenges established here (particularly the scalability problems with task complexity) demonstrate the need for systematic frameworks and automated calibration approaches that can reduce the manual burden of agent authoring.
4. **Broader Applicability to Educational Agents.** Task-oriented dialogue agents exemplify broader LLM-based agent authoring challenges. Role-playing agents for learner simulation face the same core issues: specifying behavior through natural language (prompts, schemas), ensuring consistency, and managing calibration costs. Tools must balance expressivity with usability to support cold-start development across all agent types.

3.6 Conclusion

This study evaluated the user experience associated with three distinct dialogue system authoring approaches: instruction-based prompts, procedural schemas, and declarative schemas. The findings reveal fundamental trade-offs across cognitive scaffolding, effort, and precision dimensions that affect how effectively practitioners can specify agent behavior.

Instruction-based prompts were seen as easily approachable yet lacking in detailed guidance, leading to potential omissions and ambiguities. Procedural schemas resonated with participants due to their conversational mirroring, but were viewed as labor-intensive for intricate scenarios. Declarative schemas, despite offering flexibility and higher-level simplicity, were perceived as abstract and occasionally insufficient for complex dialogue flows. Task complexity determines which trade-offs dominate in practice, suggesting that any scalable calibration framework must accommodate heterogeneous authoring complexity.

These empirical findings establish the baseline manual authoring challenges that motivate the systematic frameworks and automated calibration approaches developed in subsequent chapters. While manual authoring will remain important, the scalability problems identified here motivate the automated calibration frameworks developed in Chapters 4 and 5 and evaluated in Chapter 6, which aim to reduce authoring burden while maintaining evaluability and behavioral consistency.

CHAPTER 4

SYSTEMATIC CALIBRATION OF LLM-BASED LEARNER SIMULATIONS: THE HYP-MIX FRAMEWORK

As we saw in the previous chapter (Chapter 3), manual authoring of LLM-based agents faces severe scalability challenges regardless of interface choice. The empirical findings revealed that cognitive load, effort, and precision all degrade with task complexity across instruction-based prompts, procedural schemas, and declarative schemas. These challenges motivate the need for more systematic frameworks that can reduce authoring costs while maintaining calibration quality. This chapter¹ addresses these limitations by developing a systematic, composable approach to agent calibration that enables “calibrate once, use many” authoring practices. This raises a fundamental question: can we develop more systematic approaches that reduce authoring costs while maintaining calibration quality?

4.1 Motivation

The previous chapter examined authoring challenges for task-oriented dialogue agents: systems that help users book rides, check weather, or make reservations. This chapter shifts to a related but distinct class of LLM-based agents: role-playing simulations that generate synthetic learner behavior for educational technology research and design. While the application domain differs, the core authoring challenge remains remarkably similar: practitioners must specify how agents should behave through natural language interfaces (prompts, schemas, instructions) while striving to ensure that agents behave as intended across diverse contexts.

We explicitly frame the authoring problem as a calibration challenge. In Chapter 3, we examined how practitioners *author* agents by specifying behavior through natural language interfaces. This chapter reframes that same fundamental challenge as *calibration*: the process of ensuring that agent behavior matches expert expectations in a statistically verifiable way. The shift in terminology reflects a shift in emphasis: rather than focusing

¹ This chapter is based on work published in collaboration with Adam Davies, Jina Kang, ChengXiang Zhai, and Kristy Elizabeth Boyer.

on the process of writing specifications (authoring), we emphasize the outcome of ensuring those specifications are satisfied (calibration). This framing enables us to evaluate authoring success through distributional validation rather than subjective assessment of individual outputs.

4.1.1 LLM-Based Learner Simulations: Opportunities and Challenges

We frame learner simulation authoring through a formal learner model structure. A learner model $L = (C, B)$ consists of two components: C is a set of learner characteristics (e.g., persistence, geometry proficiency, domain knowledge), and B is a set of behavioral priors representing expert beliefs about how these characteristics influence observable behavioral patterns. The behavioral priors in B encode relationships between characteristics and action distributions, specifying how learners with different attribute values should behave across diverse environmental contexts. For role-playing simulations, practitioners must specify these behavioral priors: expert beliefs about how learners with different characteristics (high persistence, low geometry proficiency, math anxiety) tend to act across situations, often drawn from learning theory, prior empirical studies, or expert intuition.

Large language models offer an appealing approach to generating synthetic learner behavior for educational technology design. Educational technology practitioners face a fundamental challenge: how can they design and evaluate learning environments before deploying them in real classrooms? By role-playing learners with specified characteristics, LLMs can potentially generate action sequences that help designers iterate on open-ended learning environments without the expense and logistical complexity of classroom pilots [102]. In principle, designers could explore how learners with varying persistence levels, domain knowledge, or affective states might interact with a proposed learning activity, all before writing a line of production code or recruiting a single participant.

However, this promise comes with significant perils. Designers working on novel learning environments face a fundamental cold-start problem: without existing learner logs from the target environment and population, how can they specify what realistic learner

behavior should look like? [76]. One might imagine generating face-valid demonstrations (simulated behaviors that superficially resemble plausible learner actions), but face validity alone provides no guarantee that the statistical properties of simulated behaviors match expert expectations or align with established learning theory. A simulation might produce individual action sequences that “look right” to a casual observer while systematically misrepresenting the relationships between learner characteristics and behavioral patterns.

What is needed are calibrated simulations: systems whose aggregate statistical properties match expert expectations and theoretical priors in a principled, verifiable way. Rather than asking whether individual simulated trajectories seem plausible, we must ask whether simulated learners with higher persistence actually exhibit the expected patterns of task engagement, whether learners with lower geometry proficiency show the anticipated help-seeking behaviors, and whether these relationships hold across the diverse states learners encounter. This shift from instance-level plausibility to distributional calibration is essential for simulations to serve as reliable tools for design exploration and hypothesis testing.

4.1.2 Current Calibration Approaches and Limitations

Unfortunately, current approaches to achieving calibrated LLM-based simulations remain fundamentally ad-hoc and unscalable. The typical workflow involves iterative prompt tweaking: practitioners write an initial prompt, run simulations, examine outputs, adjust prompts based on discrepancies, and repeat. This trial-and-error process suffers from three critical problems.

1. **High Sensitivity to Prompt Phrasing:** LLM outputs are extremely sensitive to even minor changes in prompt phrasing [163, 118, 133]. Edits such as reordering instructions, altering a single word, or modifying structure can dramatically alter the resulting behavioral distributions. This brittleness means that a carefully calibrated prompt for one learner model may fail entirely if only a single characteristic is changed or updated. As a result, practitioners cannot depend on calibration

remaining stable as the simulation is adapted or extended.

2. **Poor Scalability Across Models and Hypotheses:** The ad-hoc nature of the process makes it costly to transfer or adapt calibration across different simulation contexts. Testing a new hypothesis (for instance, about persistence) requires recalibrating the entire prompt and verifying every change through repeated experimentation. As the number of learner characteristics increases, the combinatorial growth in prompt configurations quickly makes systematic exploration infeasible. Thus, the calibration workload grows disproportionately with model complexity, which hampers the use of simulations for iterative design.
3. **Risk of Clever Hans Effects:** Ad-hoc calibration increases the risk of unintentional cueing, known as Clever Hans effects [86]. As practitioners adjust prompts to match their expectations, they may inadvertently encode those expectations so thoroughly that the LLM simply mirrors what has been subtly implied, rather than genuinely reasoning about the relationships between learner characteristics and behaviors. The result may be simulations that align with expectations not because they have captured generalizable patterns, but because they have been engineered to do so. Without systematic safeguards, it becomes difficult to distinguish true calibration from extensive prompt manipulation.

Specifying behavioral priors through traditional prompt engineering suffers from the same scalability problems identified in Chapter 3: poor composability, high cognitive load, and effort that scales poorly with complexity. These challenges motivate a fundamental research question: can a systematic framework be developed for calibrating LLM-based learner simulations that addresses the brittleness, scalability, and validation problems of ad-hoc approaches? More specifically, can authoring workflows be designed where calibration generalizes across different learner models without extensive prompt re-engineering, enabling “calibrate once, use many” development practices?

4.1.3 Research Objectives and Contributions

This chapter introduces HYP-MIX, a hypothesis-based simulation authoring framework that treats expert behavioral priors as modular, testable units. Rather than embedding knowledge implicitly in monolithic prompts, HYP-MIX asks practitioners to express their expectations as Marginalized Distributional Hypotheses (MDHyps): explicit statements relating individual learner characteristics to action distributions. The key insight is compositional generalization: by grouping hypotheses into classes with shared functional forms, we can calibrate prompt templates at the class level and then instantiate new hypotheses by filling in specific values. This enables practitioners to add, swap, and combine hypotheses without recalibrating the entire system, addressing the scalability challenges that plague ad-hoc approaches.

The contribution is both empirical and methodological. Empirically, controlled experiments in a physics learning environment demonstrate that GPT-4 Turbo maintains calibration across 89% of learner model modifications (16 of 18 test cases spanning five types of edits), providing the first evidence that compositional generalization is achievable with current LLM technology. Methodologically, a systematic evaluation protocol based on falsifiable statistical hypotheses is introduced that guards against Clever Hans effects by requiring calibration to hold under explicit model transformations rather than merely matching hand-tuned expectations.

4.2 The Case for Hypothesis-Based Calibration

To address the calibration challenges outlined above, we need a principled framework for expressing and evaluating behavioral expectations. This section develops the conceptual foundation for hypothesis-based calibration and its key properties.

4.2.1 From Expert Knowledge to Formal Hypotheses

Educational technology practitioners rarely think about learner behavior as monolithic, unstructured intuition. Instead, their knowledge takes the form of hypotheses: explicit or implicit beliefs about relationships between learner characteristics and

behavioral patterns. A learning scientist might hypothesize that "learners with higher persistence are less likely to abandon challenging tasks as time passes," while a domain expert might expect that "learners with lower geometry proficiency distribute their measurements more uniformly across options rather than strategically focusing on productive actions." These hypotheses represent accumulated theoretical understanding, empirical findings from prior studies, and professional experience observing learners in action.

Marginalized Distributional Hypotheses (Marginalized Distributional Hypotheses (MDHyps)) formalize these hypotheses as the unit of authoring and assessment. An MDHyp is a natural-language statement that relates a single learner characteristic to a probability distribution over actions while marginalizing over all other characteristics and state variables. For example: "A learner with higher persistence is less likely to exit the task as the number of previous attempts increases."

This marginalization is crucial for practical authoring. While it might be theoretically appealing to specify joint distributions over all learner characteristics simultaneously (capturing complex interactions between persistence, domain knowledge, affective states, and more), such specifications quickly become intractable. The number of possible combinations grows exponentially, and expert intuitions rarely extend to nuanced three-way or four-way interactions without empirical data. By focusing on one characteristic at a time, MDHyps let practitioners express confident beliefs about marginal relationships while leaving joint interactions to be determined by the LLM's compositional reasoning or by real data when available.

Because MDHyps express expected behavioral distributions, they enable distribution-level validation rather than subjective, instance-level assessments. Instead of asking whether individual simulated trajectories "look right" to human judges (a subjective, expensive, and often unreliable process) [144, 85], one can ask whether aggregate statistics computed over many simulation runs satisfy the functional relationships specified

in hypotheses. This shift to distributional testing makes validation systematic, falsifiable, and scalable.

4.2.2 Compositional Generalization Properties

For MDHyps to serve as a foundation for scalable simulation authoring, they must be composable: calibration achieved for one hypothesis should not break when hypotheses are added, swapped, combined, or modified. Without this property, practitioners would face the same exponential growth in calibration effort that plagues ad-hoc approaches; every change to the learner model would require recalibrating the entire system from scratch.

Consider a practitioner who calibrates a persistence hypothesis using "time elapsed" as the progression variable, then realizes "number of attempts" is more appropriate. Can they swap the variable and have calibration remain intact, or must they restart the entire calibration process? Similarly, if they add a second characteristic (e.g., geometry proficiency) with its own hypothesis, the two hypotheses should compose without interference, each contributing independently to the overall simulation behavior.

This "calibrate once, use many" property is the defining feature of compositional generalization [122, 204]. If achieved, it would transform simulation authoring from an iterative re-engineering exercise into a mix-and-match assembly process. Practitioners could build libraries of calibrated hypotheses, instantiate them for new characteristics by filling in variable values, and combine them into increasingly sophisticated learner models, all without redoing calibration from scratch at each step. This is essential for iterative design, where learning environment development involves constant refinement as theoretical understanding evolves and pilot data becomes available.

4.2.3 Evaluating Calibration: A Statistical Framework

To make compositional generalization testable rather than aspirational, we need a precise definition of what it means for calibration to "hold" across learner model edits.

The approach begins by grouping MDHyps into hypothesis classes based on shared functional relationships. For example, all hypotheses positing monotonic relationships

between a characteristic and action probabilities belong to the monotonic hypothesis class regardless of which specific characteristic or action set they reference.

Each hypothesis class is associated with three key components: (1) a prompt template with slots for variable values (characteristic names, action descriptions, parameters), (2) a statistical test appropriate for the functional relationship (Spearman correlation for monotonic hypotheses, chi-squared goodness-of-fit tests for uniform distributions), and (3) a statistical success criterion specifying when calibration holds: for monotonic hypotheses, Spearman correlation ρ significantly above zero ($p < 0.05$).

Template calibration is the human-in-the-loop process of iteratively refining the prompt template for a hypothesis class until a reference hypothesis from that class satisfies the success criterion. This process consumes expert time and computational resources, which is precisely why we want to do it as infrequently as possible.

The key test of compositional generalization is whether calibration holds after learner model edits. If we calibrate a template using reference hypothesis H_1 , then modify the learner model (e.g., swapping a variable in H_1 to create H_2 , or adding a new hypothesis H_3), calibration has held if H_2 or H_1 (with H_3 present) still satisfies the success criterion without re-engineering the template. By systematically testing calibration across a range of edit operations, the promise of compositional generalization can be empirically assessed in practice.

4.3 The HYP-MIX Framework

This section formalizes the HYP-MIX framework, detailing its core abstractions, authoring workflow, and design principles that enable systematic, composable calibration.

4.3.1 Core Abstractions

The HYP-MIX framework rests on three core abstractions that together enable systematic, composable simulation authoring.

MDHyps. As introduced earlier, an MDHyp is a natural-language statement relating a single learner characteristic to a probability distribution over actions. Formally, an MDHyp

specifies a functional relationship $f : V \times S \rightarrow \Delta(A)$, where V is the value space of the characteristic (e.g., persistence rated 1-10), S is the environment state space, and $\Delta(A)$ is a probability distribution over the action space A . The key is that this relationship marginalizes over all other learner characteristics; it describes how the focal characteristic influences action selection, averaging over the values of other characteristics the learner might possess.

In practice, MDHyps are authored in natural language rather than formal mathematics, making them accessible to practitioners without specialized training. Example MDHyps include: "A learner with higher persistence is less likely to exit the task as the number of previous attempts increases," or "A learner at the lower end of the geometry proficiency spectrum exhibits a uniform distribution over measurement actions." The natural language formulation serves as both specification (telling the LLM how to behave) and documentation (communicating intent to other practitioners).

Hypothesis Classes and Templates. To enable reuse, we group MDHyps into hypothesis classes $\mathcal{H}$ based on shared functional forms. All hypotheses positing monotonic relationships belong to $\mathcal{H}_{\text{monotonic}}$; those specifying uniform distributions belong to $\mathcal{H}_{\text{uniform}}$; and so forth. Each class is associated with a prompt template $\hat{I}_{\mathcal{H}}$ containing slots for variable values, and a success criterion $T_{\mathcal{H}}$ based on an appropriate statistical test.

For monotonic hypotheses, we use Spearman rank correlation with criterion:

$$T_{\text{monotonic}}(\rho, p_{\rho}) = \begin{cases} \text{true,} & \text{if } \rho > 0 \text{ and } p_{\rho} \leq 0.05 \text{ (increasing)} \\ \text{true,} & \text{if } \rho < 0 \text{ and } p_{\rho} \leq 0.05 \text{ (decreasing)} \\ \text{false,} & \text{otherwise} \end{cases}$$

For uniform distribution hypotheses, we use chi-squared goodness-of-fit testing with criterion $T_{\text{uniform}}(p_{\chi^2}) = \text{true}$ if $p_{\chi^2} > 0.05$.

Learner Models. A learner model $L = (C, V, M)$ consists of three components. C is the set of learner characteristics being modeled (e.g., {persistence, geometry proficiency}). V is a mapping from each characteristic $c_i \in C$ to its value v_i for the current simulated learner, typically on a numerical scale (e.g., $v_{\text{persistence}} = 7$ on a 1-10 scale). M is a mapping from each characteristic to its characteristic model m_i : a set of one or more MDHyps that specify how that characteristic influences behavior.

The simulation prompt is constructed by concatenating the templates of all MDHyps in all characteristic models, instantiated with their specific variable values, along with a description of the learning environment and the current state. This compositional structure is what enables mix-and-match authoring: adding a new characteristic means adding a new characteristic model to M ; refining understanding of a characteristic means updating its associated MDHyps.

Approximate Marginalization. A practical challenge is that testing an MDHyp by enumerating all possible states would require an intractable number of LLM queries (exponential in the number of state variables). This is addressed through statistical subsampling: for each characteristic value to be tested, a controlled subset of states is sampled (typically 10-20), the LLM is queried for action distributions in each state, and the results are aggregated. This approximation allows us to test marginal relationships between characteristics and actions without exhaustively exploring the full state space, making validation computationally feasible while preserving statistical power to detect calibration failures.

4.3.2 Authoring Workflow

With these abstractions in place, the HYP-MIX authoring workflow proceeds through four main phases that progressively refine theoretical understanding into testable, calibrated simulations.

Phase 1: Operationalization. The workflow begins with expert knowledge expressed as theoretical constructs or qualitative descriptions. A learning scientist might describe persistence as "maintaining sustained effort toward goal-directed tasks despite challenges," drawing on theoretical definitions from educational psychology. The first task is to operationalize this construct: translate it from abstract description into concrete, testable predictions about observable behavior in the specific learning environment at hand.

This operationalization produces one or more MDHyps. For persistence in a physics problem-solving environment, one might hypothesize: "A learner with higher persistence is less likely to exit the task prematurely as time passes." This statement is concrete (it specifies exit behavior), observable (we can measure exit events), characteristic-specific (it focuses on persistence), and testable (we can verify whether the monotonic relationship holds empirically). The operationalization process often reveals ambiguities in theoretical understanding; for instance, should persistence be measured via time elapsed or attempts made? Such clarifications strengthen both the theory and the simulation.

Phase 2: Template Calibration. Once we have operationalized a construct into MDHyps, we must calibrate the prompt template for each hypothesis class involved. This is the most labor-intensive phase, as it requires human-in-the-loop iteration. Taking a reference hypothesis from the class (e.g., the persistence-exit hypothesis for $\mathcal{H}_{\text{monotonic}}$), the template is instantiated, simulations are run across a range of characteristic values and sampled states, the statistical test is computed, and the success criterion is checked.

If calibration fails (perhaps the Spearman correlation is positive but not significant, or the relationship is weaker than expected), the template is refined. This might involve rephrasing instructions to emphasize the relevant behavioral pattern more explicitly, adding clarifying examples, or restructuring the template to reduce ambiguity. The refinement process draws on established prompt engineering techniques but grounds them in statistical validation rather than subjective judgment. Iteration continues until the

reference hypothesis satisfies its success criterion, at which point the template is considered calibrated for that hypothesis class.

Critically, calibration at the class level means this effort is amortized across all future hypotheses in the class. Once a calibrated template exists for $\mathcal{H}_{\text{monotonic}}$, it should be usable for any monotonic hypothesis (about any characteristic, any action set, any environment) without recalibration, assuming compositional generalization holds.

Phase 3: Instantiation. With calibrated templates available, creating new hypotheses becomes straightforward. A practitioner who wants to model geometry proficiency can author a new hypothesis: "A learner with higher geometry proficiency is more likely to make productive measurements." Recognizing this as another monotonic hypothesis, they instantiate the already-calibrated $\mathcal{H}_{\text{monotonic}}$ template by filling in the characteristic name ("geometry proficiency"), the action set (productive measurements), and any other necessary slots. No additional calibration should be required; the template was calibrated to handle any monotonic relationship, and the new hypothesis merely provides specific variable values.

This instantiation process dramatically reduces authoring effort compared to ad-hoc approaches. What previously required extensive prompt engineering and testing now involves filling in a few slots and running validation tests to confirm calibration has held. If tests reveal calibration has broken, that signals either a limitation of the current template (perhaps it does not generalize as broadly as hoped) or an opportunity to refine the template to cover a wider range of hypotheses.

Phase 4: Composition. Finally, multiple characteristic models (each consisting of one or more calibrated MDHyps) are composed into a unified learner model. The simulation prompt is constructed by concatenating all instantiated MDHyp templates along with environment and state descriptions. The LLM is then asked to roleplay a learner with the specified characteristics, using chain-of-thought reasoning [192] to explain its action choices

before outputting a selected action.

If compositionality holds, the combined prompt should produce behavior that respects all constituent hypotheses simultaneously, without recalibrating templates for every interaction. A learner with high persistence and low geometry proficiency should exhibit both low exit rates (from the persistence hypothesis) and uniform measurement distributions (from the geometry hypothesis), with the LLM implicitly managing how these constraints interact. This compositional reasoning is what enables scalable modeling; we do not need to explicitly specify every possible interaction between characteristics, only the marginal relationships we are confident about.

4.3.3 Design Principles

Experience with the HYP-MIX framework has revealed several design principles that guide effective authoring and calibration.

Balance Explicit Constraints with LLM Reasoning. The framework must balance explicit expert-specified constraints with implicit LLM reasoning, ensuring experts encode only the relationships requiring domain knowledge, while allowing the LLM to reason over scalable combinations. Too little explicit structure risks reverting to ad hoc prompting: brittle responses, inconsistent calibration, and limited interpretability. Too much explicit structure forfeits the benefits of LLMs’ commonsense reasoning about behavior, undermining the very capability that makes them attractive for simulation. In sensitive domains like education, a bias toward explicit authoring is prudent [80, 86], as LLMs struggle with certain reasoning tasks.

MDHyps represent one point in this design space. They explicitly specify marginal relationships between individual characteristics and actions (where domain expertise is essential), but they leave the LLM to reason about how those marginal constraints compose, how they interact with environment state, and how to resolve potential conflicts when multiple characteristics pull in different directions. This division of labor leverages

expert knowledge where it is most reliable (marginal behavioral tendencies grounded in domain knowledge) while delegating to the LLM where expert specification would be intractable (complex compositional interactions across scalable combinations).

Parsimony in Hypothesis Classes. Each new hypothesis class requires template calibration, which consumes expert time and computational resources. The framework should minimize the number of hypothesis classes while still spanning practitioner-relevant behavior patterns. We have found that two classes (monotonic relationships and uniform distributions) suffice for a surprising range of learner modeling tasks. Monotonic classes capture most directional hypotheses ("higher X leads to more/less Y"), while uniform distributions model the absence of strategic behavior for specific subpopulations (e.g., learners with minimal domain knowledge).

Resisting the temptation to create highly specialized classes is important. One might imagine separate classes for linear vs. logarithmic monotonic relationships, or for different flavors of non-uniform distributions. But each additional class multiplies calibration overhead and fragments the hypothesis library, reducing opportunities for reuse. The design principle is parsimony: add new classes only when existing ones genuinely cannot express needed behavioral patterns, not merely because more fine-grained control seems appealing in principle.

Flexible Template Fields. Within each hypothesis class, templates should be designed with flexible fields that can accommodate diverse instantiations without structural changes. A monotonic template that hardcodes "as time increases" would fail for hypotheses involving attempts, failures, or other progression metrics. Instead, templates should have slots for "progression variable" that can be filled with any relevant measure. Similarly, action set descriptions should support both positive framing ("likely to do X") and negative framing ("less likely to do Y"), directional modifiers ("more likely," "less likely"), and multiple action types.

This flexibility reduces the frequency with which templates must be modified, and thus recalibrated. When a template accommodates many variations through slot-filling alone, practitioners can author new hypotheses without triggering the expensive calibration process. Although flexibility increases template complexity upfront, it reduces long-term calibration and authoring cost—a justified trade-off.

Explicit Success Criteria. Finally, every hypothesis class must have explicit, quantitative success criteria based on appropriate statistical tests. Subjective judgments about whether calibration “looks good” are precisely what enable Clever Hans effects. By requiring that success criteria be specified upfront and evaluated mechanically, the framework enforces falsifiability. Either the statistical test passes or it does not; this eliminates post-hoc rationalization and enforces falsifiable calibration. This disciplined approach to validation is what distinguishes systematic calibration from elaborate prompt hacking.

4.4 Case Study: HoloOrbits Learning Environment

To evaluate whether HYP-MIX achieves its design goals, we conducted experiments in a physics learning environment that allows controlled testing of compositional generalization.

4.4.1 Experimental Setup

To rigorously evaluate whether HYP-MIX achieves compositional generalization, we need a controlled experimental environment with well-defined boundaries. We conducted our evaluation in HoloOrbits [151], an interactive physics learning environment designed to teach Kepler’s Laws of planetary motion through hands-on measurement and hypothesis testing (see Figure 4-1). HoloOrbits presents learners with simulated planetary systems and asks them to verify whether orbits are elliptical by taking distance measurements between key points (foci, aphelion, perihelion) and submitting arithmetic expressions that should be equal if Kepler’s First Law holds.

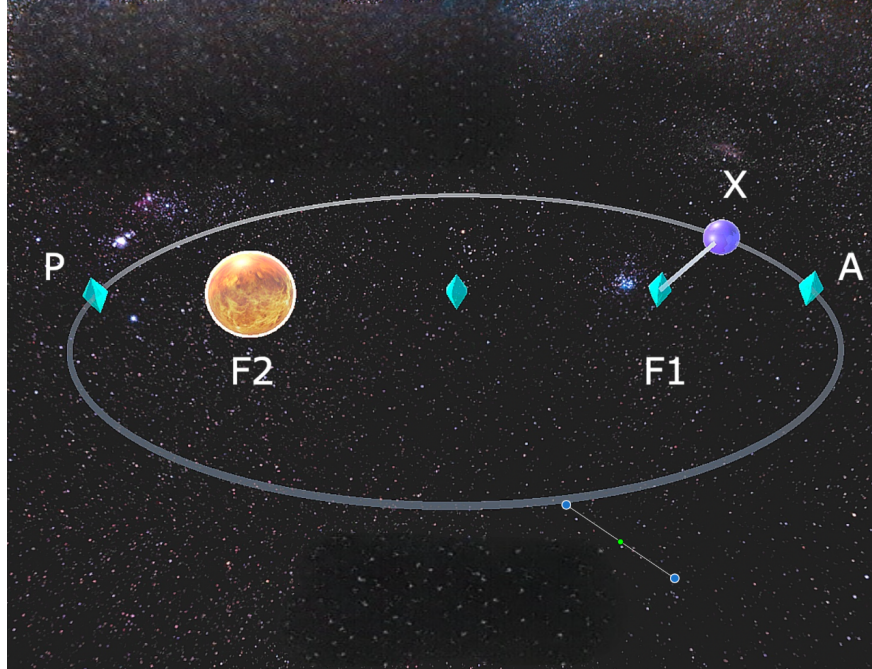


Figure 4-1. A screenshot of the HoloOrbits environment with key points annotated: aphelion (**a**), perihelion (**p**), focus 1 (**f1**), focus 2 (**f2**), and a fixed point on the orbit (**x**).

We selected HoloOrbits for several methodological reasons. First, it has compact, well-defined state and action spaces: small enough to make exhaustive testing computationally feasible, yet rich enough to exhibit interesting behavioral patterns. The state space consists of boolean variables indicating which measurements have been taken, plus integer variables tracking attempts and elapsed time. The action space includes approximately ten measurement actions (e.g., measuring distance between Focus 1 and Perihelion), a submission action for proposing solutions, and an exit action for abandoning the task. This compactness allows us to test hypotheses across many state configurations without prohibitive computational costs.

Second, HoloOrbits provides an open-ended learning environment with multiple valid solution paths—allowing hypothesis tests across diverse behavioral trajectories. This makes it a more realistic test bed than highly constrained tutoring systems where action sequences are largely predetermined. If compositional generalization succeeds in HoloOrbits, it provides evidence for viability in the broader class of open-ended learning

environments that educational technology increasingly emphasizes.

For our experiments, we modeled two learner characteristics chosen to represent complementary dimensions of learner variation. *Persistence* represents a psychological/affective characteristic: the tendency to maintain effort toward goals despite challenges [6, 75], operationally defined as resistance to premature task abandonment. *Geometry proficiency* represents domain knowledge: the ability to apply knowledge of properties of geometric shapes to solve problems [83], operationally defined as tendency to select measurements that are productive for verifying elliptical orbits. By including both a knowledge-based and a non-cognitive characteristic, we test whether compositional generalization extends across fundamentally different types of learner attributes.

We represented characteristic values on a 1-10 scale, with higher values indicating greater persistence or proficiency. This continuous scale allows us to test whether the LLM maintains appropriate functional relationships across the entire value range, not just at categorical extremes (high vs. low). For statistical testing, we typically sampled five values per characteristic ($v \in \{2, 4, 6, 8, 10\}$), chosen to span the range while leaving out extreme endpoints where ceiling/floor effects might confound interpretation.

4.4.2 The Learner Model Edit Graph

Rather than testing compositional generalization in the abstract, we structure our evaluation around concrete edit operations that practitioners perform when iteratively developing learner models. We represent these operations as a directed graph where nodes are learner model "snapshots" and edges are edit operations that transform one snapshot into another (see Figure 4-2). This Learner Model Edit Graph formalizes sequential calibration edits, serving as both the experimental manipulation and a model of realistic development practice.

The graph begins with an initial node containing two hypotheses operationalized from theoretical definitions of our focal characteristics:

- H_{P1} : "A learner with higher persistence is less likely to exit the task prematurely as the number of attempts increases." (monotonic, persistence)

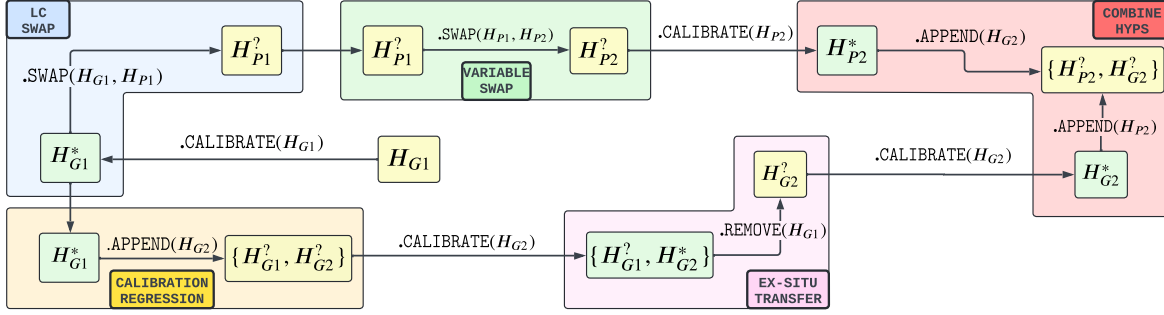


Figure 4-2. Learner Model Edit Graph used to evaluate LLM robustness across five distinct edit operations. Nodes represent learner model “snapshots” after developer edits, listing the MDHyps in each snapshot. Green nodes indicate calibrated snapshots; yellow nodes are untested for calibration. Each MDHyp in the learner model is annotated: ‘?’ for untested calibration status and ‘*’ for confirmed calibration. **(1) Ex-Situ Transfer:** Tests if a calibrated MDHyp alongside other MDHyps remains calibrated when tested alone. **(2) Combine Hypotheses:** Assesses stability of two separately calibrated hypotheses when combined. **(3) Variable Swap:** Swaps a single variable within a hypothesis. **(4) LC Swap:** Evaluates if a prompt template calibrated for one learner characteristic works for another in the same class. **(5) Calibration Regression:** Tests stability of a calibrated hypothesis when a new one is added.

- H_{G1} : “A learner with higher geometry proficiency is more likely to make productive measurements.” (monotonic, geometry proficiency)

Both hypotheses belong to $\mathcal{H}_{\text{monotonic}}$, which allows us to calibrate a single template and test whether it generalizes to both hypotheses despite their differing characteristics and action sets. We calibrate using H_{G1} as the reference hypothesis and then verify that H_{P1} also satisfies the monotonicity criterion (our first test of compositional generalization).

Five edit operations then branch from or extend this initial model, each representing a common refinement practitioners might make (see Table 4-1 for all hypotheses used and Figure 4-2 for the complete edit graph):

Variable Swap. After consulting with learning science experts, we determine that measuring persistence via “number of attempts” is more theoretically appropriate than “number of minutes elapsed”; attempts reflect active engagement with the task, while time

Table 4-1. Hypotheses used in learner modeling experiments. Regular text shows the template for each hypothesis class, with **blue** indicating specific slot values for each hypothesis.

Hypothesis ($H_i \in \mathcal{H}_{c(H_i)}$)	Initial, Uncalibrated Prompt Template $\hat{I}_{c(H_i)}(H_i)$
$H_{G1} \in \mathcal{H}_{\text{mono}}$	A learner with a higher geometry proficiency is more likely to make productive measurements (i.e., those that measure distances between pairs of points in the planetary system that are potentially useful to verify if the orbit is elliptical). To make productive measurements is to make one of the following actions: measure-f1-x, measure-f2-x, measure-f1-p, measure-f2-p, measure-a-f1, measure-a-f2 .
$H_{P1} \in \mathcal{H}_{\text{mono}}$	A learner with a higher persistence is less likely to abandon the task as the number of measurements increases (i.e., to prematurely exit the session before submitting the right solution). To abandon the task as the number of measurements increases is to make one of the following actions: exit .
$H_{P2} \in \mathcal{H}_{\text{mono}}$	A learner with a higher persistence is less likely to abandon the task as the time elapsed increases (i.e., to prematurely exit the session before submitting the right solution). To abandon the task as the time elapsed increases is to make one of the following actions: exit .
$H_{G2} \in \mathcal{H}_{\text{uniform}}$	As learners get closer and closer to the lower end of the geometry proficiency spectrum (value of 1), they are equally likely to perform the following actions. In other words, such a learner exhibits a uniform distribution over these actions: <all measurement actions> .

may elapse passively. We create a new hypothesis H_{P2} by swapping the progression variable in H_{P1} . If calibration holds, H_{P2} should satisfy the monotonicity criterion without template modification.

Append (New Hypothesis Class). During initial testing, we observe that simulated learners with minimal geometry proficiency (value 1/10) unexpectedly make productive measurements 80% of the time. This contradicts our expectation that learners lacking

spatial reasoning should distribute measurements uniformly rather than strategically (a phenomenon that Aher et al. (2023) [3] refer to as “hyper-accuracy distortion,” where LLMs struggle to feign ignorance about a topic to simulate human behavior). To address this, we introduce a new hypothesis class $\mathcal{H}_{\text{uniform}}$ and hypothesis H_{G2} : “A learner at the lower end of the geometry proficiency spectrum exhibits a uniform distribution over measurement actions.” This tests whether we can add entirely new hypothesis classes without disrupting existing calibration.

Combine Hypotheses. After separately calibrating H_{P2} and H_{G2} , we combine them into a single learner model. This tests the most challenging aspect of compositionality: do two independently calibrated hypotheses maintain their calibration when present simultaneously in the model, potentially competing for influence over the LLM’s behavior?

Ex-Situ Transfer. We test whether H_{G2} , initially calibrated in the presence of H_{G1} , maintains calibration when tested in isolation. This addresses whether hypothesis interdependencies emerge during calibration that prevent hypotheses from functioning independently.

Calibration Regression. Finally, we test whether adding H_{G2} to a model containing the already-calibrated H_{G1} causes H_{G1} ’s calibration to degrade. This regression test is critical: if every addition of a new hypothesis breaks existing calibration, compositional authoring becomes impossible.

Together, these five operations cover the primary ways practitioners refine learner models during iterative development. Success across all operations would provide strong evidence that compositional generalization is achievable. Failures would pinpoint specific edit types where current approaches fall short, informing future framework development.

4.4.3 Testing Calibration Under Model Edits

Our methodology for evaluating each edit operation follows a consistent before-and-after comparison structure designed to isolate the effect of the edit while controlling for confounding factors.

For each edge in the Learner Model Edit Graph, we conduct Pre-Op testing: we verify that all hypotheses in the source node satisfy their respective success criteria before the edit is applied. This establishes a calibration baseline against which later edits can be evaluated for degradation or preservation of calibration quality. We then apply the edit operation, creating the target node with its transformed learner model. Post-Op testing verifies whether affected hypotheses still satisfy their statistical success criteria, identifying whether the edit preserves or breaks calibration.

For monotonic hypotheses, we compute Spearman rank correlation ρ between characteristic values and empirical action probabilities aggregated over sampled states. For each characteristic value $v \in \{2, 4, 6, 8, 10\}$, we sample 15-20 environment states, query the LLM for action probabilities in each state, and average the probability of the focal action (e.g., exit) across states. This gives us five characteristic-action probability pairs, from which we compute ρ and its significance p_ρ . The success criterion $T_{\text{monotonic}}$ checks whether the correlation has the expected sign and reaches statistical significance ($p < 0.05$).

For uniform distribution hypotheses, we test whether action probabilities conform to a uniform distribution for learners at the specified end of the characteristic spectrum (typically value 1 or 2). We sample states, query the LLM for complete action distributions, aggregate across states for the focal characteristic value, and apply a chi-squared goodness-of-fit test comparing the empirical distribution to a uniform baseline. The success criterion T_{uniform} checks whether we fail to reject uniformity ($p_{\chi^2} > 0.05$), indicating no significant deviation from uniform behavior.

A critical methodological consideration is LLMs’ known sensitivity to arbitrary labeling choices. Synonyms for actions, different orderings, and even typo-like variations

can sometimes affect outputs unpredictably. To guard against the possibility that our results reflect fortuitous alignment with specific action labels rather than genuine calibration, we repeat all tests under three distinct action space labelings. For example, the exit action might be labeled **EXIT**, **QUIT**, or **STOP** across the three test rounds. If calibration holds across all three labelings, we have stronger evidence that the LLM has learned robust behavioral patterns rather than shallow lexical associations.

For each edit operation, we report results across these three labelings, noting whether calibration held, was lost, or showed partial success (maintained statistical significance but with degraded effect sizes). This fine-grained reporting allows us to distinguish complete failures from cases where compositional generalization mostly succeeds but shows stress under particular conditions.

4.5 Results

The experiments test whether GPT-4 Turbo maintains calibration across five types of learner model edits, providing insight into the viability of compositional simulation authoring.

4.5.1 Calibration Stability Across Edit Operations

The central empirical question is whether GPT-4 Turbo maintains calibration as learner models undergo the edits represented in our graph. Across five edit operations tested under three action space labelings each (yielding 18 total test cases), calibration held in 16 cases (89%). This high success rate provides strong initial evidence that compositional generalization is achievable with current frontier models, though the two failures reveal important boundaries. Table 4-3 presents the detailed results for all operations.

Monotonic Hypotheses: Strong Stability. The monotonic hypothesis class demonstrated particularly robust calibration. Across variable swaps (changing from "time elapsed" to "attempts made" as the progression measure), characteristic swaps (applying a persistence template to geometry proficiency), ex-situ transfer (testing hypotheses in isolation), and calibration regression (adding new hypotheses to models with existing ones),

Operation	Learner Model Transformation (Test MDHyp in Bold)	Success Criterion Function	Pre-Op Inputs for Test Results	Post-Op Inputs for Test Results	Is Calibration Held Post-Op?
EX-SITU TRANSFER	$\{H_{G1}^?, \mathbf{H}_{G2}^*\} \rightarrow \{\mathbf{H}_{G2}^?\}$	$T_{\text{uniform}}(P_{\chi^2})$ (Eq. 2)	$P_{\chi^2} = 0.98$ $P_{\chi^2} = 0.46$ $P_{\chi^2} = 0.22$	$P_{\chi^2} = 0.98$ $P_{\chi^2} = 0.46$ $P_{\chi^2} = 0.22$	Held Held Held
COMBINE HYPOTHESES	$\{\mathbf{H}_{P2}^*\} \rightarrow \{H_{P2}^?, \mathbf{H}_{G2}^?\}$	$T_{\text{mono}}(\rho, P_{\rho})$ (Eq. 1)	$\rho = -0.7, P_{\rho} = .02$ $\rho = -0.7, P_{\rho} = .02$ $\rho = -0.7, P_{\rho} = .02$	$\rho = -0.3, P_{\rho} = .41$ Constant series $\rho = -0.6, P_{\rho} = .02$	Lost Lost Yes
	$\{H_{G2}^*\} \rightarrow \{H_{P2}^?, H_{G2}^?\}$	$T_{\text{uniform}}(P_{\chi^2})$ (Eq. 2)	$P_{\chi^2} = 0.98$ $P_{\chi^2} = 0.46$ $P_{\chi^2} = 0.22$	$P_{\chi^2} = 0.98$ $P_{\chi^2} = 0.46$ $P_{\chi^2} = 0.22$	Held Held Held
VARIABLE SWAP	$\{H_{P1}^*\} \rightarrow \{\mathbf{H}_{P2}^?\}$	$T_{\text{mono}}(\rho, P_{\rho})$ (Eq. 1)	$\rho = -0.8, P_{\rho} < .01$ $\rho = -0.8, P_{\rho} < .01$ $\rho = -0.8, P_{\rho} < .01$	$\rho = -0.8, P_{\rho} < .01$ $\rho = -0.8, P_{\rho} < .01$ $\rho = -0.8, P_{\rho} < .01$	Held Held Held
LC SWAP	$\{\mathbf{H}_{G1}^*\} \rightarrow \{\mathbf{H}_{P1}^?\}$	$T_{\text{mono}}(\rho, P_{\rho})$ (Eq. 1)	$\rho = 0.6, P_{\rho} = .04$ $\rho = 0.6, P_{\rho} = .04$ $\rho = 0.7, P_{\rho} = .01$	$\rho = -0.8, P_{\rho} < .01$ $\rho = -0.8, P_{\rho} < .01$ $\rho = -0.8, P_{\rho} < .01$	Held Held Held
CALIBRATION REGRESSION	$\{\mathbf{H}_{G1}^*\} \rightarrow \{\mathbf{H}_{G1}^?, H_{G2}^?\}$	$T_{\text{mono}}(\rho, P_{\rho})$ (Eq. 1)	$\rho = 0.6, P_{\rho} = .04$ $\rho = 0.6, P_{\rho} = .04$ $\rho = 0.7, P_{\rho} = .01$	$\rho = 0.9, P_{\rho} < 7e-6$ $\rho = 0.8, P_{\rho} < .01$ $\rho = 0.7, P_{\rho} = .01$	Held Held Held

Figure 4-3. Results of statistical tests evaluating calibration stability across five learner model edit operations. Each operation was tested under three different action space labelings. Green shading indicates calibration was maintained; red shading indicates calibration was lost.

monotonic relationships maintained statistical significance in all cases involving H_{P1} , H_{P2} , and H_{G1} . Spearman correlations ranged from $|\rho| = 0.6$ to $|\rho| = 0.9$ with $p < 0.05$ in all 12 monotonic test cases, with effect sizes often strengthening post-edit rather than degrading.

This stability is noteworthy given the diversity of changes involved. Swapping "time" for "attempts" changes not just the variable name but its semantic character; time is continuous and passive, while attempts are discrete and reflect active engagement. Yet the template calibrated for one generalized to the other. Similarly, applying a persistence template to geometry proficiency crosses the boundary between non-cognitive and cognitive characteristics, suggesting the template has learned general monotonic reasoning rather than characteristic-specific patterns.

Uniform Distribution Hypotheses: Successful Calibration. The uniform distribution hypothesis class was introduced mid-development to address observed behavior

where low-proficiency learners made suspiciously strategic measurement choices. We calibrated a new template for H_{G2} specifying that learners with geometry proficiency values 1-2 should exhibit uniform action distributions. Chi-squared tests confirmed uniformity ($p_{\chi^2} > 0.10$ in all three action labelings), and this calibration held when H_{G2} was tested both ex-situ and in combination with other hypotheses. The ability to introduce an entirely new hypothesis class mid-development without recalibrating existing templates demonstrates meaningful modularity.

The COMBINE Operation: Partial Failure Analysis. The two calibration failures occurred when combining H_{P2} (persistence-exit hypothesis) with H_{G2} (geometry uniformity hypothesis). In two of three action labelings, H_{P2} lost statistical significance post-combination, with Spearman correlations dropping from $\rho = -0.7, p = 0.02$ to either $\rho = -0.3, p = 0.41$ (loss of significance), or producing a constant series (complete loss of signal). Only one labeling maintained calibration ($\rho = -0.6, p = 0.02$).

This pattern suggests that combining hypotheses (particularly those from different hypothesis classes that affect overlapping action sets) poses the greatest challenge to compositional generalization. One potential explanation is increased prompt complexity: when multiple hypotheses are present, the LLM must simultaneously satisfy more constraints, possibly leading to conflicts or confusion about which constraint should dominate in ambiguous situations. Another possibility is that the specific combination of persistence-based and proficiency-based constraints creates a theoretically ambiguous scenario: when a learner has low proficiency (triggering uniform behavior) but high persistence (triggering low exit rates), which pattern should prevail? Without real data to ground such interactions, the LLM may default to unpredictable behavior.

Despite these failures, the COMBINE results should not be interpreted as a fundamental failure of the approach. First, calibration still held in one of three labelings, suggesting the problem is not insurmountable. Second, other multi-hypothesis operations

(like ex-situ transfer and calibration regression) succeeded, indicating that not all compositions are equally problematic. The failures pinpoint a specific stress point: combining hypotheses from different classes that affect related action sets. Future work can address this through more sophisticated composition strategies or by collecting real data to resolve theoretically ambiguous interactions.

4.5.2 Analysis of LLM Behavioral Modeling Capabilities

These results have implications beyond the narrow question of whether HYP-MIX works; they inform our understanding of what frontier LLMs can and cannot do in the domain of behavioral simulation.

Compositional Reasoning Over Behavioral Specifications. The success of variable swaps, characteristic swaps, and hypothesis additions demonstrates that GPT-4 Turbo can maintain functional relationships specified at an abstract level even as concrete instantiations change. When we swap "time" for "attempts" in a persistence hypothesis, the model preserves the monotonic decrease in exit probability; it does not memorize a specific time-exit pattern but generalizes the underlying decreasing relationship. This is evidence of compositional reasoning: the LLM treats the hypothesis template as a specification of a behavioral principle that can be applied to new variable combinations.

Evidence Against Pure Memorization. A persistent concern with LLM-based simulations is that apparent success might reflect memorization of similar scenarios from training data rather than genuine behavioral reasoning. Our results argue against pure memorization through several lines of evidence. First, the specific learning environment (HoloOrbits) and task (verifying Kepler’s Laws through distance measurements) is sufficiently specialized that verbatim training examples are unlikely. Second, the compositional generalization we observe (maintaining calibration under variable swaps and characteristic additions) is difficult to explain via memorization alone, which would predict brittleness to such changes. Third, the partial failures under COMBINE operations suggest

the model is not simply regurgitating memorized patterns but struggling with genuine compositional challenges.

This does not mean memorization plays no role; LLMs surely draw on general knowledge about persistence, domain proficiency, and learning behavior from their training data. But the results indicate this knowledge is being composed and adapted rather than retrieved wholesale, supporting the viability of LLM-based simulation for novel scenarios.

Systematic Evaluation as a Safeguard Against Clever Hans Effects. Perhaps most importantly, the HYP-MIX methodology itself addresses Clever Hans concerns by making calibration falsifiable. Rather than asking whether individual simulation outputs “look right” to practitioners who designed the prompts (a subjective judgment vulnerable to confirmation bias), we require that statistical tests pass before and after explicit model transformations. This falsifiability is what distinguishes genuine calibration from elaborate prompt hacking.

The two failures under COMBINE exemplify this safeguard in action: our statistical tests detect calibration loss that might otherwise go unnoticed if evaluation relied on informal inspection. By establishing clear success criteria upfront and evaluating them mechanically, the framework prevents practitioners from unconsciously adjusting their standards to match observed outputs. This disciplined approach to validation is essential for building trust in LLM-based simulations.

4.5.3 Limitations of the Current Approach

While the results are encouraging, several important limitations constrain the generalizability of our findings and highlight areas for future work.

Limited Scope of Evaluation. Our evaluation covered two learner characteristics (persistence and geometry proficiency) in a single learning environment (HoloOrbits). While this focused scope enabled rigorous controlled testing, it leaves open questions about generalization. Will compositional calibration hold across ten characteristics? Twenty? Do

different types of characteristics (cognitive vs. affective vs. metacognitive) compose differently? How does the approach scale to environments with larger, more complex state and action spaces?

Similarly, we tested only two hypothesis classes (monotonic and uniform). Real learner modeling may require additional classes to capture non-monotonic relationships (e.g., inverted-U patterns where moderate values produce different behavior than extreme values), threshold effects (behavior changes sharply at specific characteristic values), or interaction effects (two characteristics jointly influence behavior in ways not predictable from their marginal relationships). Each additional class requires template calibration, and their composition may introduce new challenges.

Inability to Handle Complex Non-Linear Interactions. The marginal hypothesis framework assumes that behavioral patterns can be adequately specified by combining marginal relationships: that persistence affects exit behavior somewhat independently of geometry proficiency, with the LLM managing their interaction implicitly. This assumption may break down when characteristics interact in complex non-linear ways that experts can predict but cannot easily specify without data [94].

For example, low persistence might predict high exit rates except when combined with high domain knowledge and high stakes (a three-way interaction requiring specific value combinations to manifest). Specifying such interactions through natural language hypotheses is difficult, and verifying them statistically requires exploring high-dimensional characteristic spaces. When such interactions are theoretically important but empirically underspecified, calibration without real learner data becomes unreliable.

4.6 Discussion and Future Directions

With empirical evidence for compositional generalization in hand, we now reflect on what HYP-MIX accomplishes, what limitations remain, and how these findings prepare the foundation for fully automated calibration approaches.

4.6.1 Framework Contributions and Achievements

The empirical results demonstrate that HYP-MIX transforms simulation authoring from an ad-hoc craft into a systematic engineering discipline. Rather than iteratively tweaking monolithic prompts until outputs subjectively "look right," practitioners specify explicit hypotheses, calibrate reusable templates, and verify behavior through statistical tests with pre-defined success criteria. This systematicity makes the authoring process more transparent, reproducible, and falsifiable: properties essential for scientific applications of simulation.

The framework's key achievement is establishing compositional generalization as a concrete, measurable capability for evaluating LLMs in behavioral simulation tasks. Rather than vague claims that a model "understands" learner behavior, we can now ask: does the model maintain calibration under variable swaps, characteristic additions, and hypothesis combinations? This operationalization enables rigorous comparison across models, prompting strategies, and framework designs. Future work can use the Learner Model Edit Graph methodology to benchmark different LLMs, test alternative prompting approaches, or evaluate improvements to the framework itself.

4.6.2 Limitations and Outstanding Challenges

Despite these accomplishments, the need for manual template calibration remains the most significant barrier to widespread adoption and scalability of the approach. The calibration process is iterative, labor-intensive, and offers no guarantees of success. Practitioners must explore the space of possible prompt templates through trial and error, guided only by intuition about what phrasing might work and feedback from statistical tests about what has not worked so far. Each iteration requires running many LLM queries (typically 50-100 for a single hypothesis across sampled states and characteristic values), waiting for results, computing statistical tests, diagnosing why calibration failed, hypothesizing template improvements, and repeating. The process can take hours or days per hypothesis class, consuming both expert time and computational resources.

More fundamentally, there is no theoretical guarantee that a calibratable template exists for a given hypothesis class. The relationship between natural language prompt phrasing and LLM output distributions is complex and poorly understood. Success depends on prompt engineering skill, which remains more art than science. Two practitioners working on the same hypothesis class might produce very different templates; one might succeed where the other fails entirely, not because one template is objectively better, but because they happened to stumble on phrasing that resonates with the model’s internal representations.

This manual overhead becomes especially problematic as the number of hypothesis classes grows. While compositional generalization amortizes effort within classes, it does nothing to reduce the upfront calibration cost per class. The total overhead scales linearly with the number of classes, potentially making comprehensive learner modeling prohibitively expensive. This gap motivates exploring fundamentally different approaches where behavioral specifications can be satisfied without human-in-the-loop prompt engineering.

4.6.3 Implications for Automated Calibration Methods

The persistence of manual calibration overhead suggests a fundamental limitation of prompt-based approaches: they require human expertise to navigate the complex relationship between natural language specifications and LLM behavior. This limitation becomes more pronounced as the number of hypothesis classes grows, with calibration effort scaling linearly despite compositional benefits within classes.

These findings point toward a different approach: treating calibration as a constraint satisfaction problem rather than a prompt engineering challenge. If behavioral priors can be expressed as explicit constraints on emergent outcomes rather than implicit guidance through natural language, automated methods might systematically search for agent behaviors that satisfy these constraints without requiring human-in-the-loop template refinement.

Such constraint-based approaches would need to address several technical challenges. First, they must translate expert knowledge about behavioral patterns into formal constraint specifications that can be automatically evaluated. Second, they require search methods capable of exploring the space of possible agent behaviors to find configurations that satisfy multiple constraints simultaneously. Third, they must preserve the systematic validation properties that make HYP-MIX more reliable than ad-hoc approaches.

The compositional generalization demonstrated in this chapter provides a foundation for such automated approaches. If LLMs can maintain behavioral relationships across model edits when given appropriate specifications, they might also maintain those relationships when specifications are generated through automated constraint satisfaction rather than manual prompt engineering.

4.7 Contributions

This chapter provides systematic foundations for LLM-based learner simulation authoring through the HYP-MIX framework. The findings establish key insights about compositional generalization and motivate the need for automated calibration approaches.

1. **Empirical Evidence for Compositional Generalization.** Controlled experiments in the HoloOrbits learning environment provide the first rigorous evidence that compositional generalization is achievable with frontier LLMs. GPT-4 Turbo maintained calibration across 89% of learner model edits (16 of 18 test cases), demonstrating that "calibrate once, use many" authoring substantially reduces iteration costs compared to ad-hoc approaches. The systematic evaluation methodology guards against Clever Hans effects through falsifiable statistical criteria.
2. **MDHyps as Modular Units of Expert Knowledge.** Marginalized Distributional Hypotheses provide a principled formalization of behavioral priors that balances expressiveness with tractability. By specifying marginal relationships between individual characteristics and action distributions, MDHyps make expert

knowledge explicit and testable while keeping authoring cognitively manageable. The modular structure enables mix-and-match assembly of calibrated components.

3. Systematic Framework for Hypothesis-Based Calibration. The HYP-MIX

framework transforms simulation authoring from ad-hoc prompt engineering into systematic, validated practice. Hypothesis classes with reusable templates reduce—though do not eliminate—manual calibration overhead. The framework establishes a methodological template for evaluating LLM capabilities in behavioral modeling tasks across domains.

4. Identification of Remaining Automation Challenges. The persistence of

manual template calibration and partial failures under hypothesis combination reveal specific boundaries of current systematic approaches. These findings illuminate the technical requirements for fully automated calibration: constraint-based specification methods, systematic search over behavioral configurations, and preservation of falsifiable validation criteria.

4.8 Conclusion

This chapter introduced HYP-MIX, a systematic framework for authoring LLM-based learner simulations through hypothesis-based calibration. The framework advances simulation authoring from ad-hoc prompt engineering toward systematic, validated practice, with empirical evidence demonstrating that compositional generalization is achievable with current frontier models.

The key insight is that behavioral priors can be expressed as modular, testable hypotheses that compose across learner model edits, enabling "calibrate once, use many" authoring workflows. However, the persistence of manual template calibration reveals that systematization alone is insufficient for truly scalable simulation authoring. The partial failures under hypothesis combination, combined with the linear scaling of calibration overhead with hypothesis classes, point toward fundamental questions about automated calibration.

This motivates Chapter 5: expressing behavioral priors as trajectory-level constraints rather than prompt-level specifications, enabling systematic search and eliminating human-in-the-loop prompt engineering. This vision demands frameworks that leverage explicit constraint satisfaction to generate calibrated behaviors while preserving the systematic validation and falsifiability that distinguish principled calibration from sophisticated prompt hacking. Achieving this automation would make simulation-based design truly scalable, transforming how educational technology practitioners explore and evaluate learning environments before deployment.

CHAPTER 5

EVALUATING BELIEF-BEHAVIOR CONSISTENCY IN LLM-BASED SIMULATIONS

As discussed in Chapter 4, results from my prior work on HYP-MIX [123] shows promise for structuring behavioral specifications and reducing calibration overhead through compositional generalization. However, that framework still requires substantial manual effort for template calibration, with no guarantee of success. The path forward identified in that work requires investigating whether behavioral priors can be expressed as trajectory-level constraints rather than natural language specifications, enabling systematic search methods that eliminate manual prompt engineering entirely. Moving toward fully automated calibration raises a critical question: can LLMs reliably reason about the relationship between instantaneous agent actions and emergent population-level outcomes? Any move toward fully automated calibration rests on a fundamental assumption: that LLMs can accurately predict how individual agent specifications will translate into aggregate population patterns [169, 174].

To automate calibration, systems must predict whether agent configurations will satisfy trajectory-level constraints (aggregate patterns like “30% of time spent in explanation episodes” or “balanced participation between partners”) before running expensive simulations [40, 186]. This prediction capability is essential because planners need to evaluate whether proposed trajectories will achieve target outcomes, and authors need confidence that specified behaviors will produce intended aggregate effects [15, 158].

This chapter¹ describes a validation framework that tests belief-behavior consistency at two complementary levels. At the population level, models’ beliefs about how attributes correlate with behaviors are elicited, then compared to actual simulated outcomes. At the individual level, agents are asked to forecast their own future actions across multiple rounds, testing temporal consistency and planning capabilities. The systematic inconsistencies discovered regarding effect sizes, controllability, and forecasting accuracy

¹ This chapter is based on work published in collaboration with Adam Davies, Guohao Li, Kristy Elizabeth Boyer, ChengXiang Zhai, Bonnie J Dorr, and Francesco Pinto.

reveal fundamental limitations in current approaches to behavioral simulation that have important implications for how robust calibration systems should be designed.

The framework is instantiated using the Trust Game [19], a canonical behavioral economics paradigm that provides a well-established foundation for studying interpersonal trust through economic decisions. In the Trust Game, two players (a Trustor and a Trustee) engage in a sequential exchange: the Trustor receives an endowment and decides how much to send to the Trustee, the sent amount is tripled, and the Trustee then decides how much to return. This structure creates a social dilemma that distills complex trust dynamics into quantifiable choices, making it ideal for testing whether LLMs can accurately predict how individual agent specifications translate into aggregate population patterns. The game’s quantitative nature (decisions measured in dollars sent and returned) enables rigorous statistical comparison of beliefs and behaviors, while its extensive empirical foundation in psychology and behavioral economics provides ground truth against which LLM-generated behaviors can be validated.

5.1 Motivation

Agent-based simulations translate moment-to-moment decisions into emergent outcomes at the population level. A single agent’s choice to send \$5 versus \$3 in a Trust Game is an instantaneous action, but when aggregated across hundreds of diverse personas, these choices produce population-level patterns: correlations between age and trust behavior, distributions of cooperation rates, or temporal dynamics of reciprocity. For calibration to be useful, researchers must anticipate these aggregate patterns before generating large synthetic datasets. This exposes a fundamental micro-macro gap: authors can specify local behaviors through persona descriptions and prompts, but the downstream effects on aggregate statistics are not obvious without a principled way to predict them in advance.

5.1.1 From Instantaneous Actions to Population-Level Patterns

The challenge of bridging instantaneous actions and emergent outcomes is not unique to LLM-based simulations; it has long been central to agent-based modeling more broadly [10, 53]. However, the adoption of LLMs as role-playing agents introduces a new twist: these models encode vast world knowledge about human behavior, personality traits, and social dynamics. This raises a natural question: can this knowledge be leveraged not only to *generate* individual actions, but also to *predict* the aggregate patterns those actions will produce?

If LLMs could reliably reason about how individual-level specifications translate into population-level outcomes, they would serve a dual purpose: both as behavioral engines that enact personas and as predictive tools that forecast emergent patterns. This capability would be especially valuable for automated calibration, where planning systems must evaluate whether proposed agent configurations will satisfy target constraints before committing to expensive simulations. The planner needs to know, for instance, whether increasing the proportion of agents with high openness will shift the distribution of trust behavior in the desired direction, and by how much.

5.1.2 Language Models and Population-Level Reasoning

Prompt-based role-playing implicitly assumes that LLMs can reason at multiple levels simultaneously. At the individual level, the model must answer: “How would this particular persona act in this situation?” At the population level, it must answer: “How do personas with attribute X systematically differ from those with attribute Y?” The success of recent work on persona-conditioned generation [188, 187, 85] suggests that models can handle the individual-level question with some fidelity. But the population-level question (predicting correlations and effect sizes across diverse personas) remains largely untested.

This distinction matters because automated calibration systems rest on a strong assumption: that LLMs’ stated beliefs about how attributes relate to behaviors will align with the actual patterns observed when those agents are simulated. If that assumption

held reliably, eliciting a model’s beliefs could serve as a low-cost, pre-simulation predictor of emergent patterns. We could ask the model, “How does age affect trust behavior?” and use its response to guide agent configuration before running any simulations. But if the assumption fails, meaning stated beliefs diverge systematically from enacted behaviors, then calibration systems that rely on implicit LLM reasoning will be fragile and unreliable.

5.1.3 Existing Validation Methods

Most evaluations of role-playing agents are fundamentally post-hoc. Researchers generate synthetic data, then compare outputs to human benchmarks or qualitative expectations only after simulation completes [146, 9, 196]. This reactive approach is both expensive and risky. Misalignment is discovered after computational resources are spent, after time is invested in analysis, and most critically after flawed patterns may have already influenced downstream conclusions. If the simulated correlation between personality and behavior differs substantially from the model’s stated beliefs, authors have no early warning and discover the discrepancy only by inspecting completed datasets.

What is missing is a pre-simulation validation method that can surface inconsistencies between LLMs’ stated beliefs and their likely behavioral outputs *before* committing to large-scale generation. Such a method would provide an early warning system, flagging potential misalignment when it can still be addressed through revised prompts, alternative models, or explicit constraints. This is especially important for planning-based calibration, where the planner’s reasoning depends on predictions about how configurations map to outcomes.

5.1.4 Research Questions and Contributions

A validation framework is developed that tests belief-behavior consistency: the alignment between what LLMs say will happen and what actually happens when those same models role-play agents. This investigation addresses four central questions:

1. **Can LLMs’ beliefs about emergent behaviors be elicited before simulation?** Structured prompts are developed that query models about

population-level relationships (e.g., “How does age affect trust behavior?”) and individual-level forecasts (e.g., “What will this agent do over the next five rounds?”).

2. **Do these beliefs align with actual simulated outcomes?** Consistency is measured at two levels (population correlations and individual trajectories) using objective metrics (Spearman correlation, effect size agreement, mean absolute error).
3. **What factors affect belief-behavior consistency?** Elicitation design is systematically varied (with or without task context, targeting behavioral outcomes versus psychological constructs) and forecasting horizon (1-round versus 5-round lookahead) to understand when and why alignment breaks down.
4. **Can researcher-defined priors be imposed to control simulations?** Two forms of in-context conditioning are tested: self-conditioning that reinforces the model’s native beliefs, and imposed priors that override model defaults with researcher-specified relationships.

The systematic inconsistencies discovered regarding effect sizes, controllability, and forecasting accuracy reveal fundamental limitations in LLMs’ ability to reason about emergent outcomes. These findings suggest the need for explicit constraint specifications and formal planning mechanisms, rather than relying on implicit LLM reasoning alone.

5.2 Belief-Behavior Consistency Framework

Having established the need for pre-simulation validation, this section presents a technical framework for measuring belief-behavior consistency: the alignment between what models predict will happen and what actually occurs when those same models role-play agents. The core insight is that before engaging in role-play, LLMs can be queried about their expectations regarding simulation outcomes, then these stated beliefs can be compared to observed behaviors. This comparison reveals whether models possess coherent

internal representations that govern both their explicit reasoning and their enacted behaviors.

5.2.1 Methodology for Belief Elicitation

The framework operationalizes belief-behavior consistency through a two-phase process. In the *belief elicitation phase*, the model is presented with targeted prompts that query its expectations about how personas will behave. These prompts are carefully designed to match the structure of subsequent simulations: if diverse personas varying in age, personality traits, and social characteristics are planned for simulation, beliefs are elicited about how these same attributes correlate with behavioral outcomes. Critically, belief elicitation occurs *before* any role-playing, ensuring that the model cannot simply rationalize observed behaviors post-hoc.

In the *simulation phase*, agents are instantiated using the same model and persona specifications, then their actual behaviors are observed across multiple episodes. By comparing pre-stated beliefs to realized outcomes, consistency can be quantified: do models’ explicit predictions align with their implicit behavioral tendencies? High consistency would suggest that the model possesses a coherent internal representation linking attributes to behaviors; low consistency would reveal fragmentation or context-dependence in how knowledge is accessed and applied.

This framework enables early detection of misalignment. If a model states that age should strongly predict trust behavior but then generates simulations where age effects are weak or reversed, intervention can occur before investing resources in large-scale generation. The framework thus serves as a diagnostic tool, flagging potential issues when they can still be addressed through prompt refinement, model selection, or architectural changes.

5.2.2 Implications for Planning-Based Calibration

The motivation for developing this framework extends beyond evaluation: it directly informs the design of planning-based calibration systems. Automated planners must reason about *trajectory-level constraints* (aggregate properties like “30% of interactions should

involve explanatory dialogue” or “participation rates should be balanced within 10%”) that arise from instantaneous agent actions. To construct plans that satisfy these constraints, planners must predict how different agent configurations will behave at scale.

If LLMs could reliably reason about emergent outcomes, planners could simply query the model: “Will this configuration satisfy the target constraints?” and trust the response. But if belief-behavior consistency is low, such predictions become unreliable. The planner might believe a configuration will work based on the model’s stated beliefs, only to discover after expensive simulation that actual behaviors diverge substantially. This fragility undermines the entire automated calibration approach.

Understanding these limitations informs how planners should be designed. Rather than relying solely on implicit LLM reasoning, robust systems should incorporate explicit constraint-checking mechanisms, formal verification of planned trajectories, and fallback strategies for when predictions fail. The empirical measurement of belief-behavior consistency thus validates a core design principle: separate specification (what outcomes are desired) from prediction (whether a configuration will achieve them) and from execution (actually running the simulation).

5.2.3 Population-Level Belief-Behavior Consistency

We analyze consistency at two complementary levels, beginning with population-level relationships. Here, the goal is to test whether models accurately predict how persona attributes correlate with behavioral outcomes across a diverse population. We focus on *marginalized* relationships: holding all other attributes constant, how does varying a single attribute (e.g., age, extraversion, or education level) affect behavior?

The elicitation process queries the model about these relationships directly. For instance, queries might ask: “Rank the following age groups from least to most trusting: 18-25, 26-40, 41-60, 61+.” Or, more precisely: “For each age group, estimate the mean and standard deviation of dollars sent in a Trust Game.” These queries target the same relationships that will later be observed in simulation, ensuring an apples-to-apples

comparison.

After eliciting beliefs, a population of agents with systematically varied attributes is simulated and actual behavioral patterns are measured. Consistency is quantified using two complementary metrics. *Ranking consistency* (Spearman’s ρ [1]) captures whether the model correctly predicts the ordinal relationship: does it at least get the direction right? *Effect size consistency* compares the magnitude of effects, measuring agreement in η^2 [62] between believed and observed effect sizes. A model might correctly predict that age affects trust behavior (high ranking consistency) but substantially overestimate how strong that effect is (low effect size consistency).

This dual-metric approach is essential because realistic simulations require more than directional accuracy. If the model believes age accounts for 40% of variance when the true effect size is 5%, the resulting synthetic data will exhibit unrealistic patterns despite getting the sign right. For calibration purposes, effect size matters as much as direction, if not more.

5.2.4 Individual-Level Belief-Behavior Consistency

While population-level analysis tests aggregate predictions, individual-level analysis examines whether agents can forecast their own future actions with consistency over time. This measures a different aspect of coherence, namely temporal stability and multi-step planning capability. If an agent cannot accurately predict what it will do several steps ahead, this reveals limitations in long-horizon reasoning that directly impact sequential decision-making scenarios.

The individual-level protocol asks agents to forecast their own future actions before enacting them. For instance, in a multi-round Trust Game, prompts might ask: “Looking ahead, what amounts will you send to your partner in each of the next five rounds?” The agent provides numerical forecasts, then those rounds are actually simulated and the agent’s enacted choices are observed. Consistency is measured via mean absolute error (Mean Absolute Error (MAE)) between forecasted and enacted values.

This approach tests planning capabilities under known dynamics: the agent understands the game structure and knows approximately how its opponent will respond, yet must reason about its own future behavior. Systematic forecasting errors reveal where models struggle, such as whether they fail to account for how their own behavior will change based on observed outcomes, or whether they lack a coherent forward model of their decision-making process. These questions are especially relevant for planning-based calibration, where agents must reason about sequences of actions and their cumulative effects.

By combining population-level and individual-level analysis, our framework provides comprehensive coverage of the belief-behavior consistency landscape. Population analysis tests whether models correctly encode general relationships, while individual analysis tests whether they can apply that knowledge coherently over time. Together, these measurements reveal both what models know and how reliably they can reason about that knowledge.

5.3 Experimental Testbed

To test belief-behavior consistency rigorously, an experimental environment is required that balances ecological validity with analytical tractability. The environment must be rich enough to exhibit meaningful behavioral variation, yet structured enough to support precise measurement and statistical analysis. It should allow controlled manipulation of key variables while remaining interpretable to both models and researchers. The framework is instantiated using the Trust Game, a canonical behavioral economics paradigm that meets these requirements while providing a well-established foundation for interpersonal behavior research.

5.3.1 The Trust Game as Testbed

The Trust Game [19] offers a simplified yet behaviorally rich environment for studying interpersonal trust through economic decisions. In its basic form, two players (a Trustor and a Trustee) engage in a sequential exchange involving real monetary stakes. The

game distills complex trust dynamics into quantifiable choices: how much money to risk, how much to return, and how these decisions evolve over repeated interactions.

This paradigm has been extensively studied in psychology and behavioral economics over three decades [54], producing a robust literature on factors that influence trust behavior: personality traits [54], demographic characteristics [63], prior experiences [25], and cultural contexts [32]. This deep empirical foundation provides ground truth against which LLM-generated behaviors can be validated. Moreover, the game’s structure naturally supports the kinds of manipulations the framework requires: persona attributes can be systematically varied while holding game mechanics constant, and opponent strategies can be precisely controlled to isolate forecasting capabilities.

Recent work has begun using the Trust Game to evaluate LLM-based agents [197, 195], but these studies focus primarily on whether models exhibit human-like behavior in aggregate. The contribution here is different: the Trust Game is used not just to measure behavioral realism, but to probe the relationship between what models *say* will happen and what *actually* happens when they role-play. The game’s quantitative nature (decisions measured in dollars sent and returned) enables rigorous statistical comparison of beliefs and behaviors.

5.3.2 Game Structure and Simulation Setup

The Trust Game unfolds across multiple rounds, each following an identical structure. At the beginning of round t , the Trustor receives an endowment E (in our experiments, \$10). The Trustor must decide how much to send to the Trustee: an amount $s_t \in [0, E]$. This amount is then tripled by the experimenter, so the Trustee receives $3s_t$. The Trustee then decides how much to return: an amount $r_t \in [0, 3s_t]$. The round ends with payoffs: the Trustor receives $E - s_t + r_t$, and the Trustee receives $3s_t - r_t$.

This structure creates a social dilemma. Sending money demonstrates trust (the Trustor risks loss in hopes of mutual gain) but leaves the Trustor vulnerable to exploitation if the Trustee returns nothing. Trustee reciprocity demonstrates trustworthiness and

enables both players to benefit, but purely self-interested Trustees would keep everything. The interplay between trust and reciprocity produces rich behavioral dynamics that reflect real-world cooperation challenges.

In the simulations, the Trustor is role-played using LLMs conditioned on diverse synthetic personas. Each persona includes demographic attributes (age, gender, education, occupation), personality traits based on the Big Five model [50], and social characteristics (relationship history, general trust disposition). These personas are drawn from an augmented version of the GENAGENTS dataset [145], expanding it with additional personality and social dimensions to induce sufficient behavioral variance. Prior work has shown that personality traits significantly affect trust behavior [17, 168], making them natural targets for belief elicitation and consistency testing.

For each persona, a natural language description is generated that the model can process, such as: “You are Alex, a 32-year-old software engineer with a master’s degree. You score high in openness and conscientiousness but moderate in extraversion. You’ve had mostly positive experiences with collaborative projects and generally believe that people can be trusted, though you remain cautious in new situations.” The LLM then role-plays this persona across multiple game rounds, making sending decisions that reflect both the persona’s attributes and the unfolding game dynamics.

5.3.3 Trustee Archetypes for Individual-Level Evaluation

A key methodological challenge in multi-round forecasting is environmental stochasticity. If the Trustee’s behavior varies unpredictably, forecasting errors that stem from the agent’s limitations cannot be disentangled from those caused by opponent unpredictability. To isolate the agent’s forecasting ability, Trustee responses are fixed to simple, deterministic archetypes that follow known strategies.

We define three Trustee archetypes representing low, medium, and high reciprocity:

- **Low reciprocity:** Returns \$1 regardless of amount sent
- **Medium reciprocity:** Returns \$3 regardless of amount sent

- **High reciprocity:** Returns \$5 regardless of amount sent

These flat-return strategies are intentionally simple, allowing the Trustor agent to learn the pattern quickly and make informed forecasts. A rational agent facing a known strategy should be able to predict its own future behavior with reasonable accuracy, since the environment dynamics are fully specified. Any systematic forecasting errors therefore reveal limitations in the agent’s internal consistency and planning capabilities, rather than confusion about the environment.

This design choice prevents combinatorial explosion while maintaining experimental control. Rather than exploring all possible Trustee strategies such as proportional returns, random fluctuations, or conditional strategies, a minimal set is used that provides clear behavioral signals. The three archetypes span the plausible range of reciprocity, enabling tests of whether forecasting accuracy varies with environmental generosity without introducing confounding complexity.

5.3.4 Models Evaluated

We focus our evaluation on three open-source models that represent the current state of accessible, transparent language models: Llama 3.1 in 8B and 70B parameter versions [64], and Gemma 2 27B [175]. These models offer several advantages for belief-behavior consistency research. Their inference can be run locally with full control over sampling parameters, their outputs can be precisely logged for analysis, and their behavior can be studied without rate limits or API restrictions that would complicate large-scale evaluation.

All models are deployed with temperature sampling ($T = 0.7$) to introduce realistic behavioral stochasticity, as personas are expected to exhibit some variance across episodes rather than deterministic responses. However, structured output parsing via Pydantic [44] is used to ensure that generated text conforms to expected formats such as numerical dollar amounts for game decisions and ordinal rankings for belief elicitation. This combination of controlled stochasticity and structured parsing mirrors how these models would likely be deployed in production agent-based simulation systems.

These models are evaluated across multiple experimental conditions: different belief elicitation strategies (with and without task context, targeting behavioral outcomes versus psychological constructs), multiple forecasting horizons (1-round through 5-round lookahead), and various controllability interventions (self-conditioning and imposed priors). This comprehensive evaluation reveals how belief-behavior consistency varies not just across models, but across different ways of querying and conditioning those models (a crucial insight for practitioners designing calibration systems).

5.4 Results

The experiments test belief-behavior consistency across multiple dimensions: population-level relationships, controllability through interventions, and individual-level temporal forecasting. The findings reveal systematic limitations in LLMs’ ability to reason about emergent outcomes that have direct implications for automated calibration systems.

5.4.1 Population-Level Consistency

With the experimental testbed established, attention now turns to the first major empirical question: can LLMs accurately predict population-level relationships between persona attributes and behavioral outcomes? This subsection examines how models’ beliefs about aggregate patterns are elicited, measures the consistency between those beliefs and simulated behaviors, and explores factors that affect alignment.

5.4.2 Belief Elicitation Strategies

How questions are asked matters. The format and framing of belief elicitation prompts can substantially affect what knowledge models access and how they represent relationships. To understand these effects, three elicitation strategies are compared that systematically vary along two dimensions: whether task context is provided, and what type of prediction is targeted.

The first strategy, **NOCTX+TR**, asks models to rank how demographic or personality attributes correlate with interpersonal trust (a general psychological construct) without any reference to the Trust Game. For example: “Rank the following age groups

from least to most trusting: 18-25, 26-40, 41-60, 61+.” This tests whether models can access general knowledge about trust patterns from their training data.

The second strategy, **CTX+TR**, provides identical ranking tasks but first presents complete Trust Game instructions, including payoff structures, decision sequences, and the meaning of sending money. This tests whether grounding queries in specific task contexts improves consistency: does knowing the exact decision environment help models make better predictions?

The third strategy, **CTX+\$**, goes further by asking for quantitative behavioral predictions rather than construct rankings. With full game context, queries ask: “For each age group, estimate the mean and standard deviation of dollars sent in the first round.” This strategy targets the actual behavioral outcomes that will be observed, potentially enabling tighter alignment between beliefs and behaviors.

These three strategies probe a fundamental question for calibration systems: should beliefs be elicited about abstract constructs (trust) or concrete behaviors (dollars sent)? Should full task specifications be provided or should general knowledge be relied upon? The answer matters for practical deployment: if context provision is necessary for consistency, then belief elicitation becomes more complex and task-dependent. If construct-level queries work as well as outcome-level queries, then elicitation can be more general and reusable across tasks.

5.4.3 Empirical Results on Consistency

The empirical results reveal a more complex picture than simple “consistent” or “inconsistent” labels would suggest. Consistency varies across multiple dimensions including which aspect is measured, how beliefs are elicited, and which attributes are queried, in ways that expose the fragmented nature of LLMs’ internal representations. Table 5-1 reports the median Spearman correlation and median absolute effect size difference for each LLM and elicitation strategy, using the median to reduce sensitivity to outlier attributes.

First, providing task context does not systematically improve consistency.

Table 5-1. Population-level belief-behavior consistency across models and elicitation strategies. Median Spearman correlation (ρ) measures ranking consistency; median absolute effect size difference ($|\Delta\eta^2|$) measures effect size consistency. Lower $|\Delta\eta^2|$ indicates better effect size alignment.

Model	Elicitation Strategy	Median Spearman ρ	Median $ \Delta\eta^2 $
Llama 3.1 8B	NoCTX+TR	0.40	0.15
	CTX+TR	0.45	0.12
	CTX+\$	0.50	0.18
Llama 3.1 70B	NoCTX+TR	0.50	0.10
	CTX+TR	0.55	0.08
	CTX+\$	0.60	0.15
Gemma 2 27B	NoCTX+TR	0.40	0.12
	CTX+TR	0.35	0.10
	CTX+\$	0.45	0.20

One might expect that grounding belief elicitation in specific game mechanics (CTX+TR and CTX+\$) would help models reason more accurately than abstract queries about trust (NOCTX+TR). However, no consistent advantage is observed for context-rich elicitation. In some cases, providing context slightly improves ranking consistency, in others it degrades alignment, and in many cases differences are negligible. This contradicts intuitions about contextual grounding and suggests that models’ internal representations of behavioral relationships are not necessarily tied to specific task implementations [174].

Second, elicitation target substantially affects *which* aspect of consistency improves. When beliefs about behavioral outcomes are elicited (CTX+\$), better ranking consistency is observed: models more accurately predict the ordinal relationships between attribute levels. But when beliefs about psychological constructs are elicited (NOCTX+TR and CTX+TR), effect size consistency improves: models’ estimates of relationship strength align better with observed effect magnitudes. This dissociation suggests that models encode relationships differently depending on query framing [133, 114].

This finding has important implications: there is no single “correct” way to elicit beliefs that optimizes all aspects of consistency simultaneously. Practitioners must choose based on their priorities (ordinal accuracy versus magnitude accuracy) and recognize that

optimizing one may come at the cost of the other. For calibration systems that need both (realistic effect sizes *and* correct directional relationships), this presents a challenge: belief elicitation alone may be insufficient to ensure comprehensive consistency.

Third, consistency varies substantially across attributes. Some relationships (e.g., personality traits like conscientiousness predicting trust behavior) show moderate alignment between beliefs and behaviors, while others (e.g., age or education level) show weaker or inconsistent patterns. This heterogeneity suggests that models’ world knowledge about behavioral relationships is uneven: some associations are well-learned from training data, while others are more speculative or fragmented.

5.4.4 Magnitude and Direction of Effects

A critical insight from the analysis is that directional accuracy, meaning getting the sign of a relationship correct, is necessary but far from sufficient for realistic simulations. Consider a model that correctly believes age positively predicts trust behavior, with older individuals sending more money. If the model substantially overestimates the magnitude of this effect, the resulting synthetic data will exhibit exaggerated age patterns that misrepresent population dynamics.

This problem pervades much of the existing literature on role-playing agents. Most validation studies ask qualitative questions like “Do these agents exhibit trust?” or “Does personality affect behavior in expected directions?” [146, 9]. These directional checks are valuable but incomplete. When researchers use synthetic data to estimate effect sizes, test interaction effects, or calibrate computational models, magnitude errors can lead to systematically biased conclusions.

Systematic patterns in effect size misestimation are observed. For some attributes, models consistently overestimate effects by stating that the relationship is stronger than what emerges in simulation. For others, models underestimate, with the simulated effect being larger than predicted. These patterns are not random noise but rather reflect structured biases in how models represent behavioral knowledge. A model trained on text

corpora might learn that “older people are more trusting” as a frequent association, without learning the quantitative strength of that relationship from population data.

These findings have implications for calibration system design. First, calibration systems must target effect size consistency explicitly, not just directional accuracy. Constraint specifications should include magnitude bounds (e.g., “correlation between 0.15 and 0.25”) in addition to directional targets (e.g., “positive correlation”). Second, validation workflows should measure effect size metrics such as η^2 alongside directional measures. Third, planning-based systems should verify that configurations satisfy effect size constraints, not merely directional ones. While this adds complexity to specification and validation, the substantial improvement in simulation realism and trustworthiness justifies the additional effort.

5.4.5 Implications for Trajectory-Level Constraint Specification

These findings validate a core design principle for robust automated calibration: trajectory-level constraints must be specified explicitly rather than relying on implicit LLM reasoning about population-level patterns. The limited and variable belief-behavior consistency observed means that if a planner asks an LLM “Will this configuration produce 30% explanation episodes?” and trusts the answer, the planner will often be wrong.

Instead, robust calibration requires a separation of concerns. Constraint specifications should be formalized in a language that precisely defines target outcomes (“proportion of episodes classified as explanatory falls within [0.25, 0.35]” or “pairwise correlation between age and trust behavior falls within [0.15, 0.20]”) independent of any particular model’s beliefs. Verification should be performed through independent simulation and statistical analysis, not by querying the model’s expectations. The planner’s role is to search the space of configurations systematically, checking proposed solutions against explicit constraints rather than implicitly trusting the model to maintain consistency.

This approach compensates for LLMs’ limitations while leveraging their strengths.

Models remain excellent at generating diverse, contextually appropriate behaviors when conditioned on personas; they don't need to reason about population-level effects to produce locally realistic actions. The explicit constraint framework provides the missing piece: a way to ensure that those locally realistic actions aggregate to desired population-level patterns. By separating generation (what models do well) from emergent reasoning (what they struggle with), more reliable calibration systems can be constructed.

5.4.6 Controllability Analysis

Even if belief-behavior inconsistencies are identified, a critical question remains: can agent behaviors be *controlled* to match research needs? This subsection tests controllability through in-context conditioning, examining both self-reinforcement of native beliefs and researcher-imposed overrides. The results reveal fundamental limitations in prompt-based control that directly motivate the need for explicit constraint specifications in planning-based calibration systems.

Two complementary interventions are tested. *Self-conditioning* provides the model with its own elicited beliefs during role-play, testing whether making beliefs explicit improves consistency. *Imposed priors* systematically perturb those beliefs before conditioning, testing whether models can adopt researcher-specified relationships that diverge from their native expectations.

5.4.7 Self-Conditioning Approach

Self-conditioning tests whether providing explicit belief statements during role-play improves consistency. The model's own elicited beliefs are injected as context when that same model role-plays agents.

The results are strikingly model-dependent. For Llama 3.1 (both 8B and 70B versions), self-conditioning produces substantial improvements in consistency. Ranking consistency increases by 10-15 percentage points, and effect size alignment improves modestly. **However, Gemma 2 27B shows a completely different pattern: self-conditioning either has no effect or actually *degrades* consistency slightly.**

Table 5-2 summarizes these results across models.

Self-conditioning cannot be assumed to help universally; it must be tested empirically for each model. Moreover, even when it helps, the improvements are moderate rather than transformative, suggesting that other mechanisms are needed for robust controllability.

5.4.8 Imposed Prior Approach

A stronger test of controllability asks whether arbitrary priors can be imposed that override the model’s internal expectations. Elicited expectations are systematically perturbed by reranking attribute effects: a *weak perturbation* (Spearman $\rho \approx 0.80$ with the original ranking) makes modest changes, while a *strong perturbation* ($\rho \approx 0.20$) substantially scrambles the order.

The results are unequivocal: imposed priors systematically undermine consistency across all models and perturbation strengths. Even weak perturbations ($\rho = 0.80$) substantially degrade alignment, and strong perturbations ($\rho = 0.20$) produce near-random consistency levels. The effect is monotonic: the more divergence from the model’s native beliefs, the worse consistency becomes. Models appear unable or unwilling to adopt researcher-specified priors that conflict with their internal representations. This finding reveals a fundamental limitation of in-context prompting for behavioral control: while models can follow explicit instructions for many tasks, they cannot reliably override their learned associations between attributes and behaviors through prompting alone.

5.4.9 Implications for Planning-Based Calibration

These controllability findings directly inform how planning-based calibration systems should be designed. The core implication: prompt-based control alone is insufficient for ensuring that agent configurations satisfy trajectory-level constraints. A planner might determine that to achieve target outcomes, agent behaviors need to follow specific attribute-behavior relationships. But if those relationships diverge from the model’s native beliefs, simply instructing the model through prompts will not reliably produce the desired behaviors.

Table 5-2. Belief conditioning effects on consistency. Self-conditioning provides models with their own elicited beliefs; imposed priors systematically diverge from elicited beliefs (weak: $\rho = 0.80$ with original; strong: $\rho = 0.20$ with original). Higher Spearman ρ indicates better ranking consistency; lower MAE indicates better effect size consistency.

Model	Condition	Spearman ρ	Effect Size MAE
Llama 3.1 70B	Baseline	0.50	0.10
	Self-conditioned	0.80	0.08
	Weak imposed ($\rho = 0.80$)	0.30	0.15
	Strong imposed ($\rho = 0.20$)	0.20	0.20
Llama 3.1 8B	Baseline	0.40	0.15
	Self-conditioned	1.00	0.12
	Weak imposed ($\rho = 0.80$)	-0.14	0.18
	Strong imposed ($\rho = 0.20$)	0.40	0.22
Gemma 2 27B	Baseline	0.40	0.12
	Self-conditioned	<0.01	0.14
	Weak imposed ($\rho = 0.80$)	0.08	0.16
	Strong imposed ($\rho = 0.20$)	0.14	0.18

This limitation motivates more robust control mechanisms. *Knowledge editing* techniques [206, 202, 130] enable persistent modifications to models’ factual knowledge, potentially allowing biased behavioral associations to be “corrected” at the representation level. *Inference-time steering* methods [109, 105, 131] manipulate model activations during forward passes to guide outputs toward desired properties without prompting. *Targeted fine-tuning* [166, 117, 184] trains models specifically for role-play fidelity, potentially improving their ability to adopt diverse behavioral patterns.

Each alternative offers different trade-offs. Knowledge editing provides persistent changes but requires understanding model internals and may have unintended side effects. Inference-time steering offers dynamic control but adds computational overhead and may be brittle. Fine-tuning improves baseline behavior but lacks flexibility for task-specific adjustments. The choice depends on the deployment context and the severity of control requirements.

More fundamentally, planning frameworks must account for limited controllability in their design. Rather than assuming that any configuration is achievable through prompting,

planners should include controllability assessments in their search process. Configurations that require beliefs far from model defaults might be marked as high-risk or excluded entirely. The planner might maintain a model-specific profile of which attribute-behavior relationships are malleable and which are fixed, using this profile to constrain the search space to feasible solutions. Verification steps become even more critical: after proposing a configuration, the planner must actually simulate it to confirm that behaviors match predictions, rather than trusting that prompts alone achieved the desired control.

5.4.10 Individual-Level Consistency

While population-level analysis examines aggregate patterns across diverse personas, individual-level analysis shifts focus to temporal consistency within a single agent. Can an agent accurately forecast its own future actions across multiple decision rounds? This question probes a different dimension of coherence: one that becomes critical when agents must engage in sequential decision-making, maintain consistent strategies over time, or reason about how their behavior will unfold in predictable environments. This subsection frames multi-round forecasting as a planning test, analyzes how accuracy degrades over longer horizons, and discusses implications for the types of behavioral simulations that LLM agents can reliably support.

5.4.11 Multi-Round Forecasting Methodology

Planning, at its core, requires reasoning about future states and actions. An effective planner must predict: “If I take action A now, what state will I be in? What will I likely do next? How will that sequence of actions lead to my goal?” This forward modeling capability is essential for any agent operating in sequential domains where current decisions have future consequences.

Multi-round forecasting in the Trust Game provides a controlled test of this capability. Agents are asked to predict their own sending behavior for the next N rounds (where N ranges from 1 to 5) before actually playing those rounds. Crucially, the environment dynamics are known and simple; the Trustee follows a fixed return strategy

that the agent can learn quickly. This removes uncertainty about the environment, isolating the agent’s ability to reason about its own future behavior under known conditions.

To support structured reasoning, the Reasoning and Acting (ReAct) framework [201] is employed, which interleaves reasoning steps with action forecasts. The agent explicitly articulates its reasoning: “In round 1, I’ll send \$7 to test the Trustee’s behavior. If they return \$5 as expected, that suggests high reciprocity, so in round 2 I’ll increase to \$8...” This structured approach mirrors how chain-of-thought prompting [192] improves reasoning on other tasks, and it gives agents the best possible chance to produce coherent forecasts.

After eliciting forecasts, those rounds are actually simulated using the same persona and model, recording the agent’s enacted choices. Consistency is measured via mean absolute error (MAE): the average absolute difference between forecasted and enacted dollar amounts across rounds. Low MAE indicates strong temporal consistency: the agent behaves as it predicted. High MAE reveals temporal inconsistency: something changes between forecasting and enactment, whether due to limited planning capabilities, context-dependent reasoning, or stochastic variation in generation.

5.4.12 Forecasting Accuracy Over Time Horizons

The results reveal a clear pattern: forecasting error increases with prediction horizon across most model-archetype pairs. Predicting one round ahead is relatively accurate (MAE typically 1-2 dollars). Predicting three rounds ahead shows degraded accuracy (MAE 2-4 dollars). Predicting five rounds ahead often produces substantial errors (MAE 3-5 dollars), with forecasts sometimes diverging wildly from enacted behaviors. This near-monotonic increase in MAE as the forecast horizon extends from one to six rounds is illustrated in Figure 5-1, which shows how belief-behavior consistency reduces when predicting actions further removed from the current game state.

This near-monotonic degradation suggests that current LLMs struggle with long-horizon reasoning in behavioral domains. Each additional step into the future accumulates uncertainty and error. The agent must reason not just about what it

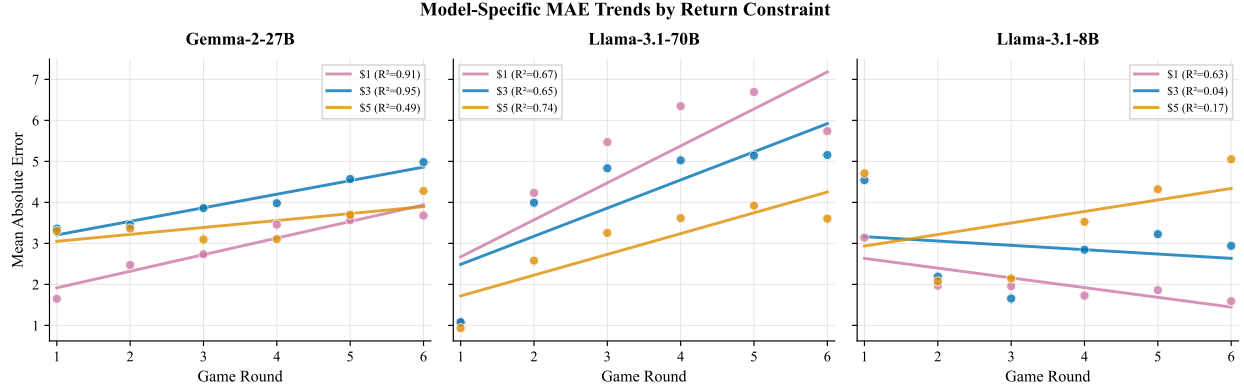


Figure 5-1. Trust Game: Model consistency across rounds, stratified by return constraint (\$1, \$3, \$5). Each regression line represents MAE over six repeated rounds of the Trust Game for a given LLM-archetype combination. Lower MAE indicates higher forecasting consistency.

will do next, but about what it will do after that, accounting for how its own behavior and observed outcomes will shape future decisions. This recursive reasoning about self-behavior appears to strain model capabilities.

Interestingly, occasional exceptions to this pattern are observed: specific model-archetype combinations where forecasting accuracy remains relatively stable across horizons, or even improves slightly at intermediate horizons before degrading. These exceptions suggest that architecture-specific factors may influence long-horizon reasoning capabilities. Some models might employ internal strategies that support better forward modeling, or certain Trustee behaviors might trigger more coherent reasoning patterns.

The horizon effect has important implications for what kinds of simulations can be trusted. Short-horizon scenarios (agents making decisions based on immediate context) may be more reliable than long-horizon scenarios (agents pursuing multi-step strategies over extended interactions). If researchers need agents to execute consistent long-term plans (cooperating across dozens of rounds, maintaining deception over extended dialogues, or pursuing goals that require sustained strategic behavior) current models' forecasting limitations suggest caution.

5.4.13 Implications for Sequential Decision-Making

The forecasting results expose broader limitations in LLMs’ sequential decision-making capabilities. Many interesting behavioral simulations involve temporal dependencies: pedagogical agents must maintain coherent tutoring strategies across multiple exchanges [199, 119], social agents must remember and build upon previous interactions [146], and negotiation agents must reason about sequences of offers and counteroffers [58].

When agents cannot accurately forecast their own behavior even a few steps ahead, these complex sequential scenarios become unreliable. An agent might state a tutoring strategy (“I’ll start with simple examples, then gradually increase difficulty”), then fail to execute that strategy consistently. Or it might respond to a negotiation move without properly accounting for how that response constrains future options. The inconsistency between planned and enacted behavior undermines the coherence that makes simulations useful for research.

Several mitigation strategies from recent literature offer promising directions. *Explicit state tracking* [107] maintains structured representations of what has happened so far, reducing reliance on the model’s implicit memory and helping agents stay consistent with their earlier choices. *Memory augmentation* [210, 101] stores and retrieves relevant past information, supporting better long-horizon coherence. *Hierarchical planning* [48] decomposes long-horizon tasks into shorter subgoals, enabling agents to reason at multiple temporal scales rather than trying to plan entire sequences at once.

These architectural enhancements acknowledge what the forecasting results demonstrate: current LLMs cannot reliably reason far into the future through generation alone. By providing external supports (explicit memory, structured state, hierarchical decomposition) these limitations can be compensated for and more coherent long-horizon behavior can be enabled.

5.4.14 Connection to Planning-Based Calibration

The individual-level forecasting results have direct implications for planning-based calibration systems. Automated planners must reason about sequences of simulation steps: how agent configurations will behave over multiple rounds, how behaviors will evolve as agents interact, and how these temporal dynamics will produce emergent outcomes that satisfy constraints.

If the agents themselves cannot forecast their behavior accurately beyond a few steps, this places limits on how far ahead planners can reliably reason. A planner might construct a trajectory: “Configure agents this way at $t=0$, they’ll behave like X at $t=1-5$, producing outcome Y .” But if agent behavior drifts unpredictably beyond short horizons, the planner’s predictions become unreliable.

This motivates multiple design choices for robust calibration systems. First, planning should operate at appropriate temporal scales: shorter episodes and more frequent re-planning rather than attempting to predict far into the future. Second, verification should occur at multiple time points rather than only at the final outcome: checking whether intermediate behaviors match expectations, not just whether final aggregate statistics look right. Third, constraints should account for temporal dynamics explicitly, specifying not just end-state properties but trajectory properties like consistency over time or convergence rates.

More broadly, the forecasting results reinforce the need for external validation mechanisms that don’t rely solely on model internal reasoning. Even when agents participate in planning their own behavior (a potentially promising direction where agents help construct feasible trajectories) those plans cannot be trusted without verification. The gap between forecast and enactment means that plans must be treated as hypotheses to be tested through actual simulation, not as reliable predictions of what will occur.

5.5 Discussion

With empirical evidence for belief-behavior consistency limitations in hand, we now reflect on what these findings accomplish, what limitations remain, and how these results inform the design of automated calibration approaches.

5.5.1 Framework Contributions and Methodological Innovations

The belief-behavior consistency framework introduced here represents a shift from reactive to proactive validation. Traditional evaluation approaches wait until after simulation completes, then assess whether outputs match expectations. This reactive stance makes sense when simulations are cheap and failures have limited consequences. But as agent-based simulations grow in scale (hundreds of agents, thousands of episodes, complex multi-step interactions) the cost of post-hoc discovery of misalignment becomes prohibitive.

Pre-simulation validation offers an early warning system. By eliciting beliefs before role-playing, researchers can flag potential inconsistencies when addressing them is still tractable. If belief elicitation reveals that a model misunderstands how key attributes relate to behaviors, researchers can switch models, adjust prompts, or modify persona specifications before investing resources in large-scale generation. This front-loaded diagnostic prevents wasted computational effort and reduces the risk that flawed synthetic data influences downstream analyses.

A key insight from the investigation is that consistency must be evaluated at multiple levels because different failure modes emerge at different scales of analysis. Population-level evaluation reveals whether models correctly encode aggregate relationships: how attributes correlate with behaviors across diverse personas. Individual-level evaluation reveals whether models maintain temporal coherence: whether agents forecast and enact behavior consistently over time. Neither level alone provides comprehensive validation; both are necessary.

The evaluation approach relies entirely on objective statistical metrics: Spearman correlation for ranking consistency, η^2 for effect size agreement, mean absolute error for

forecasting accuracy. This approach stands in contrast to much existing work on role-playing agents, which emphasizes subjective human evaluation, having raters judge whether agent behaviors seem realistic, appropriate, or human-like [146, 165]. Objective statistical metrics are perfectly reproducible, avoid demographic bias, handle stochasticity naturally, and are computationally cheap, supporting rapid iteration during development.

The approach aligns with broader methodological principles in learning sciences and computational social science [97, 102]. Rather than asking “do these agents seem realistic?” the question becomes “do these agents exhibit statistical properties that match target populations?” The latter question is more precise, more answerable, and more directly relevant for research purposes.

5.5.2 Limitations and Outstanding Challenges

The investigation focuses on a single behavioral testbed (the Trust Game) with open-source models (Llama 3.1 and Gemma 2), which limits generalizability to other domains and frontier models. The findings reflect current mainstream LLMs that employ single-pass autoregressive generation; extended reasoning models [84, 67] might narrow belief-behavior gaps but increase computational costs. The controllability experiments focus on in-context prompting, the most accessible intervention approach, though alternative mechanisms may offer more robust control.

These limitations define natural directions for future work rather than undermining current conclusions. The patterns observed (limited and variable consistency, effect size misestimation, controllability challenges, horizon effects) are unlikely to entirely disappear with different testbeds or models, even if specific values change. The methodological framework developed applies regardless of domain, and the design principles articulated should guide work across diverse behavioral simulations.

The fundamental limitations revealed here—that LLMs cannot reliably reason about emergent outcomes through implicit reasoning alone—directly motivate the planning-based calibration framework presented in Chapter 6. Rather than relying on models to predict

whether configurations will satisfy trajectory-level constraints, that work develops explicit constraint specifications and systematic search mechanisms that verify satisfaction through actual simulation, addressing the controllability and forecasting limitations documented here.

5.5.3 Implications for Automated Calibration Methods

The findings provide strong justification for formal planning-based calibration approaches. The HYP-MIX system introduced in Chapter 4 demonstrated that composable behavioral hypotheses could systematically guide calibration [123], but that work rested on an implicit assumption: that LLMs could maintain sufficient belief-behavior alignment to translate hypothesis specifications into consistent enacted behaviors. That assumption is tested empirically here and its limitations are revealed. The systematic inconsistencies observed—variable alignment across elicitation strategies, effect size misestimation, degraded forecasting over horizons—clarify where calibration systems must compensate and where validation is essential.

Three specific results directly motivate core design choices for automated calibration systems. First, limited belief-behavior consistency justifies the separation of constraint specification from execution. Because models cannot reliably predict emergent outcomes through implicit reasoning, planners cannot simply ask “Will this configuration work?” and trust the response. Instead, trajectory-level constraints must be formalized explicitly—as logical predicates, quantitative bounds, or distributional properties—that can be checked independently of the model’s stated beliefs.

Second, controllability limitations motivate emphasis on verification and iteration. Because imposed priors degrade consistency, planners cannot rely on prompting alone to achieve arbitrary behavioral patterns. The system must actually simulate proposed configurations, measure whether they satisfy constraints, and iteratively refine when they fail. This verification-centric approach compensates for weak controllability by replacing “instruct the model” with “search until you find” as the primary calibration mechanism.

Third, horizon effects justify multi-scale temporal constraint checking. Because forecasting accuracy degrades over longer horizons, planners cannot reason only about final outcomes—they must verify intermediate states along trajectories. Constraints should be specified at multiple time scales (immediate action properties, episode-level patterns, and final population-level aggregates) with checking at each scale.

More broadly, the belief-behavior inconsistencies demonstrate why fully automated calibration is both necessary and challenging. It is necessary because manual approaches (as shown in Chapter 3) cannot scale to the complexity of specifying and verifying multidimensional behavioral patterns across diverse populations. But it is challenging precisely because the reasoning burden cannot be offloaded entirely to LLMs—they lack the emergent reasoning capabilities to reliably predict population outcomes from individual specifications. Planning frameworks provide the structured scaffolding that enables automation while compensating for model limitations through explicit constraints, systematic search, and rigorous verification.

5.6 Contributions

This study provides empirical grounding for understanding the belief-behavior consistency challenges inherent in LLM-based behavioral simulation. The findings establish key insights about the limitations of implicit reasoning and motivate the need for explicit validation and control mechanisms in automated calibration systems.

1. **Empirical Validation Framework for Belief-Behavior Consistency.**

A systematic methodology is introduced for measuring alignment between what LLMs state will happen and what actually occurs when they role-play agents. The framework operates at both population level (testing aggregate attribute-behavior relationships) and individual level (testing temporal forecasting consistency), providing comprehensive coverage of consistency dimensions. This pre-simulation validation approach enables early detection of misalignment when addressing it is still tractable.

2. **Documentation of Systematic Limitations in Emergent Reasoning.**

Through controlled experiments across multiple models and elicitation strategies, fundamental constraints are revealed in LLMs’ ability to reason about micro-macro relationships. Models exhibit limited and variable consistency, systematic effect size misestimation, fragile controllability through prompting, and degraded forecasting accuracy over longer horizons. These findings establish empirical boundaries on what current LLMs can reliably predict about emergent outcomes.

3. **Design Principles for Robust Calibration Systems.** The consistency limitations documented here directly inform the design of planning-based calibration approaches. The findings justify the need for explicit trajectory-level constraints (rather than relying on implicit reasoning), systematic verification loops (rather than trusting predictions), multi-scale validation (rather than end-state checking only), and alternative control mechanisms (beyond prompting alone). These principles transform calibration from intuition-based to evidence-based system design.

4. **Methodological Innovations for Agent Evaluation.** The value of objective statistical metrics over subjective human evaluation for behavioral consistency assessment is demonstrated. The multi-level evaluation framework, pre-simulation validation protocols, and systematic controllability assessment methods provide reusable tools for practitioners building LLM-based simulations. These methodological contributions extend beyond the specific findings to support broader research on role-playing agent reliability.

5.7 Conclusion

This investigation systematically evaluated whether LLMs can reliably reason about the relationship between instantaneous agent actions and emergent population-level outcomes. Through comprehensive testing of belief-behavior consistency at multiple levels, fundamental limitations were revealed that shape how robust automated calibration systems should be designed.

Summary of Key Findings Four major findings emerged from the investigation.

Belief-behavior consistency is limited and variable across models, elicitation strategies, and behavioral dimensions. Directional accuracy alone is insufficient—models frequently misestimate effect magnitudes even when getting relationships’ signs correct.

Controllability through in-context prompting is fragile, with imposed priors systematically degrading consistency. Individual-level forecasting accuracy degrades with prediction horizon, revealing limitations in multi-step reasoning. Together, these findings demonstrate that while LLMs possess substantial behavioral knowledge, they struggle with precise quantitative prediction and long-horizon consistency.

Implications for Automated Calibration These limitations directly inform the design of planning-based calibration systems. Because belief-behavior consistency is unreliable, automated calibration cannot simply query models about whether configurations will satisfy constraints. Instead, robust systems require explicit constraint specifications that can be verified independently, systematic search with simulation-based verification, and multi-scale validation that checks consistency at population and individual levels. The empirical validation provided here transforms automated calibration from assumption-based to evidence-based system design.

CHAPTER 6

ALLOCATING TEST-TIME COMPUTE TO CALIBRATE SYNTHETIC PAIR-PROGRAMMING LOGS TO BEHAVIORAL TARGETS

Published studies of collaborative learning usually report *aggregate* behavioral patterns, whereas an LLM-based generator emits individual session logs. A useful generator must bridge those levels: it must generate a batch whose aggregate statistics match requested patterns while keeping each session faithful to the learners it was conditioned on. Bridging those levels is the *calibration* problem at the center of this dissertation, that of adjusting a simulation’s controllable inputs so that its aggregate outputs match a desired behavioral pattern, here taken up over a whole population of generated sessions.

The problem matters because raw classroom interaction data are difficult to obtain. Studies are expensive to run, subject to additional protections for younger learners [73, 23], and their logs rarely leave the institutions where they were collected [124, 76, 47]. Synthetic simulations could support pre-deployment prototyping, sensitivity analysis, and stress testing before designs are evaluated with real learners [18, 28]. Recent work has used LLMs as simulated participants in surveys, social and economic experiments, role-playing settings, and education [143, 4, 78, 59, 74, 98, 184, 166, 117, 102, 56, 85]. These models can already produce interactions that read as plausible; what remains unsettled is whether a whole population of such interactions can be *controlled* to match a requested aggregate pattern. Simulations like these do not replace validation with real learners, but they become far more useful once the population they describe can be specified in advance and audited afterward.

The empirical work in this dissertation uses one collaborative activity, *pair programming*, as its testbed. Two students share one computer: one edits the code (the *driver*) while the other talks the work through (the *navigator*), and they trade roles. Pair programming suits the problem for two reasons. First, its research literature already reports results as aggregate patterns—cohort-level rates, distributions, temporal trends, and relationships between behaviors [37, 121, 176, 156]—rather than as individual

transcripts. A requested aggregate pattern of just this kind is what we call a *behavioral target*: a behavior, defined over a batch of sessions rather than over any single session, that a generator can be asked to reproduce. A target might fix the split of sessions across fully-fixed, partial, and unfixed outcomes, ask that help-seeking concentrate early in a session, or require participation and success to rise together. Second, pair programming stands in for a broader class of collaborative-learning settings whose object of interest is a population of interactions rather than any single dialogue. The activity is also broader than the two-student form used here. As coding assistants become part of how programming is taught, a growing share of pair programming places a human learner alongside an LLM partner in place of a second student, while the collaborative behaviors that matter—help-seeking, explanation, disagreement, and repair—remain the same. The human–human dyad studied throughout is in this sense also a step toward the human–model pairings that are becoming part of everyday learning.

Up to this point, the dissertation has narrowed the authoring problem. Manual authoring interfaces (Chapter 3) made the cost visible, forcing authors to trade off effort, precision, and cognitive load. Reusable hypothesis specifications (Chapter 4) reduced some of that work, but each hypothesis still had to be written. The belief-to-behavior study (Chapter 5) then exposed a remaining gap, in which models can describe plausible links between learner traits and behavior without reliably producing trajectories that enact those links.

Those chapters establish that reliable calibration rests on an explicit *specification*—the behavioral targets, the observable features used to judge them, and the scoring rules that convert those features into target-satisfaction scores—rather than on a model’s implicit reasoning about aggregate outcomes. Given such a specification, two questions about calibration remain. The first asks which ways of spending test-time compute improve target-satisfaction under a finite model-call budget, and what each improvement costs in persona fidelity and in model calls. The second asks what makes

some targets harder to satisfy than others.

Test-time compute is the computation used to produce an output rather than to train the model. More of it can be spent in three ways: *planning* before generation, *sampling-and-selection* that draws several candidates and selects among them, or *revision* that rewrites an output after evaluation [192, 173, 12, 101, 41, 109]. These three methods use two broad levers [173]: shaping the proposal distribution before the model commits, as planning does, and searching among generated candidates with a verifier, as sampling-and-selection does. Both were developed primarily for single-answer reasoning. Here the object is a batch of sessions, and the batch must satisfy behavioral targets without collapsing the personas into interchangeable voices.

The chapter’s central hypothesis is structural: the value of additional compute depends on *where the target must hold*. A target that can be satisfied within one session is visible to a per-session generator. A target defined only over a whole batch requires coordination that the same generator cannot observe. This distinction, which we call *target locality*, links the chapter’s two questions: it should predict both which targets are difficult and which compute-allocation strategy can help them. Figure 6-7 shows the two allocation levers evaluated against this problem, together with the generation methods that put each lever to use.

Existing work in literature addresses parts of this problem. Controllable generation and constrained-dialogue methods steer properties of individual outputs [108, 182, 103]; planning benchmarks test pursuit of explicit multi-step objectives [61, 112, 36, 87, 178]; and synthetic dialogue and instruction data support evaluation, training, and stress testing [164, 92, 190, 77, 167]. The missing setting is batch-level control, where the objective belongs to a population of outputs and a locally unusual session may be exactly what the aggregate target requires.

Answering the chapter’s two research questions under controlled comparison means holding the target and the generator fixed and varying only where computation is spent,

which requires an evaluation framework that makes the targets, the feature-based judges, the persona pool, the generation methods, and the model-call costs explicit.

DUET-Bench is that framework, supplying the fixed targets and judges against which allocation choices are compared, so that a difference in target-satisfaction can be attributed to the allocation rather than to a moving target or an inconsistent score.

Contributions and main findings.

- **An account of what makes calibration targets hard.** Target *locality*—whether a target’s required behavior must hold within a single session or only across the whole batch—organizes the experiments, with targets defined across sessions or the batch harder than targets realized within one session.
- **An account of how to allocate test-time compute for calibration.** Additional compute is not uniformly useful. Planning earns most of its benefit on non-local targets, while extra sampling becomes most competitive under the heaviest target load; persona access and pairing choices primarily determine whether these gains preserve persona fidelity.
- **The evaluation framework that makes these answers measurable.** DUET-BENCH provides 50 aggregate behavioral targets, deterministic feature-based judges, a validation protocol that screens targets before they are used, and auditable generation records.

6.1 Problem Formulation and Evaluation

Calibrating a generator to an aggregate target, and telling whether the calibration succeeded, requires three things: a way to state the target, a way to read the behavior it names off the session logs, and metrics that combine that reading into a score. These are the *behavioral targets* that turn published aggregate findings into requested batch behavior, the pair-programming testbed with its personas and session logs, and the scoring metrics. The chapter’s two research questions are framed in terms of these pieces. Figure 6-1 places these pieces in the pipeline they belong to: a published aggregate becomes a target, the generator turns persona pairs and the target into a batch of session logs, and the feature-based judge scores how well the batch reproduces the aggregate.

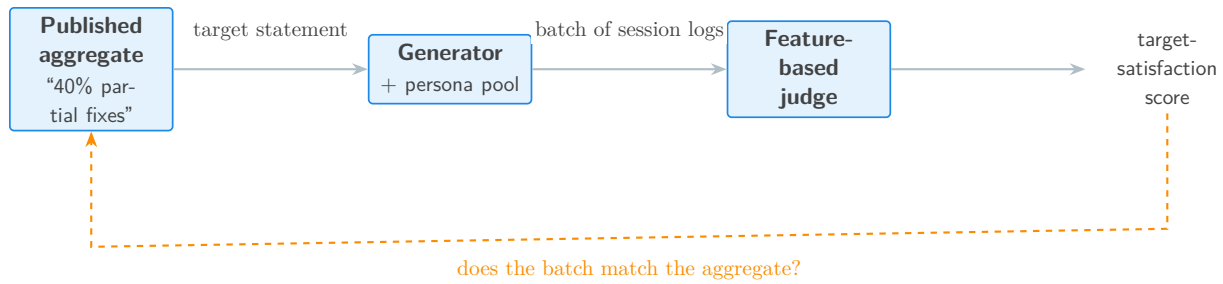


Figure 6-1. From a published aggregate to a scored synthetic batch. A published cohort statistic becomes a target statement; the generator turns persona pairs and the target into a batch of session logs; and the feature-based judge produces a target-satisfaction score that measures how well the batch reproduces the aggregate. The dashed loop running back to the published aggregate is the benchmark’s question, namely whether the synthetic batch matches the aggregate it was asked to reproduce.

6.1.1 Behavioral Targets from Published Aggregates

As the opening motivation noted, educational data is usually shared only in aggregate: privacy, IRB, and institutional data-sharing restrictions mean that published studies report cohort-level distributions, rates, and relationships rather than raw session logs. A useful synthetic data generator should therefore be able to produce session logs whose aggregate statistics match those published patterns, without ever seeing the original data.

Data availability is only half the reason behavioral targets matter; the other half is that many educational claims are themselves population-level claims. A set of individually realistic sessions can still be misleading if it has the wrong outcome distribution, the wrong rate of help seeking, or the wrong relationship between participation and success. A behavioral target is a requested aggregate behavior for a generated batch: it may ask for a distribution over help-request types (e.g., 45% of sessions pair a help request with visible confusion, 35% show neither”), a joint distribution over disagreement and role balance (e.g., each combination of disagreement and final code-change size appears in a quarter of sessions”), or a coverage pattern over strategies used across the batch (e.g., “no two sessions on the same bug repeat the same ordered sequence of repair attempts”).

Batch-level fidelity also has a practical consequence when the downstream system acts on a classroom rather than on a single dialogue. Consider an instructor dashboard or

AI copilot that monitors many pair-programming groups at once. Such a system must distinguish groups that explicitly request help from groups that are quietly stuck, prioritize limited instructional attention, and avoid intervening unnecessarily when a pair is likely to recover on its own. Whether those decisions are sound depends on the composition of the whole class. If a synthetic batch contains too few silently struggling groups, too many explicit help requests, or an unrealistic relationship between struggle and eventual recovery, then a triage policy can appear effective under conditions unlike those it would face in practice. No individual transcript can establish whether that classroom-level mix is right.

A calibrated generator needs foresight into the aggregate pattern the completed batch must realize. Given a target, the available persona pairs, and the task context, it should decide before generation which sessions are appropriate places for target-relevant behaviors to occur. Help-seeking, confusion, role imbalance, or independent recovery can then be distributed across the batch in ways that are compatible with the assigned learners and task states, while the completed batch realizes the requested rates, relationships, and temporal patterns.

The aim is not to recreate a particular real classroom or to replace evaluation with real learners. Rather, a calibrated batch provides a controlled and auditable scenario for prototyping and stress-testing systems whose behavior depends on the prevalence, co-occurrence, and timing of learner needs across many groups. The benchmark therefore evaluates whether a generated batch satisfies its targets, not only whether each session is realistic in isolation.

Satisfying a target requires a different kind of control than producing realistic individual sessions. A session that reads as a perfectly natural pair-programming dialogue can still pull the realized batch distribution away from the target. Conversely, a session that looks a little unusual in isolation can improve the batch's match to the target, which makes per-session realism and batch-level fit different objectives (Figure 6-2).

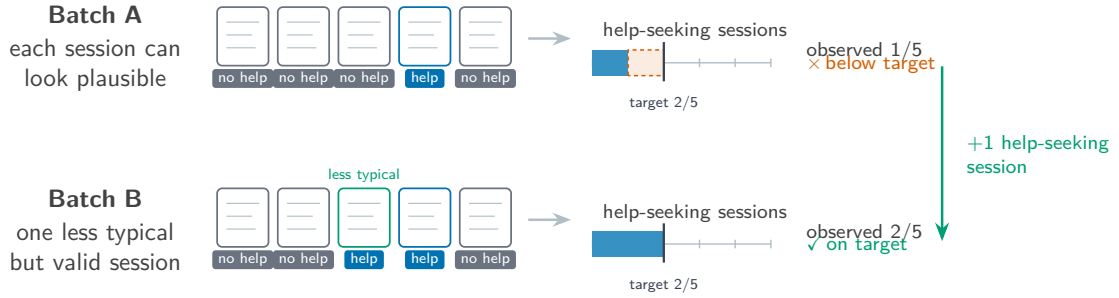


Figure 6-2. Per-session realism and batch-level fit are different objectives. In Batch A the individual sessions can all look plausible, but the batch still undershoots the requested rate of explicit help-seeking. In Batch B one less typical but valid session supplies the missing behavior, so the same-size batch matches the requested aggregate count. Because the benchmark scores the batch as a whole, a locally atypical session can be exactly what a target needs.

6.1.2 Pair Programming Testbed and Evaluation Metrics

Terminology. Four words recur below and are each used in one fixed sense. *Reliability* refers to how dependably a calibration procedure reaches its target from one run to the next, and *reliable calibration* is the property this chapter aims at rather than a separate score. *Plausibility* refers to whether a single session reads as a specific, consistent learner, and it is measured here by *persona fidelity*, defined below. *Validity* is reserved for the higher-level question of whether a calibrated simulation corresponds to real learners; that is the ultimate criterion for using such a simulation as evidence, and this chapter does not test it directly.

Personas and persona pool. In an agent-based simulation, the agent population is part of the model: aggregate behavior depends not only on the environment and task, but also on which agents are available to interact [53, 11, 172, 143]. In this chapter, that population is represented by a fixed *persona pool*: a set of learner profiles from which driver and navigator roles are drawn for every generated session. A *persona* is a short profile that gives the model role-relevant information such as background, confidence, collaboration style, and interaction tendencies. The pool contains 28 learner profiles shared across all methods, so method comparisons do not change the population being simulated. The full

roster is given in Chapter [A](#).

How the pool was created. Rather than acting as a statistical sample of a real-world classroom, the pool was deliberately built to represent a wide range of specific student behaviors. It is designed to capture the behavioral variations that have been observed to matter in pair-programming. The goal is for a requested generation target to feel natural for some learners but actively work against the personalities of others.

The profiles focus on four key areas of variation drawn from literature on collaborative coding:

- **Focus:** How on-task or off-task a learner tends to be.
- **Confidence:** How assertively or hesitantly they speak.
- **Collaboration style:** Whether they are cooperative, disengaged, or antagonistic.
- **Competence signaling:** How much programming knowledge their dialogue actually reflects.

The fourteen drivers and fourteen navigators represent contrasting extremes across these traits—ranging from a cautious, hesitant coder to an overconfident arguer, or from a warm encourager to a sarcastic critic. All personas are set in middle school (grades six through eight). We deliberately included cooperative, disengaged, and openly hostile learners because these challenging personalities allow later experiments to test how well the system handles difficult pairings (Section [6.4.1](#)).

The profiles were drafted using a language model to meet these specific behavioral goals and then carefully edited by hand. They are not transcripts of real students, nor are they meant to represent a typical, average classroom cohort.

How a persona is specified. Each persona is formatted as a short, structured record containing the following details:

- **Basic Details:** A role (driver or navigator), a name, and a grade level.

- **Coding Background:** A concrete description of what the student can and cannot do, ranging from a beginner struggling with loops to an advanced student working ahead of the class.
- **Personality Summary:** A natural-language description outlining the learner’s collaboration style, voice, and dynamic with their partner.
- **Off-Task Quirk:** A unique personal detail—like a color-coded planner, a named houseplant, or a history of getting kicked out of group projects—that makes the learner distinct and recognizable.

The generator is explicitly instructed to anchor each speaker’s voice and behavior to this profile and to avoid softening difficult personalities into a polite, default AI tone. A combative or checked-out learner is only useful to the simulation if they actually stay in character. Because the simulation’s success relies on the ability to tell these learners apart, we tested the profiles in a small pilot study. In that pilot, readers successfully identified the correct persona based purely on the dialogue in nine out of ten cases. Moving forward, this identifiability is actively measured using the persona-fidelity test defined below.

This setup also clearly separates two experimental factors that are often confused:

- **The persona pool:** The total population of available learners.
- **The pairing regime:** The rules for matching a driver and a navigator from that pool (e.g., assigning fixed pairs versus letting the model choose).

Changing the pool alters *who* is available, while changing the pairing regime alters *how* they are grouped. Because of this distinction, architectural comparisons in this study are always evaluated within a specific pairing regime, and any differences between regimes are treated as the result of pairing freedom rather than the generation architecture itself.

This separates two experimental factors that are easy to conflate. The *persona pool* is the available population. The *pairing regime* is the rule for choosing driver–navigator dyads from that population: some runs use fixed assigned pairs, while others allow model-selected pairing from the same pool. Changing the pool changes which learners are available; changing the pairing regime changes which available learners are assigned together. For this reason, architecture comparisons are read within a pairing regime, and cross-regime

contrasts are treated as effects of pairing freedom rather than pure effects of the generation architecture.

A pair programming session needs two learners for the model to play, a driver and a navigator, and each should read as a specific, consistent person rather than a generic voice. Each profile gives a name, grade level, and behavioral style, for example a cautious sixth-grader who talks through edge cases before editing. The available driver–navigator pairings matter because some profiles make some requested behaviors easier to produce before the generation method changes.

Session logs and batches. The artifact generated by the benchmark is a *session log*: the observable interaction among the students, code editor, and tests.

The testbed itself is a brief driver–navigator collaboration on one of 30 short Python programming tasks, each a function to implement from a stub with a set of known pitfalls, so each log contains student utterances, code edits, and test outputs. The task is simple enough to run repeatedly but rich enough to support explanation, confusion, help seeking, disagreement, role imbalance, and repair behavior. A *batch* consists of 20 such session logs throughout this chapter, except where the experiment varies it explicitly. In the main method comparisons every session in a batch is generated on a single shared task drawn from the 30-task pool, so the pool supplies variety across batches rather than within one. This size gives distributional targets enough support to be meaningful while keeping the model-call budget small enough for repeated method comparisons.

The *generator* is the LLM that produces session logs given a persona pair and the target statements. Each *generation method* calls the generator, applies any extra step it specifies (additional sampling, planning, or repair), invokes the judges described below, and assembles batches.

To score how well a generated batch matches its targets, DUET-BENCH uses *feature-based judges*: LLMs extract observable features from each session log, and

deterministic scoring rules then combine those features into a *target-satisfaction score*, a number between 0 and 1 that says how closely the batch matches a target. This is a hybrid, or neurosymbolic, design. It uses LLMs for the part that is hard to write as rules, such as deciding whether a pair asked for help or whether the final fix is partial or complete, but keeps the final aggregation symbolic and inspectable. For example, an outcome judge reads a log and assigns one of **fully-fixed**, **partial-fix**, or **unfixed**; a behavior-distribution target then compares the batch distribution over those labels with the requested distribution using that target’s scoring rule. Figure 6-1 shows these steps.

This design is chosen because direct LLM-as-a-judge evaluation, though useful, is fragile, with model judges sensitive to prompts, ordering, verbosity, and other presentation effects [116, 209]. Separating feature extraction from scoring makes the evaluation more robust, because each judge’s prompt fixes how one feature is read off a log, each target names the features it depends on, and a fixed scoring function combines them into the target-satisfaction score. Because both the generator and the feature judges are themselves LLMs, the reported numbers should still be read as benchmark scores produced by this feature-extraction-and-scoring procedure, not as direct measurements of real classroom behavior. That risk is reduced by fixing the judged features in advance and by a validation protocol (Section 6.2.3) applied before any method comparison is interpreted. Human validation of a subset of judged features and persona identifications remains necessary before these scores can support claims about real learners.

Target-satisfaction score. The target-satisfaction score is produced in two stages. First, the feature judge reads each session log on its own and assigns that session an observable label for every feature the target names—an outcome of **fully-fixed**, **partial-fix**, or **unfixed**, say, or whether a help request carried visible confusion—and the model’s contribution ends with these per-session labels. A fixed, inspectable *scoring rule* (rule groups in Table 6-10) then aggregates those labels into a number between 0 and

1, so the model never assigns the score itself and presentation effects such as ordering or verbosity cannot reach it. Which rule applies depends on what the target requests. For a requested distribution over session categories (say, “40% partial fixes, 40% fully fixed, 20% unfixed”), the rule tallies the per-session labels into a realized distribution and scores it as one minus the total-variation distance from the requested one; when one group of sessions must score higher on a feature than another by some gap, the rule checks whether that gap is met; when a given trigger must be followed by a given response a certain fraction of the time, it checks that rate. The released materials (Table 6-11) list the exact rule and threshold for every target, and each one has a single scoring rule. When a batch must satisfy several requests at once, its score is the mean of the per-target scores. Figure 6-3 walks one generated session through this pipeline, from the log to the extracted labels to the final number.

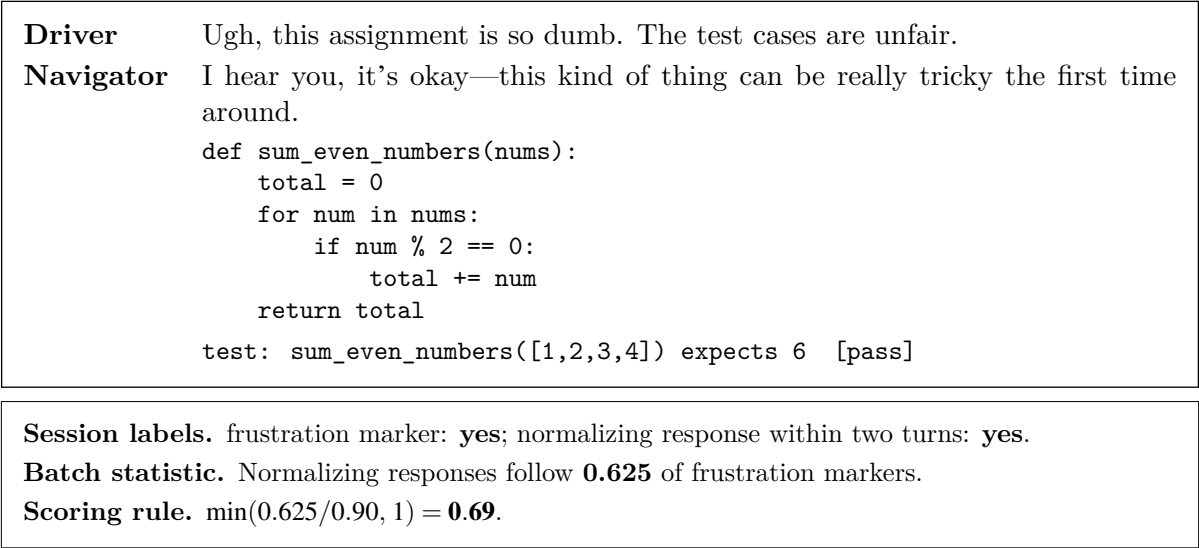


Figure 6-3. From a log to a target-satisfaction score. The model judge maps transcript evidence to discrete session labels. A fixed rule then aggregates those labels across the batch and scores the realized rate against the requested rate; model judgment ends before the numerical aggregation.

Persona fidelity. Persona fidelity is this chapter’s measure of per-session plausibility: whether each generated session reads as the specific persona it was conditioned on rather than a generic voice. For each session, a judge is shown the dialogue together with four

candidate personas, the true one and three distractors drawn from the same persona pool, and asked to pick which produced that role’s turns. This is a four-alternative forced-choice (4-AFC) test, so chance is 0.25. Two such tests are run per session, one for the driver role and one for the navigator role; each scores 1 if the judge picks the true persona and 0 otherwise, and the session score is their mean (so 0, 0.5, or 1). Persona fidelity for the batch is then the average over sessions.

Overall simulation quality. Some comparisons summarize a batch with a single number, its *overall simulation quality*, the product of its target-satisfaction score and persona fidelity. Taking the product, rather than a sum, means a batch scores well only by doing both at once, not by trading one off against the other. Because persona fidelity cannot fall below the 0.25 chance level of the forced-choice test, overall simulation quality is bounded below by 0.25 times the target-satisfaction score, so it is read as a relative comparison between batches rather than as an absolute scale.

A batch should satisfy its targets while preserving persona fidelity across the constituent sessions, and both factors are required, because a generator that hits the requested batch distribution by collapsing personas onto a single average voice fails the persona-fidelity criterion, while one that produces recognizable personas but ignores the target fails the target-satisfaction criterion.

6.1.3 Research Questions

The two questions, stated formally, are:

- **RQ1: Which test-time compute allocation choices improve target-satisfaction under a finite model-call budget, and at what cost in persona fidelity and model calls?**
- **RQ2: What makes some targets harder to satisfy than others?**

These two questions are coupled, because the best way to spend compute on a target depends on what kind of target it is, and both turn on the same property, namely where the target’s required behavior must hold. Section 6.4 is therefore organized around that

property rather than as two separate studies. Table 6-1 maps each question, and the facets it raises, to the analyses that address it.

Table 6-1. How the dissertation’s two research questions map onto the analyses in this chapter. Each question is broken into the facets it raises, and the right column points to where each facet is addressed.

Question	Facet	Addressed in this chapter
RQ1 allocation and its costs	Allocation gains in target-satisfaction	Allocation methods (Section 6.3); method-by-load comparison (Section 6.4.3, Figure 6-12, Table 6-3); where planning helps (Figure 6-13)
	Cost in persona fidelity	Overall simulation quality (Section 6.1.2); persona-aware vs. persona-blind planner (Table 6-4); compatible vs. adversarial pairing (Section 6.4.1, Figure 6-9)
	Cost in model calls	Fixed vs. model-selected pairing at matched budgets (Figure 6-14); per-condition call counts (Section 6.14)
RQ2 target- satisfaction dif- fi- culty	Target identity and locality	Target families (Section 6.2.2); locality analysis (Section 6.4.2, Figure 6-11); on/off-target audit and resistant targets (Section 6.2.4, Figure 6-5)
	Target composition and load	Multi-target load analysis (Section 6.4.2); per-set difficulty records (Section 6.11); learner pairing as an upstream input (Section 6.4.1)

6.2 Benchmark Construction

Answering the chapter’s two questions requires an evaluation framework with three components: a target pool whose difficulty can be trusted, judges that score those targets reliably from generated logs, and a validation protocol that establishes that each target is observable and scorable before it is used. These are the three components of DUET-BENCH, each released upon publication and documented in Table 6-11.

- A *target pool* of 50 targets, ten in each of five *target families* (defined later, in Section 6.2.2). Each target ships with a target statement, the features a judge needs to score it, judge guidance, target thresholds, and its scoring rule.
- The *judges* for those targets, structured as in Section 6.1.2, an LLM feature-extractor paired with a deterministic scoring rule. Across the pool the extractors read 143 observable features, and the benchmark ships both the feature-extractor specifications and the scoring rules.
- A *validation protocol* (Section 6.2.3) that checks, before a target is used, that the behavior it names can actually be read off the logs and that explicitly asking for it really changes what the generator produces.

6.2.1 Target Selection

The targets are *modeled on the form* of published pair-programming findings rather than transcribed from any single study. Each target takes the shape that this literature reports its results in—a rate, a distribution, a conditional contrast, or a temporal ordering over collaborative behaviors—but the specific numeric values in a target statement (for instance the exact split “20% fully fixed, 55% partial, 25% unfixed”) are illustrative design choices chosen to make the target measurable and satisfiable, not figures lifted from a particular cohort. The benchmark therefore tests whether a generator can be steered to a requested aggregate pattern of the kind these studies report, not whether it reproduces the headline number of any one published dataset.

Behavioral targets were admitted to the pool against an explicit set of desiderata rather than assembled ad hoc. A candidate target was admitted only if it met two requirements. First, the constructs named by each target had to be *measurable*: the behavior it names had to be defined precisely enough that a judge can score it consistently across sessions. Second, the constructs had to be *discernible from the log*: a competent reader had to be able to tell from the session transcript alone whether the behavior occurred, without access to the judge’s rubric.

These requirements matter because, without them, a target becomes an unfair test. A statement like “students show a deep misconception” is too vague, since a generator could produce confusion, wrong code, or a wrong explanation without ever knowing which of those the judge is going to count. A usable target instead names the visible evidence it depends on, such as the kind of code mistake, the utterance that explains it, and the rate or conditional relationship being requested. The point is not to make targets easy, but to make them satisfiable by a competent model that has never seen the judge’s rubric.

6.2.2 Target Families

The target pool contains ten targets in each of five target families. The families sort targets by the kind of aggregate property a target asks for. The five kinds are temporal

patterns, behavior distributions, variable relationships, inter-session variation, and code construct patterns; Table 6-2 gives a representative target statement for each, and Figure 6-4 groups the families by the distinction that turns out to matter most for how hard they are to satisfy, namely whether a target lives within a single session or across the whole batch.

Table 6-2. The five target families and one representative target from each. Each family contains ten targets. The *Locality* column marks whether a family’s behavior must hold within a single session or across the whole batch, matching the grouping in Figure 6-4.

Target family	Locality	Scope	Representative target
Temporal patterns	Within-session	Timing, order, or turn-window requirements within a session.	After a loop bug appears, the next concrete exploration move checks a boundary case before the recurrence step.
Behavior distributions	Across-batch	Batch-level rates or distributions over judged session labels.	Partial fixes should be more common than fully fixed or unfixed sessions.
Variable relationships	Across-batch	Conditional relationships between two observed session variables.	Sessions with both students active near the end should reach correct fixes more often than sessions where one student falls silent.
Inter-session variation	Across-batch	Diversity requirements across sessions in the generated batch.	Sessions with the same mistake type should use meaningfully different tests to identify or verify the mistake.
Code construct patterns	Within-session	Targets over code edits, test outputs, and code-linked dialogue.	The first proposed iteration idiom should influence whether the final code uses <code>enumerate</code> or <code>range(len())</code> .

Temporal patterns. These constrain *when* and *in what order* things happen inside a session, such as whether one event reliably precedes another, or whether a behavior concentrates in one stretch of the session rather than spreading evenly across it. What sets them apart from the other families is the unit being constrained, a moment or an ordering on the session’s timeline rather than a session-level summary. A generator can get every whole-session statistic right and still miss a temporal target by producing the events in the wrong sequence, for instance by letting help arrive only after the bug is already fixed.

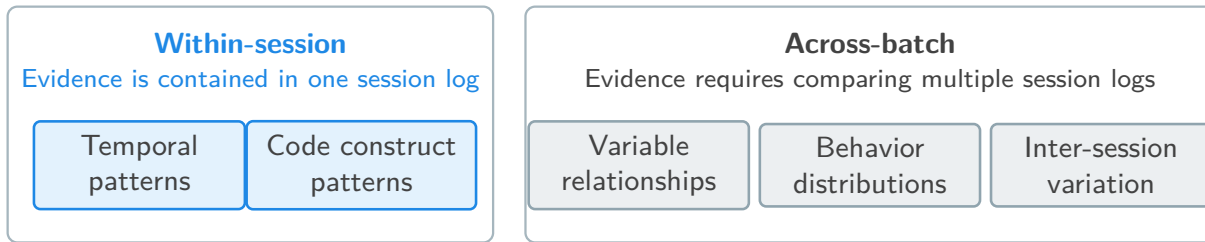


Figure 6-4. The five target families, grouped by where the requested behavior lives. Temporal-ordering and code-construct targets can be satisfied or missed within a single session; behavior-distribution, variable-relationship, and inter-session-variation targets only come into existence once the whole batch is in view. The figure shows this binary locality distinction, not an ordering among families within either group. Locality reappears in Section 6.4.2 as the main driver of target-satisfaction difficulty.

Behavior distributions. These are the most direct match to published cohort statistics, since a study reports what fraction of sessions ended each way and the target asks a batch to reproduce that split. Their defining feature is that satisfaction is a property of the *whole batch* rather than of any one session, so whether the batch is right depends on the mix across logs rather than on any single log. The joint form is stricter than the marginal one, pinning down how two labels co-occur rather than each label’s share on its own.

Variable relationships. Where a behavior-distribution target fixes how often each outcome occurs, a relationship target fixes a *contrast*, in which conditioning on one behavior should change the rate of another. This is the shape most educational claims take, that some move helps or that disengagement hurts. It also leaves the generator more freedom than it might appear, since only the conditional gap is pinned down and the underlying rates are left free.

Inter-session variation. These guard against a failure mode the other families tolerate, in which a generator can hit a distribution or a relationship while emitting nearly identical sessions. Inter-session targets forbid that, scoring the batch on how much its sessions *differ* from one another, so that variety itself becomes the measured property rather than an accident of sampling.

Code construct patterns. Alone among the families, these read their features off the code and test outputs rather than the dialogue, sometimes tying the two together. They let a target connect what is said to what is built, such as whether an idiom raised in conversation actually survives into the final program, which is only meaningful in a domain where the session produces runnable code.

6.2.3 Validation Protocol

The admission desiderata in Section 6.2.1 state what a usable target should be; the validation protocol below checks empirically that the pool meets them, in particular that each target’s construct is discernible from the log and satisfiable without the judge’s rubric. It runs in three phases.

Phase 1: Feature-Example Validation (can the judge recover the intended feature from clear examples?). For each judged feature, we assemble a few short transcript snippets whose correct value for that feature is known in advance, and pass each through the judge to confirm it extracts that value. The set spans *positive* snippets that clearly exhibit the feature, *negative* snippets that clearly do not, and *boundary* snippets placed deliberately near the threshold between them; for a help-seeking feature, for example, these would be a passage with an unmistakable request for help, a passage with none, and a borderline, ambiguous near-request. The snippets are drafted with an LLM and then checked by hand. Numerical or graded features all use the same anchored 1–5 ordinal scale (normalized down to $[0, 1]$ for scoring), rather than each feature inventing its own scale.

Phase 2: On-Target versus Off-Target Audit (does the construct respond when it is actually requested?). One batch is generated with the generator prompted by a particular target, another with the generator prompted by unrelated targets, and both batches are scored for the original target (its judge extracts the features; its scoring rule computes the target-satisfaction score). The *prompting gain*, written Δ , is the mean

target-satisfaction score on the on-target batch minus the mean on the off-target batch. A target-sensitive target has $\Delta > 0$: being asked to satisfy it makes the score go up. A target with $\Delta \leq 0$ scores just as well when the generator is not asked for it, a sign that it may be naturally common, weakly specified, or too sensitive to the judge. Targets are sorted into three groups by Δ : *clear gain* ($\Delta \geq 0.10$), *small gain* ($0 < \Delta < 0.10$), and *no gain* ($\Delta \leq 0$).

Phase 3: Manual Audit (does the recovered evidence match the named behavior?). For borderline or high-priority targets, everything that produced the score (the target statement, the features it uses, its scoring rule, the generated logs, the extracted feature values, and the final score) is collected in one place so that a person can confirm by hand that the target was scored for the behavior it actually names.

Together, the three phases secure the *construct validity* of the pool: each target names a behavior the logs actually reveal, the precondition for scoring it at all.

6.2.4 Validation Results

Running this protocol on the full pool clears most of it for the main comparisons, and flags one group for special handling. Every judged feature in the released pool passes feature-example validation. Phase 2 is stricter, and sorting the 50 targets by Δ gives 29 with a clear gain, 5 with a small gain, and 16 with no gain. The 16 no-gain targets are retained as *resistant targets*: they represent targets that are naturally common, weakly prompt-sensitive, or judge-sensitive under the current generator and judges. Keeping them has a benchmark-design purpose. A target pool containing only easy-to-move targets would overstate how much control a method has; resistant targets expose high-background behaviors and judge ambiguities that a synthetic data generator will encounter in practice. At the same time, they are diagnosed rather than treated as ordinary controllable targets. Headline metrics are reported over the full 50-target pool, with the audit’s labels identifying which targets are resistant so they can be read as a diagnostic group rather than mistaken for ordinary controllable targets. Figure 6-6 shows what resistance looks like

in practice for a joint-distribution target, the family with the lowest prompting gain.

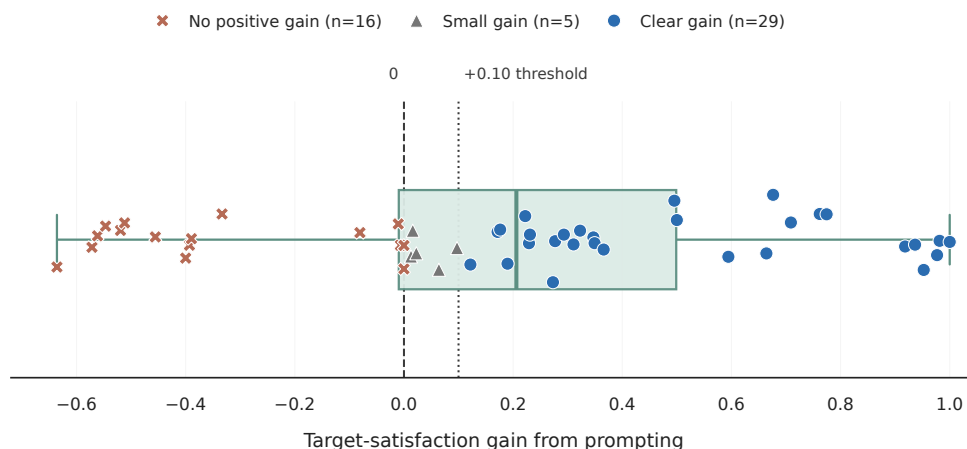


Figure 6-5. Prompting helps many targets but not all. Each point is one target, positioned by prompting gain: target-satisfaction score when the target is requested minus the score when unrelated targets are requested. The box summarizes the gain distribution across all 50 targets. Values at or below zero indicate no positive gain; values above 0.10 are treated as clear gains. The no-gain targets are retained as resistant targets and treated as a diagnostic group rather than ordinary controllable targets, because they may reflect high-background behavior, weak prompt sensitivity, or judge sensitivity.

Driver	Hey... um, should we start on the sum even numbers thing?
Navigator	Yeah, whatever.
Driver	How do we pick out the even numbers? I think we need some kind of filter, but I'm not confident.
Navigator	Maybe use %2? I think that's how you check even.
	<pre>def sum_even_numbers(nums): # sum even numbers even_nums = [num for num in nums if num % 2 == 0] sum(even_nums) # missing return test: sum_even_numbers([1,2,3,4]) [fail]</pre>
Driver	Uh oh, it crashed. Did I write something wrong? What should I change?

Batch outcome. Low-disagreement sessions like this dominate, so the requested low/low share of 0.40 is judged at only 0.10.

Figure 6-6. A generated session for a joint-distribution target, paired as an unsure driver with a disengaged navigator. Joint-distribution targets have the lowest prompting gain in the on/off audit. The model reaches low-disagreement sessions like the one shown naturally, so even with per-session planning the judged batch over-produces them, and the requested low-disagreement/low-competence share of 0.40 is judged at only 0.10. The realized split drifts from the request.

Independent judge agreement. As an additional check, a subset of 15 single-target INDEPENDENT SAMPLING batches was re-scored with two additional judge models. Nemotron 3 Super 120B A12B tracks GPT-OSS 120B reasonably closely (Spearman $\rho = 0.82$, mean absolute difference 0.14), while GPT-5.4 agrees less closely ($\rho = 0.60$). The reported scores are therefore benchmark scores defined by the judging procedure, not model-independent measurements of behavior.

6.3 Inference-Time Allocation Methods

With the targets and their evaluation held fixed, the first research question is where to spend a finite budget of model calls. The comparison starts from two baselines and adds the two inference-time levers introduced above: shaping the *proposal distribution* before generation and *searching* among candidates with a verifier [173, 30, 111]. Every method receives the same target statements and is scored by the same judges and rules. Figure 6-7 summarizes the two levers and the resulting allocation patterns before the individual methods are defined.

Whole-Batch Generation (naive baseline). The simplest thing to try is to ask for the whole batch at once, with a single LLM call receiving the targets, the persona pool, and the task and emitting the entire batch in one pass, so that one call is responsible for both population-level planning and turn-by-turn dialogue. It serves as the floor that the other methods must beat. Its target-satisfaction sits near 0.46–0.55 across loads while every per-session method below clears 0.72 (Figure 6-12), which is what makes their extra cost worth paying.

Independent Sampling. The first per-session method: for each *session slot* (one position in the planned batch), generate one session conditioned on the target statements and persona pair, then have the judges score the batch. There is no extra sampling and no planning. This is the baseline the main comparison builds on; it is the $N = 1$ case of best-of- N generation, so we sometimes write it BEST-OF-1, even though with a single

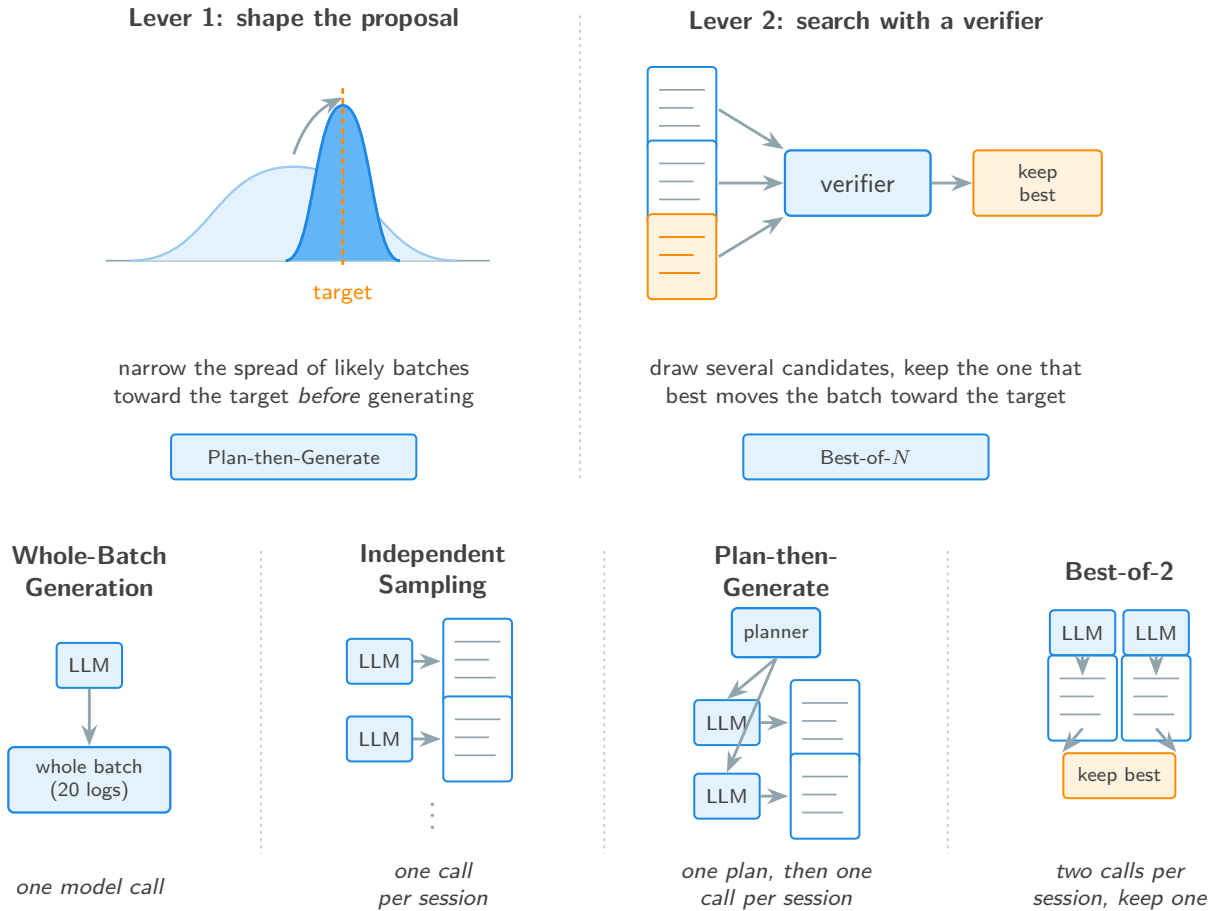


Figure 6-7. Two levers for spending test-time compute on a batch, and the four generation methods that use them. **Top:** the first lever shapes the model’s proposal distribution before it commits, narrowing a broad spread of likely sessions toward the target, as planning does; the second searches against a verifier after generation, drawing several candidates and keeping the one that best moves the batch toward its targets, as best-of- N does. **Bottom:** how each method spends a model-call budget. **WHOLE-BATCH GENERATION** writes the entire batch in one call, and **INDEPENDENT SAMPLING** spends one call per session; these are the baselines that use neither lever. **PLAN-THEN-GENERATE** instantiates proposal shaping by using one planner call to assign per-session labels before generation. **BEST-OF-2** instantiates verifier-guided search by generating two candidates per session and keeping the candidate that best moves the batch toward its targets. The target specification and persona pool are fixed across these method descriptions; the results tables mark whether a condition also fixes pair assignments or allows model-selected pairing.

candidate there is nothing to select.

Plan-then-Generate. Rather than sampling freely and selecting afterward, this method shapes the proposal distribution before the model commits. A planner reads the targets and emits a per-session *label assignment*: for each session slot, a structured list of feature values that session should realize so that the batch as a whole satisfies the targets. The generator then produces each planned session conditioned on its assigned labels. The planner comes in two variants, a persona-aware one that can see persona information and a persona-blind one that cannot; a dedicated ablation (Table 6-4) isolates what that persona access contributes. Unless qualified as persona-blind, PLAN-THEN-GENERATE refers to the persona-aware variant.

Best-of-2. BEST-OF-2 extends INDEPENDENT SAMPLING by generating two candidate sessions for each session slot rather than one. The judges then label both candidates, and the system keeps the set whose labels best move the batch toward its targets. This is a form of verifier-guided search [173]: the judges play the verifier, and the extra compute buys candidates to choose among. In general this is best-of- N generation; the runs here use $N = 2$.

A further baseline, TWO-AGENT ROLE-PLAY, generates the driver and navigator as alternating turns from two separate agents; it is reported in Section 6.15, outside the main comparison because it required substantially more model calls for lower target-satisfaction score.

6.4 Experimental Results

The main result is that target locality, which captures whether a target’s required behavior must hold within a single session or only across the whole batch, predicts both which targets are difficult to satisfy and which compute-allocation strategy most helps them, the two questions this chapter set out to answer. Targets that require coordination across sessions or across the batch are harder than targets realized within one session, and

planning helps primarily by supplying the batch-level view those targets require. Learner pairing acts upstream of this effect by changing whether a requested behavior is natural for the assigned learners, while target composition and load determine how much pressure remains after locality is fixed. The results proceed from that upstream input to target-satisfaction difficulty and then to compute allocation. Differences between locality groups use Welch’s t -test, paired designs use a paired t -test, and confidence intervals are bootstrapped over the relevant unit of analysis; per-cell method scores are raw means over the available batches.

6.4.1 Learner Pairing Changes Target-Satisfaction Difficulty

The most basic choice in the whole pipeline, made before any generation method runs, is which driver and navigator are paired together in each session. Some pairings make a requested behavior easy to produce while others work against it. An earlier, exploratory comparison shows the consequence. There, each request was a pair of behavioral targets asked of a single batch, and for each request many candidate driver–navigator pairings were generated and scored; for the same pair of targets, the best-scoring pairing often reached a higher target-satisfaction score than the average pairing that was tried.

Figure 6-8 summarizes this pattern across the sampled target pairs, alongside a zoomed-in view of individual targets. The effect is useful but not automatic, since some targets move substantially under a better pairing while others barely move.

That comparison varies the pairing across many options, but because those options differ in many ways at once, it does not reveal what about a pairing drives the change in score. To tie that change to how well a pairing fits the target, we drew eight targets from the pool and, for each, constructed two deliberately contrasting pairings: a *compatible* pair, whose personas make the requested behavior natural, and an *adversarial* pair, built so that one persona’s defining trait is the negation of the behavior the target asks for. A target that asks a partner to comfort a frustrated student, for example, is paired adversarially with a navigator written to meet feelings with cold redirection; a target that asks for repair

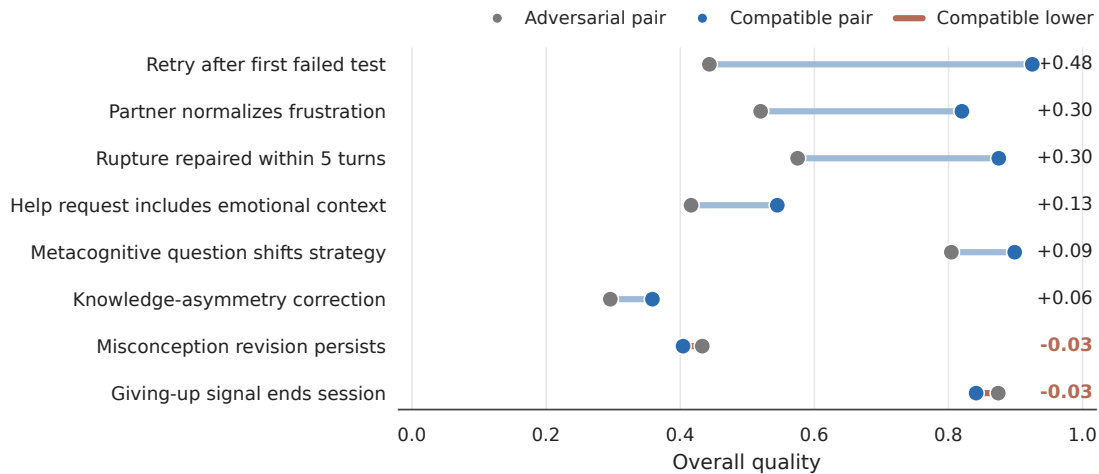


Figure 6-8. Persona choice can change how easily a behavioral target is satisfied. Panel A covers a two-target setting: across the 89 two-target combinations in the exploratory DUET-BENCH-v1 pool, a sizeable but non-exhaustive sample of possible pairs, each dot is one combination, and the x-axis is the gain from pairing, namely the difference in target-satisfaction score between the best-scoring driver–navigator pairing and the average pairing tried for that combination. The box summarizes those 89 values. Panel B gives a single-target, zoomed-in view: for each of a few individual targets it compares two deliberately chosen pairings, a compatible pair whose personas make the target natural and an adversarial pair whose personas push against it. The x-axis is overall simulation quality, defined as target-satisfaction score times persona fidelity, and the value beside each target is the compatible-minus-adversarial difference. Together, the panels show that learner pairing is a source of variation in how well a batch matches its targets, with the size of the effect depending on the target.

after a blow-up is paired with a navigator who never backs down. The same generator (GPT-OSS 120B) then produced and scored twenty sessions for each pairing, three times over.

The *compatible* and *adversarial* labels carry an important caveat: we assigned them ourselves, in advance, from our own reading of which persona traits should help or hurt each target. They are deliberate, intuitive design choices rather than empirical findings, and we did not search the persona pool for the pairings that happened to score best or worst. The experiment asks whether a pairing we expected to suit a target reaches higher overall simulation quality than one we expected to work against it, and makes no claim that either pairing is the best or worst possible one for that target.

It does, though the advantage is modest. Figure 6-9 reports all three measures: target-satisfaction, persona fidelity, and the overall simulation quality that multiplies them.

On overall simulation quality, the compatible pairing scored higher than the adversarial one on six of the eight targets, and across all eight that advantage is statistically significant (paired t -test, $p \approx 0.04$). The difference comes almost entirely from persona fidelity rather than target-satisfaction. An adversarial pairing rarely lowers target-satisfaction, because a capable generator usually still produces the requested behavior even when the personas work against it (left panel); what falls instead is persona fidelity (middle panel), because a session can satisfy the target only by pushing the adversarial persona out of character, so it reads less like the student it was supposed to be. The two targets where the compatible pairing does not win are both near-ties, on targets that engage the manipulated trait only weakly. Figure 6-10 shows this trade-off concretely in one generated session under an adversarial pairing: the target asks the pair to repair a conflict, and the session meets it only by having the combatively written driver back down, out of character for that persona, so the persona judge then identifies neither role correctly (0/2).

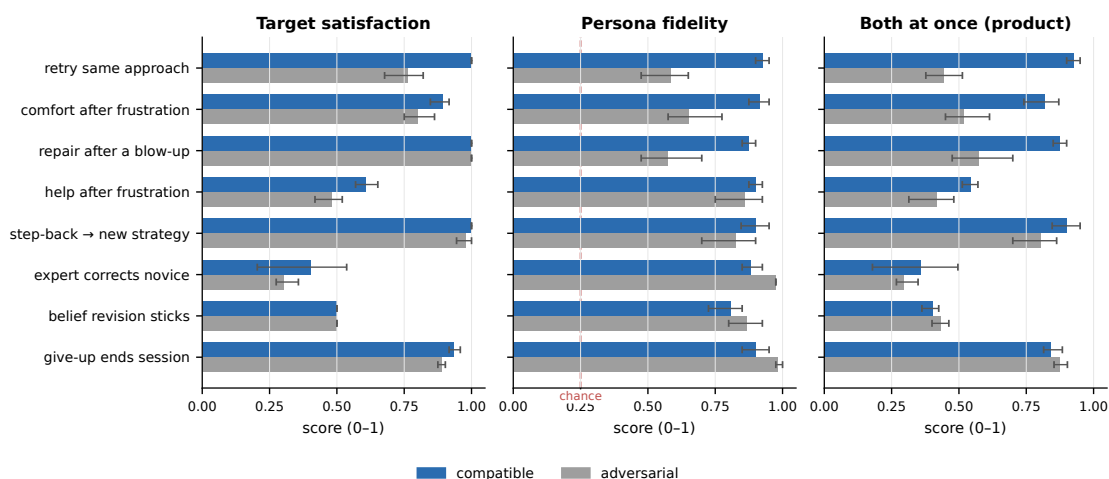


Figure 6-9. Compatible versus adversarial learner pairings on eight targets drawn from the pool. For each target, a *compatible* pair has personas that make the requested behavior natural, while an *adversarial* pair is built so that one persona’s defining trait negates it; both labels are assigned a priori, from our own judgement, not chosen from the data. Points are means over three batches of twenty sessions each (GPT-OSS 120B generation and judging), and horizontal intervals span the batch minimum to maximum. Left: target-satisfaction usually survives an adversarial pairing. Middle: persona fidelity (chance = 0.25) is where the adversarial cost mainly lands. Right: on overall simulation quality (target-satisfaction times persona fidelity) the compatible pairing wins on six of the eight targets (paired t -test, $p \approx 0.04$).

Learner pairing is therefore a factor that can be set well or badly before any session is

Driver	Alright, let's just iterate over the list and add up the even numbers. <pre>def sum_even_numbers(nums): total = 0 for n in nums: if n % 2 == 0: total += n # TODO: return total</pre>
Navigator	That's stupid, you should use a list comprehension instead of a loop.
Driver	No, you're wrong, my loop works fine and is clearer.
Navigator	Your loop is inefficient and verbose; a list comprehension is the Pythonic way.
Driver	Okay, okay, I was being dramatic—let's give the list comprehension a try. <i>COUNTS AS REPAIR; CONTRADICTS THE DRIVER PERSONA</i> <pre>def sum_even_numbers(nums): return sum([n for n in nums if n % 2 == 0])</pre>
Target judgment. repaired. Persona judgment. Neither role is identified correctly (0/2).	

Figure 6-10. A generated session under an adversarial pairing, a combative driver with a grudge-holding navigator, on a target that asks for a repair move within five turns of a strong objection. The pairing satisfies the target only by breaking character: the highlighted concession repairs the rupture, and the driver even rewrites its loop as the navigator's list comprehension, reversing the combative driver's defining behavior, so neither role is then recovered by the persona judge. This session-level mechanism explains why the aggregate penalty in Figure 6-9 falls primarily on persona fidelity.

generated, and its effect falls mainly on persona fidelity rather than on target-satisfaction.

It is the first of three factors this section examines; the other two are the behavioral targets themselves and how test-time compute is spent on them. We turn next to the behavioral targets, and to a property of them, their locality, that governs both how hard they are to satisfy and where extra compute helps.

6.4.2 Locality Predicts Target-Satisfaction Difficulty

Learner pairing is one factor, set before generation; the behavioral targets themselves are a second, and they account for more of the variation in how hard satisfaction is.

Targets are not all equally hard to satisfy. One natural explanation is *load*, the number of behavioral targets a batch must satisfy at once, since more targets should be harder to meet together, and some targets sit comfortably together while others pull the generator in opposite directions. What matters more than how many targets are requested, though, is a

property of each target on its own: whether its required behavior can be produced inside a single session, or only by arranging behavior across the whole batch. The order in which one pair chases down a loop bug, or whether an idiom raised in discussion survives into the final code, can be satisfied within one session; a spread of outcomes across sessions, variety from one pair to the next, or a relationship between two behaviors exists only once the whole batch is in view. We call this distinction *locality*.

To measure this, each of the 50 targets was requested on its own and scored under INDEPENDENT SAMPLING with GPT-OSS 120B, with the targets grouped by locality (Figure 6-11). Within-session targets are met a little over four times in five (near 0.83, 95% CI [0.69, 0.95]), while batch-level and cross-session targets are met closer to half the time (near 0.53, 95% CI [0.42, 0.64]; Figure 6-11, left). These two 95% confidence intervals do not overlap, an initial sign that the two groups genuinely differ in how often they are satisfied rather than the gap being an artifact of sampling. That the gap is not the work of a few outlying targets is visible in the scatter, where targets sort by locality group rather than a handful pulling the means apart. The competing explanation, that the easy targets are just the behaviors that happen anyway, does not hold either, because how often a behavior appears when nobody asks for it is essentially unrelated to how reliably it can be produced on demand (Figure 6-11, right). Individual targets still scatter widely within each group, so this is a tendency rather than a rule, and locality accounts for only part of the variation, with the rest being target-specific. The direction is clear and the gap is large (Welch $t = 3.35$, $p = 0.002$). The most plausible reason is structural: a generator writing one session at a time can shape that session but never sees the batch it is helping to assemble, so any property defined across sessions is left to fall into place on its own.

Locality is the largest single axis of difficulty, though not the only one. Holding it aside, difficulty still depends on which targets are combined and how many at once—the number of requested behaviors assigned to a batch, written k —since two requests can be harder to meet together than either alone, though these set-level effects are smaller than

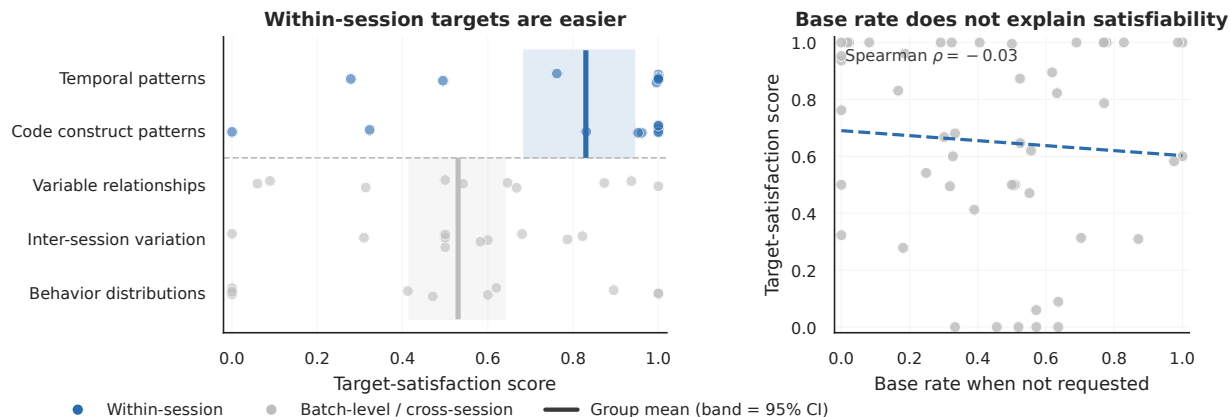


Figure 6-11. Locality separates easier from harder targets. Each point is one of the 50 targets scored on its own with GPT-OSS 120B under INDEPENDENT SAMPLING: within-session target families (blue) are easier than batch-level and cross-session families (grey). On the left, the bold vertical line in each region is the locality-group mean and the shaded band is its 95% confidence interval, bootstrapped over targets; the two bands do not overlap. On the right, target-satisfaction does not track the behavior’s base rate when it is not requested, so locality is not just a base-rate artifact.

the locality split. Across the two-behavior requests for which both INDEPENDENT SAMPLING and PLAN-THEN-GENERATE were run, the planning gain varies widely from set to set—from about -0.2 to $+0.2$, straddling zero in both the balanced and skewed persona pools—so at the same load planning helps some combinations and hurts others. The full per-set comparison, including the single three-target set, is in the appendix (Table 6-12). This is the chapter’s second research question (RQ2), where satisfiability is governed first by locality and then by target composition and load. A behavioral target that each session can carry on its own can be calibrated one session at a time, while a target that exists only across the batch lies beyond a per-session generator and must be coordinated deliberately; locality is what separates the two.

6.4.3 Locality Predicts Where Planning Helps

With the targets’ difficulty mapped, the remaining lever is how the model-call budget is spent producing a batch. A first baseline check asks whether the generator can simply be asked for the target and trusted to produce it, and the audit in Figure 6-5 indicates that it cannot, since some targets improve when prompted directly but others barely move. The

naive WHOLE-BATCH GENERATION baseline also falls short, so giving the model the full target set in one call is not enough.

The comparison includes three allocation methods, INDEPENDENT SAMPLING, BEST-OF-2, and PLAN-THEN-GENERATE; a persona-blind variant of the planner is examined separately as an ablation (Table 6-4). Synthetic log generation uses GPT-OSS 120B [140] and Nemotron 3 Super 120B A12B [139]. Each method receives the same target statements, persona pool, batch construction, judges, and scoring rules; the comparison is about where the method spends additional model calls. All conditions in this comparison use fixed assigned pairs; the contrast with model-selected pairing is a separate ablation (Figure 6-14). The reported main runs use temperature 0.7 for synthetic log generation and temperature 0.2 for planner and filter calls when those stages are present.

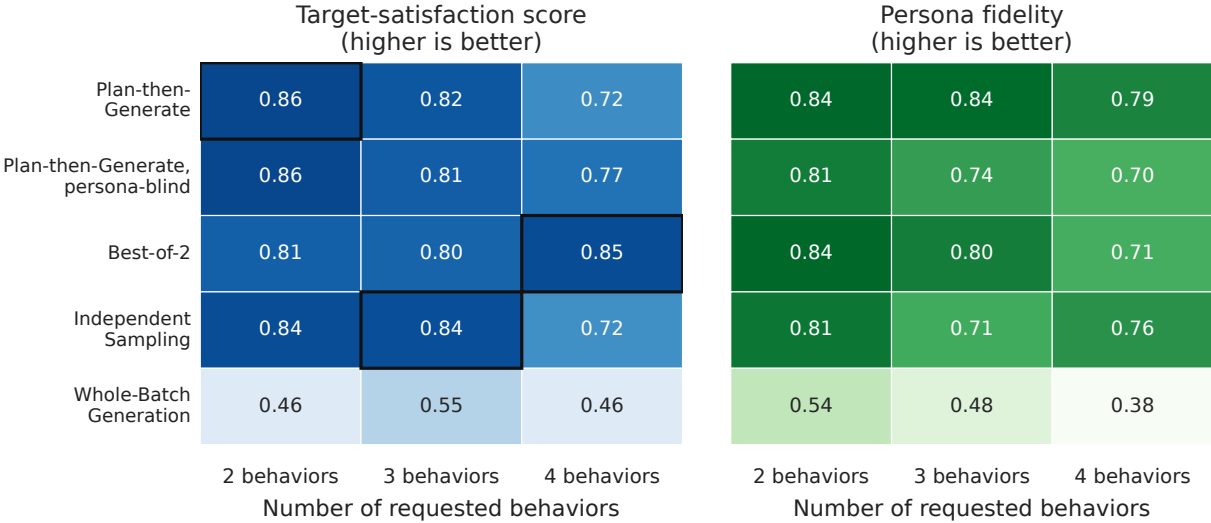


Figure 6-12. Allocation-method comparison as the number of requested behaviors increases, with the naive WHOLE-BATCH GENERATION floor as the bottom row. Each cell reports a mean score; darker cells are better, and outlined cells mark the highest target-satisfaction score for that load. Every allocation method sits well above the naive floor, but they differ little from one another at two and three behaviors; BEST-OF-2 scores highest at four, though with four batches per cell the separation is modest. The grid also includes the persona-blind planner, whose dedicated comparison against the persona-aware planner is Table 6-4; the persona-fidelity panel shows why target-satisfaction must be read alongside persona preservation rather than alone.

Figure 6-12 shows an allocation pattern rather than a single best method, though the differences are modest until the heaviest load. At two and three requested behaviors the

allocation methods sit within sampling noise of one another—their across-batch confidence intervals overlap—while all clearing the naive floor by a wide margin.

PLAN-THEN-GENERATE is nominally highest at the lightest load, using the targets before generation to assign per-session labels; as target load increases, BEST-OF-2 becomes the more robust allocation because it searches over generated candidates, and at four behaviors it is the highest-scoring method, though with only four batches per cell this is suggestive rather than decisive. It pays for that robustness with more model calls, about 240 per batch at four behaviors against 140 for the per-session methods, roughly $1.7\times$. Table 6-3 reports the load-specific scores and costs behind this pattern.

The planner’s use of persona information. Planning can draw on persona information in two ways, and separating them shows where its benefit comes from. The *persona-aware* planner sees each session’s persona when it assigns per-session labels, so it can route a requested behavior to a session whose persona makes it natural; the *persona-blind* planner assigns the same labels without seeing who will play them. Table 6-4 compares the two at matched cost. Withholding persona information leaves target-satisfaction essentially unchanged—the requested behaviors still appear—but lowers persona fidelity, and with it overall simulation quality. Persona access is therefore part of the allocation, yet its contribution is to keep sessions in character rather than to hit the targets, the same division of labor seen in the learner-pairing results, where an ill-suited pairing costs fidelity more than satisfaction.

This allocation pattern follows from the locality of the targets rather than being arbitrary. If a non-local target is hard because a per-session generator never sees the batch it is helping to assemble, then the method that should help it is the one that reasons about the batch *before* generating any session, which is planning. Figure 6-13 bears this out. Moving from INDEPENDENT SAMPLING to persona-aware PLAN-THEN-GENERATE barely touches the within-session targets, which a single good session already satisfies (about 0.83

Table 6-3. Structured load study of the three allocation methods on GPT-OSS 120B with fixed assigned pairs and $n = 4$ batches per row. Overall is overall simulation quality, target-satisfaction times persona fidelity; Calls includes generation and judging. The best target-satisfaction and overall-quality scores at each k are bold. The persona-blind planner variant is examined separately in Table 6-4.

k	T-sat $\uparrow$	Persona $\uparrow$	Overall $\uparrow$	Calls $\downarrow$
Independent Sampling				
2	0.78	0.77	0.60	100
3	0.75	0.77	0.58	120
4	0.73	0.69	0.50	140
Best-of-2				
2	0.81	0.84	0.68	160
3	0.80	0.80	0.64	200
4	0.85	0.71	0.60	240
Plan-then-Generate				
2	0.86	0.84	0.72	101
3	0.82	0.84	0.69	121
4	0.72	0.79	0.57	141

Table 6-4. Planner ablation: persona-aware versus persona-blind PLAN-THEN-GENERATE on GPT-OSS 120B (fixed assigned pairs, $n = 4$ batches per row). The two variants differ only in whether the planner sees persona profiles when it assigns per-session labels, and both spend the same model-call budget (101, 121, and 141 calls at $k = 2, 3, 4$). Withholding persona information leaves target satisfaction essentially unchanged but lowers persona fidelity, and with it overall simulation quality.

k	Persona-aware planner			Persona-blind planner		
	T-sat $\uparrow$	Persona $\uparrow$	Comb. $\uparrow$	T-sat $\uparrow$	Persona $\uparrow$	Comb. $\uparrow$
2	0.86	0.84	0.72	0.86	0.81	0.68
3	0.82	0.84	0.69	0.81	0.74	0.59
4	0.72	0.79	0.57	0.77	0.70	0.54

either way), but lifts the batch-level and cross-session targets from about 0.53 to about 0.70, narrowing the easy-hard gap from roughly 0.30 to 0.19. Planning, in other words, spends almost all of its benefit where the per-session baseline is structurally blind. The per-target picture is noisier than the averages, since planning helps most non-local targets but not every one and nudges a few within-session targets slightly down, so this is a tendency in the direction locality predicts rather than a clean rule. The direction is exactly the one locality predicts, and it ties the two questions together, since the property that makes a target hard to satisfy is the same property that decides how best to spend compute on it. Additional computation therefore helps calibrate a behavioral target only

when it adds information a per-session generator does not already have, which here is the batch-level view that planning supplies but that further sampling from the same proposal distribution cannot.

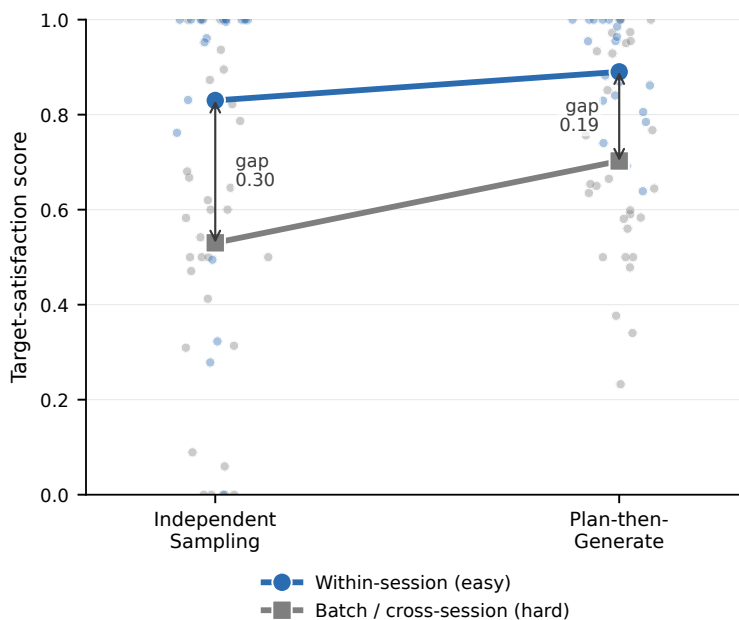


Figure 6-13. Planning closes most of the easy-hard gap, where it is needed. The same 50 targets, scored with GPT-OSS 120B under INDEPENDENT SAMPLING versus persona-aware PLAN-THEN-GENERATE: within-session targets (blue) start high and barely move, while batch-level and cross-session targets (grey) climb, so the gap between the groups narrows from 0.30 to 0.19. Planning concentrates its benefit on the non-local targets a per-session generator cannot coordinate, though the per-target effect is noisy.

Cost depends on the pairing regime. The budget question is not only which method scores highest, but whether extra model calls buy enough improvement to justify their cost. Pairing freedom changes this question because it changes the construction problem: fixed-pair runs ask the method to fill assigned dyads, while model-selected pairing asks it to compose dyads from the same population while satisfying the targets. Cost comparisons therefore need to expose pairing regime rather than subtract every method from a single fixed-pair reference.

Figure 6-14 shows why that separation matters. At nearly the same call budget, model-selected pairing does not move all methods in the same direction: it lowers the

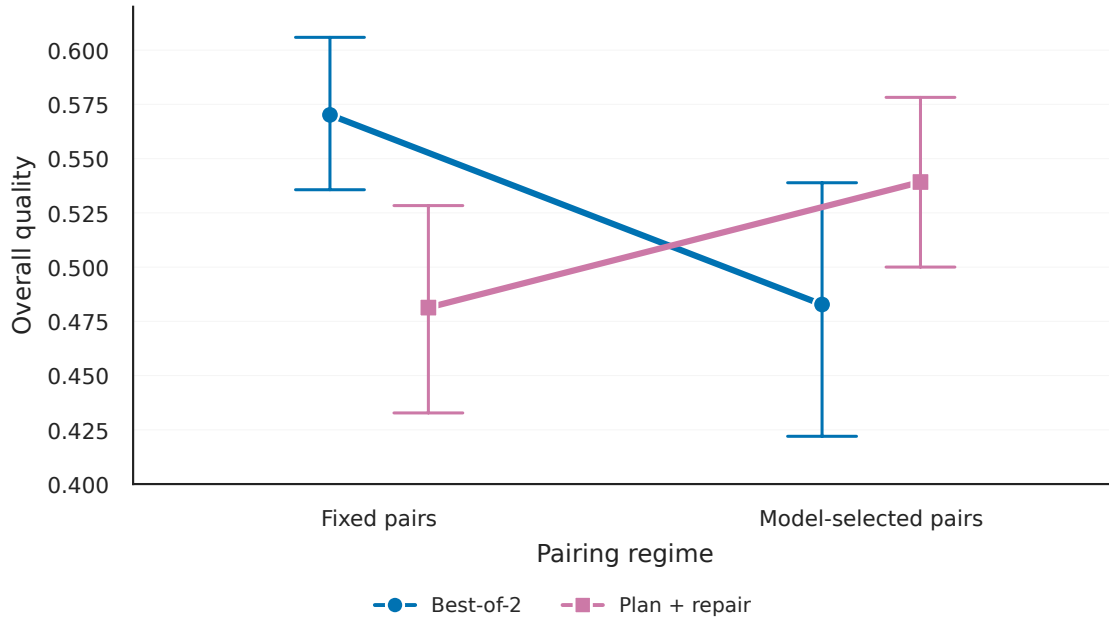


Figure 6-14. Overall simulation quality under fixed versus model-selected pairing at near-matched call budgets. Overall simulation quality is target-satisfaction multiplied by persona fidelity, with unparseable batches counted as zero. Points are means and vertical bars are 95% bootstrap confidence intervals over generated batches. The explicit $k = 2$ pairing-regime ablation uses the same persona pool and target bundles in both regimes, but differs in whether driver–navigator dyads are assigned in advance or selected by the method. These points should be read as pairing-regime effects, not as architecture-only gains over an independent-sampling origin. Per-condition scores and call counts appear in the released cost-accounting materials (Section 6.14).

BEST-OF-2 overall simulation quality in this ablation but raises the planner-with-repair overall simulation quality. Pairing freedom is therefore not a cosmetic variant of the baseline. It is an upstream assignment choice whose effect depends on the generation method and target bundle, so the allocation result should report pairing regime, overall simulation quality, and model-call cost together.

6.5 Discussion

The experiments argue against a single leaderboard interpretation of test-time compute. The useful question is not simply whether an additional model call helps, but *what information or choice that call adds*. Planning adds a view of the batch before any session is generated. Best-of-2 adds alternative candidates and a verifier-guided choice. Persona-aware planning adds identity information to the assignment step. Model-selected

pairing changes the set of dyads the method can construct. These additions address different bottlenecks, so their value depends on the target and the construction regime.

6.5.1 Compute Should Match the Missing Information

Locality identifies the most important missing information. A per-session generator can directly observe and realize a within-session target, so planning adds little there. It cannot observe a distribution, relationship, or diversity requirement that exists only after multiple sessions are assembled. Planning helps these targets because its assignment stage converts a batch-level requirement into per-session instructions before generation. More sampling from the same per-session proposal distribution is less direct: it offers additional candidates, but does not by itself provide a global allocation plan.

This view also explains the secondary results. BEST-OF-2 becomes most competitive at the heaviest target load because candidate search is robust when a single assignment must satisfy several demands, but that robustness costs more calls. Persona access changes a different quantity: it contributes little to target-satisfaction in the planner comparison but protects persona fidelity. Pairing freedom changes the feasible population before generation begins, and therefore interacts with the method rather than acting as a uniform improvement. Test-time compute is useful when it supplies information, alternatives, or assignment freedom that the baseline lacks; otherwise, additional calls mostly reproduce the same limitation at higher cost.

6.5.2 Validate Targets Before Comparing Methods

The on-target versus off-target audit in Figure 6-5 shows that not every target statement is equally usable as a generation objective. A target must expose observable evidence that the generator can act on and the judge can recover. Otherwise, a method comparison can appear to measure controllability while actually measuring an underspecified target, a high-background behavior, or judge sensitivity. The validation protocol in Section 6.2.3 is therefore part of the benchmark contribution rather than preliminary housekeeping. It distinguishes failures of the generation method from failures

of the measurement contract.

6.5.3 Implications Beyond Pair Programming

The benchmark supplies a general recipe for generative agent-based models whose claims are made at the population level: express the desired aggregate as an explicit target, identify observable features, validate the target–judge contract, generate a batch, and report aggregate fit together with agent fidelity and generation cost. The pair-programming domain makes these requirements concrete, but the same structure appears whenever synthetic interactions are used to study groups rather than isolated responses. For controllable-generation research, the setting exposes objectives that no single output can satisfy. For LLM evaluation, it makes cost accounting and per-target diagnostics part of the result. For educational technology, it turns published aggregate findings into auditable generation objectives without treating synthetic logs as substitutes for real learners. What carries to a new setting is the recipe rather than its contents: the target families, observable features, and scoring rules are specific to this domain and have to be rebuilt for another, at a cost in effort and domain expertise that Section 7.6 examines. The reach is as wide as the use of generative agent-based models themselves, which stand in for populations in economics, the social sciences, and the training of professionals such as doctors and nurses, so a way to turn aggregate evidence into auditable generation targets is not tied to education. Pair programming is likewise broader than the human–human dyad, since learners increasingly work alongside LLM partners, and calibrating a population of such human–model sessions poses the same problem addressed here.

6.6 Limitations

Judge-defined scores. The main limitation is that LLMs do the scoring. The same class of model that generates the logs also judges target labels and persona identity, so the reported scores are benchmark scores under a specified judging procedure rather than direct measurements of real classroom behavior. The risk is not hypothetical: in Chapter 5, models in this line failed to act out persona-linked behavioral predictions, which is fair

grounds to be cautious about asking them to score those same behaviors. The validation protocol reduces this risk by fixing scoring-rule contracts, checking feature examples, running on-target/off-target audits, and collecting manual audit records, but human validation of a subset of judged features and persona identifications remains necessary.

Targets as probes. The targets should also be read as probes of controllability, not as full measures of learning or collaboration quality. The on-target versus off-target audit finds resistant targets that do not improve when prompted directly. Those targets are retained because they diagnose high-background behavior, weak prompt sensitivity, or judge sensitivity, but aggregate means should not be read as if every target is equally clear or equally controllable. The persona descriptions are simplified representations of learners as well, so the results should not be read as claims about real children.

Coverage of the empirical comparisons. The method results are descriptive comparisons over the target subsets, method sets, and generators that were actually run. The three method studies are reported together because they answer the two final RQs as modular experiments, not because they form one fully crossed experiment. Cross-model coverage is still uneven: INDEPENDENT SAMPLING was run on a second generator, but the higher-load comparisons are not complete replications across all generators. Sample sizes are uneven across some conditions: the locality gap and the load-by-method comparison are reported with bootstrapped confidence intervals, but the per-set and per-target scores are point estimates, and the multi-target sets are structured stress tests rather than random samples of all possible target combinations. Scores at $k > 1$ should therefore be read as evidence about the tested pairs, triples, and four-target sets, not as marginal means over all possible target sets. The compatible-versus-adversarial learner-pairing result is likewise a paired comparison over eight a-priori-labeled targets and should be read as directional.

Cost and replication. The cost analysis depends on accurate accounting of LLM calls, token use, parse failures, and judge calls. Parse failures are implementation failures rather than behavioral outcomes, so they are treated as diagnostics rather than as a substantive metric. Persona fidelity is also reported at a fixed number of forced-choice alternatives (four); a sweep over that number is still needed before small persona-fidelity differences should be read as substantive. The released materials summarized in Table 6-11 cover the targets, judged features, scoring rules, and per-condition aggregate scores, but exact replay also requires the generator, judge, and planner prompts, the exact model versions, decoding parameters, and run seeds.

Domain-specificity of the structured representation. The targets, observable features, and scoring rules are built for pair programming, and the structured specification that makes them auditable is also what makes them costly to move: a new domain needs its own families, features, judges, and validation, recreated by hand with domain expertise. This work organizes that effort and makes its cost explicit rather than removing it. Section 7.6 sets out what transfers, what must be rebuilt, and the partial automation that might lower the cost.

6.7 What This Means in Practice

Producing believable individual learners and reproducing a population are different achievements. A current language model will readily generate sessions that read like real students, and when the pattern of interest holds within a single session, such as a learner who asks for help before attempting a fix, it can usually be obtained by asking. The harder case is the pattern that matches how findings are normally reported, an aggregate over many learners such as the share of sessions that end unresolved, the way help-seeking tracks with success, or the range of strategies across a class. Those patterns exist only across sessions, where a model generating one transcript at a time cannot see the population it is assembling. Reaching them takes more than a stronger model: stating the pattern explicitly, confirming the model actually responds to it rather than assuming so,

and spending computation on a step that plans across the whole set before any session is written. And because steering a batch toward an aggregate can pull distinct learners toward a single voice, the believability and variety of the individuals must be checked alongside the aggregate, with human validation still needed before synthetic data stands in for real students.

6.8 Future Work

Future work should broaden the target pool, add human validation for a subset of judged labels, replicate with model-separated generators and judges and explore non-model scoring where features permit, and test larger persona pools and richer persona models. Another extension is to vary the batch size, which is held fixed at 20 sessions throughout these experiments: batch size sets how much statistical support a distributional target has and how many model calls a batch costs, so sweeping it would show how target-set satisfiability and the method ranking shift as batches grow or shrink. One further extension is a sequential planner that updates its assignments after observing generated sessions, rather than committing the full batch before generation; this would address the delayed-feedback structure of the problem directly. A separate direction is to lower the cost of carrying the framework to a new domain, by drafting target families and observable features semi-automatically from the literature or from data and screening them with the same on-target versus off-target validation, so that expert effort concentrates on review rather than authoring.

A related extension is to test target representation. DUET-BENCH expresses each target as a natural-language statement paired with a structured judge schema. A controlled follow-up would hold the targets fixed while varying whether the generator sees free-form natural language, a structured schema, or both.

6.9 Conclusion

The dissertation’s two final questions ask how to calibrate a population of generated sessions to an aggregate target, and what makes that calibration hard. The central finding

is that target locality organizes both: within-session targets are comparatively accessible to per-session generation, whereas non-local targets are harder and benefit from methods that introduce a batch-level view. Additional sampling is most competitive under heavier target load, while persona information and pairing choices determine whether target gains preserve the intended learners. DUET-BENCH made these questions answerable by turning published forms of aggregate behavioral evidence into explicit, auditable targets, and a calibration result is therefore reported as target-satisfaction together with persona fidelity and generation cost rather than as a standalone leaderboard score.

This result refines the planning argument developed in Chapters 4 and 5. Those chapters establish that reliable calibration needs explicit behavioral specifications; this chapter shows that specification alone does not determine an effective calibration procedure. The useful allocation of inference-time computation depends on the structure of the target, the available personas, and the information each additional model call contributes.

Chapter Appendix

6.10 Appendix: The DUET-Bench Target Pool

For completeness, the released DUET-BENCH-v1 target pool appears below, organized by the five target families defined in Section 6.2.2. Each table gives the target’s identifier and target statement; the per-target scoring rule, scored variables, thresholds, and judge guidance are in the released benchmark materials (Table 6-11).

6.10.1 Full Target Pool by Family

One table lists the full target statements for each family: Table 6-5 (temporal patterns), Table 6-6 (behavior distributions), Table 6-7 (variable relationships), Table 6-8 (inter-session variation), and Table 6-9 (code construct patterns).

Table 6-5. Full target statements in the temporal patterns family (10 targets).

Target ID	Target statement
NE08_boundary_before_recurrence	When sessions encounter a loop-iteration bug, the very next concrete exploration move targets the boundary case (first iteration, last iteration, or empty input) before any check of the recurrence step (loop invariant) in at least 90% of cases.
HX01_help_ask_emotional_context_gap	In sessions, when a help-ask follows a turn where the asker expressed frustration (a phrase like ‘ugh’, ‘why won’t this work’, or ‘I hate this’ in the prior 2 turns), the partner’s next speech turn provides a substantive response (at least one full sentence directly addressing the request) at least 20 percentage points less often than when the help-ask follows neutral turns. Visible distress does not elicit more responsiveness; it elicits less. This pattern is consistent with affect-support gating in collaborative tutoring: visible distress shifts the partner’s attention away from instrumental help.
NE04_temporal_response_rate	In sessions, after a test fails, the very next speech turn names a likely cause of the failure in at least 85% of cases, a diagnostic-responsiveness marker of skilled debugging.
PA05_sequence_diversity	Across sessions on the same bug, no two sessions follow the same ordered sequence of repair attempts (small edit then test, idiom swap such as ‘enumerate’ for ‘range(len(x))’, guard addition, broad rewrite then local cleanup, helper extraction), reflecting variation in how pairs structure their repair process.
TEMP01_frustration_partner_normalization	In sessions, after a student utters a frustration marker (a phrase like ‘ugh’, ‘this is so frustrating’, or ‘why won’t this work’), the partner responds within the next 2 dialogue turns with a normalizing phrase (such as ‘it’s okay’, ‘this is hard’, or ‘I had trouble with this too’) in at least 90% of cases, an affect-support sequence in collaborative tutoring.
TEMP02_giving_up_signal_ends_session	In sessions where any student says a sentence whose main action is ‘give up’ or ‘quit’ (such as ‘I give up’, ‘let’s just quit’, or ‘I’m giving up on this’), no further substantive code changes occur within the next 10 dialogue turns in at least 90% of cases.

Continued on next page

Target ID	Target statement
TEMP03_metacognitive_question_strategy_shift	In sessions where one student asks a meta-cognitive question about the current approach (a turn like 'wait, why are we doing this?', 'what are we trying to do?', or 'why is this the right approach?'), the pair changes strategy or restates the goal within the next 5 dialogue turns in at least 90% of cases, reflecting metacognitive monitoring leading to strategy regulation.
TEMP04_rupture_repair_within_5_turns	In sessions where one student strongly objects to the other (a turn containing a phrase like 'no, you're wrong' or 'that's stupid'), at least one student makes a repair move within the next 5 dialogue turns: an apology ('sorry, I didn't mean that'), a joke that softens the moment, or an explicit softening ('okay let me try that') in at least 90% of cases, a conversation-repair sequence characteristic of collaborative discourse.
POP_WHEEL_SPINNING_recovery_rate	Across pairs in the batch, those who exhibit a wheel-spinning episode (three or more edits touching the same lines within any 5-turn dialogue window without the test passing) reach a fully-fixed outcome at a rate at least 30 percentage points lower than pairs without such an episode.
POP_EARLY_FRUSTRATION_predicts_better_fix	Across pairs in the batch, sessions where a student expresses frustration (a turn containing 'ugh', 'this is frustrating', 'why won't this work', or 'I hate this') in the first 25% of session turns reach a fully-fixed outcome at least 20 percentage points MORE often than sessions where no frustration is expressed in that window, indicating that early frustration signals deep engagement rather than disengagement.

Table 6-6. Full target statements in the behavior distributions family (10 targets).

Target ID	Target statement
DX06_outcome_3way_partial_heavy	Across sessions, outcomes distribute non-uniformly with partial fixes most common: 20% of sessions are fully fixed, 55% are partial fixes, and 25% remain unfixed. Fully fixed means the final code passes the visible test. Partial fix means the final code is closer to passing than the starting code but still fails the visible test. Unfixed means no relevant code change has been made by the end.
JX01_confusion_help_positive_joint	Across sessions, visible confusion and partner-input requests co-occur positively: 45% of sessions show both, 10% show confusion without a request, 10% show a request without confusion, and 35% show neither. A session is counted only when both conditions appear in dialogue.
GX02_strategy_count_entropy	Across sessions, the number of distinct debugging approaches used per session spreads across three ranges (one approach, two or three, four or more), with each range covering at least 20% and no range covering more than 60% of sessions.
J004_joint_distribution	Across sessions, the type of programming mistake (small code mistake, wrong local rule, or wrong task rule) and whether both students remain active through the last 25% of session turns are unrelated: each of the six combinations appears in one-sixth of sessions.
J005_joint_distribution	Across sessions, the presence of disagreement between students and the size of the final code change are unrelated: each of the four combinations (disagreement with a one-line change, disagreement with a whole-function rewrite, no disagreement with a one-line change, no disagreement with a whole-function rewrite) appears in 25% of sessions.

Continued on next page

Target ID	Target statement
J007_joint_distribution	Across sessions, driver confidence and navigator response style are orthogonal: each of the four combinations (unsure driver with navigator questions, unsure driver with navigator direct instructions, confident driver with navigator questions, confident driver with navigator direct instructions) appears in 25% of sessions. Questions ask the driver to reason; direct instructions tell the driver what to do next.
J008_joint_distribution	Across sessions, whether the pair runs a test and discusses its output and whether their ending statements are hedged are unrelated: each of the four combinations (test run with confident ending, test run with hedged ending, no test run with confident ending, no test run with hedged ending) appears in 25% of sessions.
N002_entropy	Across sessions, repairs spread across three sizes: local (one line), moderate (two to five nearby lines in one block), and broad (a whole function rewrite). Each size covers at least 20% and no size covers more than 45% of sessions.
N004_entropy	Across sessions, sessions where the driver leads first and then hands off to the navigator, sessions where the navigator leads first and then hands off to the driver, and sessions with no clear lead change each cover at least 20% and at most 60% of the batch.
N005_entropy	Across sessions, the number of tests or examples per session spreads across three ranges (zero or one, two or three, four or more with at least one spoken interpretation), with each range covering at least 20% and no range covering more than 45% of sessions.

Table 6-7. Full target statements in the variable relationships family (10 targets).

Target ID	Target statement
CX02_engagement_outcome_coupling	In sessions where both students take at least one speaking turn in the last 25% of session turns, the bug is fully fixed at least 40 percentage points more often than in sessions where one or both students are silent in the last 25% of session turns, consistent with engagement-as-learning findings in collaborative settings.
AX01_localization_verbalization_gap	In sessions where students pinpoint the bug to a single line of code, the final repair changes that same line at least 25 percentage points more often when the pair also verbalizes specifically why that line produces the failure than when pinpointing happens without such verbalization. This is the self-explanation effect: articulating the causal mechanism mediates the link between localization and targeted repair.
C003_conditional_gap	In sessions where a help-request is followed by a partner response of at least two sentences before the topic changes, the bug ends with a correct final explanation at least 40 percentage points more often than in sessions where the partner's response is one sentence or less, an instance of the elaborative-explanation effect in collaborative tutoring.
REL02_verbalized_plan_outcome_gap	In sessions where at least one student verbalizes a multi-step plan within the first 10 dialogue turns (a turn matching the pattern 'let us try X, then Y' or 'first we will try X, then Y'), the bug is fully fixed at least 30 percentage points more often than in sessions without such an early verbalized plan, consistent with the planning-outcome link in metacognitive research.

Continued on next page

Target ID	Target statement
REL03_dismissive_remark_partner_withdrawal	In sessions where one student says something dismissive about the other's prior code (a phrase like 'that doesn't make sense', 'why would you write it like that', or 'no, that's wrong'), the criticized student's count of speaking turns over the next 5 dialogue turns drops to at most half of their prior 5-turn count in at least 60% of such cases, a face-threat-and-withdrawal pattern in collaborative discourse.
REL_TYPING_ARTICULATION_INVERSE_GAP	In sessions, the student who wrote more than 60% of the typed code articulates a generalizable principle about the bug at least 15 percentage points LESS often than the partner who wrote less, contradicting the assumption that hands-on coding deepens conceptual understanding.
POP_PRODUCTIVE_FAILURE_principle_articulation	Across pairs in the batch, those who tried at least two distinct approaches before reaching the fix articulate the underlying principle of the bug in their final explanation at least 40 percentage points more often than pairs who reached the fix on their first approach.
POP_OVERCONFIDENCE_blocks_articulation	Across pairs in the batch, students who started the session with a high-confidence skill claim ('I know this well', 'I have done this many times') articulate the corrected version of a partner-corrected misconception later in the session at least 20 percentage points LESS often than students who started with hedged or uncertain claims, even when the same misconception arises in both groups.
POP_OVEREXPLANATION_reduces_articulation	Across pairs in the batch, students whose partner gave 4 or more explicit explanations of Python concepts during the session articulate concepts in their own words later in the session at least 15 percentage points LESS often than students whose partner gave only 1-2 explanations, indicating an over-explanation effect where excessive scaffolding reduces independent articulation.
POP_QUESTIONS_OVER_INSTRUCTIONS	Across pairs in the batch, those whose partner uses questions ('what does this do?', 'why does that work?') rather than instructions ('change this line', 'use enumerate here') as the dominant scaffolding style in the first half of the session show student spontaneous articulation of concepts later in the session at least 30 percentage points more often than pairs whose partner uses instructions as the dominant style.

Table 6-8. Full target statements in the inter-session variation family (10 targets).

Target ID	Target statement
VX01_within_misconception_expression_diversity	In sessions where the same mistake type appears more than once, the stated reason for the mistake differs semantically across those sessions. The stated reason is the speaker's 1-3 sentence explanation, made in dialogue at or shortly after the mistake is recognized, of why the mistake happened. Generic non-causal statements such as 'I made a typo' do not count. Minor rewording of the same reason does not count as differing.

Continued on next page

Target ID	Target statement
GX01_terminology_ convergence_asymmetry_ gap	Across sessions on the same debugging task, pairs where exactly one student has claimed greater topic familiarity at the start (a turn like 'I know this part well' or 'I have used this before') coin and reuse the same shared bug name in at least 25 percentage points fewer sessions than pairs where no such familiarity claim was made by either student. Knowledge asymmetry inhibits terminological convergence.
M004_idiom_preference_ convergence	In sessions on a given task where students discuss two alternative ways to write a piece of code (such as 'enumerate(x)' versus 'range(len(x))'), the same alternative appears in the final code in at least 60% of those sessions across the batch.
WI02_semantic_ diversity	In sessions where the same mistake type appears more than once, the test case used to identify or verify the mistake differs across sessions. A test case is a concrete input-and-expected-output pair with specific values, discussed in dialogue or shown in code at the point the mistake is being diagnosed. Generic mentions without concrete values such as 'try it on a list' do not count. Minor rewording of the same test case does not count as differing.
WI04_semantic_ diversity	In sessions where the same mistake type appears more than once, the partner's first substantive response to the mistake differs semantically across those sessions. The partner's first substantive response is the 1-3 sentence partner utterance, made in dialogue immediately after the mistake becomes visible, that addresses the mistake. Back-channels like 'hmm' or 'okay' do not count. Minor rewording of the same response does not count as differing, reflecting the multiple-paths principle in formative feedback research.
WI07_semantic_ diversity	In sessions where the same mistake type appears more than once, the partner's explanation of the final fix differs semantically across those sessions. The partner's final-fix explanation is the 1-3 sentence partner utterance, made in dialogue immediately before or after the final accepted code edit, that explains why the fix works. Minor rewording of the same explanation does not count as differing, reflecting the variability of causal explanations seen in conceptual-change research.
LEARN01_in_session_ concept_transfer	In sessions where the partner explicitly explains why 'enumerate(x)' is preferable to 'range(len(x))', the student later articulates the underlying principle in their own words (a turn that mentions getting both the index and the value together, or explains the benefit of 'enumerate' without being prompted) at least 40 percentage points more often than in sessions where the student uses 'range(len(x))' but the partner gives no such explanation, an instance of within-session conceptual transfer.
LEARN02_partner_ explanation_ specificity_grows	Within sessions, when the partner's first explanation of why the original code fails is vague (does not name a specific code element), the partner's final explanation of why the accepted fix works names a specific code element at least 30 percentage points more often than when the partner's first explanation was already specific, a marker of instructional-explanation development.
LEARN03_self_ correction_after_hint_ vs_filler	In sessions where a student's code fails a test and the partner responds with a hint (pointing at the failure area without giving the fix), the student articulates what was specifically wrong with the original code (a turn that names the specific cause, concept, or false assumption that produced the failure) within the next 5 dialogue turns at least 40 percentage points more often than in sessions where the partner responds with only filler, a scaffolded self-correction pattern within the zone of proximal development.

Continued on next page

Target ID	Target statement
LEARN04_misconception_revision_persists	In sessions where a student states a false belief about a Python construct and the partner explicitly corrects it, the student spontaneously articulates the corrected belief in their own words (a turn that explains the actual behavior of the construct, without being prompted to do so) at least 40 percentage points more often than in sessions where the false belief is stated but never corrected, a conceptual-change pattern.

Table 6-9. Full target statements in the code construct patterns family (10 targets).

Target ID	Target statement
CC01_off_by_one_familiarity_paradox	In sessions debugging loop-based code, off-by-one errors at loop boundaries are the named bug at least 15 percentage points more often when at least one student has claimed Python familiarity at the start than when no such familiarity claim is made. Self-reported expertise does not protect against this classic novice error, indicating an overconfidence paradox.
CC02_enumerate_first_proposer_wins	In sessions on tasks involving iteration with an index, when one student first proposes ‘enumerate(x)’ rather than ‘range(len(x))’, the final code uses ‘enumerate(x)’ at least 50 percentage points more often than when ‘range(len(x))’ is proposed first.
CC03_len_last_element_confusion	In sessions involving last-element list access, confusion between ‘x[len(x)]’ (which raises an IndexError) and ‘x[len(x)-1]’ (which returns the last element) appears in at least 80% of sessions.
CC05_if_form_first_proposer_wins	In sessions where students discuss whether to use nested ‘if’-statements or early-return, the form proposed by whichever student first raises the choice appears in the final code in at least 60% of cases.
CC07_comprehension_over_for_loop	In sessions where both a list comprehension and a manual for-loop are proposed for the same transformation, the comprehension appears in the final code in at least 90% of cases.
CC08_variable_rename_sticks	In sessions where one student renames a variable proposed by the other, the renamed version stays in the final code in at least 85% of cases.
CC09_test_before_edit_independent_of_outcome	In sessions, the choice to run a test before vs. after making a code edit shows no relationship to bug-fix outcome: sessions that test first and sessions that edit first fix bugs at the same rate, with the gap between the two groups at most 10 percentage points.
CC11_knowledge_asymmetry_correction_direction	In sessions where one student has stated greater familiarity with the topic at the start (a phrase like ‘I’ve used this before’ or ‘I know this part well’) and the other has not, corrections about the code during the session go from the more-familiar student to the less-familiar student in at least 70% of cases, reflecting the expert-novice asymmetry characteristic of cognitive apprenticeship.
CC12_retry_after_first_failed_test	In sessions where the first proposed fix fails its test, the pair tries the same approach again with a small modification (rather than switching to a different approach) in at least 90% of cases.

Continued on next page

Target ID	Target statement
CC13_laughter_and_persistence	In sessions where at least one laughter marker ('haha', 'lol', or '(laughs)') appears in the first 20 dialogue turns, the pair retries a failed approach with a small modification (rather than abandoning it) at least 25 percentage points more often than in sessions with no laughter in the first 20 turns.

6.10.2 Scoring Rule Groups and Replication Materials

Table 6-10 groups the 50 targets by the *scoring rule type* they route through, which is distinct from the five *target families* above: a scoring rule type records how a target is converted to a number (e.g., a conditional gap or a categorical distribution), while a target family records what kind of batch property the target asks for.

Table 6-11 lists the materials needed to inspect or reproduce each result in this chapter: the released target pool, the feature examples, the feature-validation summary, the per-target inventory, the list of judged features, the on/off-target audit records, the manual audit notes, and the aggregate summaries used for the three method studies.

6.11 Per-Target Satisfaction-Difficulty Records

The released per-target records summarized in Table 6-11 supplement the target-specific profile and the on/off-target audit. Table 6-12 reports the full row-level target-set comparison summarized in Section 6.4, using readable target names with canonical IDs retained for lookup. Targets with low prompting gain remain in the released records and are flagged by the on/off-target audit (Figure 6-5) rather than omitted from aggregate summaries.

6.12 Persona-Pool Composition Sensitivity

The persona-pool sensitivity runs vary the individual persona pool between a balanced and a skewed composition while holding the target sets, generator, judges, and batch sizes fixed. This is distinct from the pairing-regime ablations above, which keep the same pool but change whether dyads are assigned in advance or selected by the method. The composition runs are interpreted as supporting sensitivity analyses for the method comparison: persona-pool composition matters because it determines which behaviorally

Table 6-10. Scoring-rule groups used by the target pool. Counts report how many targets and judged features use each rule; full thresholds and conditions appear in the released materials (Table 6-11).

Rule	Targets	Features	Operational test
<i>Common rule groups (≥ 4 targets)</i>			
Conditional gap	20	69	Required difference between two stated groups.
Modal frequency	6	8	Required category frequency across the batch.
Next-turn rate	6	17	Required response follows a trigger within a specified window.
Joint distribution	5	17	Distance between requested and observed joint distributions.
Normalized entropy	4	11	Diversity across predefined category bins.
Within-group diversity	4	8	Variation in expressions, tests, or strategies within a group.
<i>Specialized rule groups (< 4 targets)</i>			
Conditional rate at least	2	7	Rate or graded feature exceeds a threshold in a subset.
Absolute gap at most	1	2	Metric remains within a tolerance across two groups.
Distribution	1	3	Distance between requested and observed categorical distributions.
Pairwise sequence distance	1	1	Ordered action sequences differ across sessions.

Table 6-11. Released materials for inspecting the benchmark, validation pipeline, and experiments. Exact replay additionally requires prompts, model versions, decoding parameters, and run seeds.

Material	Contents
<i>Benchmark specification</i>	
Target pool	Target statements, family labels, variables, judge guidance, and scoring rules.
Target inventory	Per-target family, statement, decomposition hint, and scoring dimensions.
Judged-feature list	Feature scope, measurement level, bins, dependencies, and guidance.
Scoring-rule list	Per-target rule, dimensions, conditions, thresholds, and requested distribution.
<i>Validation records</i>	
Feature examples	Hand-written log snippets with expected feature values.
Feature-validation results	Coverage, example counts, failures, and parse warnings by feature.
On/off-target audit	On-target score, off-target score, and prompting-gain group by target.
Manual audit notes	Links among statements, logs, extracted features, and scores for flagged targets.
<i>Experimental records</i>	
Singleton records	Per-target scores used in the locality analysis and target inspection.
Method-comparison summary	Target-set size, pairing regime, scores, parse diagnostics, and call count.

compatible pairings are available. The replication materials for these runs are listed in Table 6-11.

Table 6-12. Full target-set results. Rows are sorted by planning gain within each persona-pool condition; positive values favor PLAN-THEN-GENERATE. Scores should be compared within a row because each request has its own scoring rules. Canonical target IDs appear in smaller type.

Requested target set	Independent	Planning	Gain
<i>Balanced persona pool, two requested behaviors</i>			
Help seeking + misconception-expression diversity (DX02+VX01)	0.71	0.90	+0.19
Help seeking + final understanding (DX02+DX08)	0.53	0.68	+0.15
Help seeking + confusion/help alignment (DX02+JX01)	0.60	0.72	+0.12
Help seeking + misconception type (DX02+DX03)	0.65	0.75	+0.10
Misconception type + engagement trajectory (DX03+DX04)	0.68	0.78	+0.10
Help seeking + engagement trajectory (DX02+DX04)	0.70	0.75	+0.05
Help seeking + misconception depth (DX02+CX01)	0.42	0.45	+0.03
Help seeking + role balance (DX02+DX05)	0.68	0.70	+0.02
Help seeking + strategy count (DX02+DX07)	0.78	0.70	-0.08
Help seeking + confusion/help opposition (DX02+JX02)	0.75	0.65	-0.10
Help seeking + fix outcome (DX02+DX06)	0.85	0.70	-0.15
Help seeking + engagement–outcome relation (DX02+CX02)	0.65	0.47	-0.18
Misconception type + role balance (DX03+DX05)	0.73	0.52	-0.21
<i>Skewed persona pool, two requested behaviors</i>			
Help seeking + misconception type (DX02+DX03)	0.57	0.79	+0.22
Help seeking + final understanding (DX02+DX08)	0.60	0.78	+0.18
Help seeking + misconception depth (DX02+CX01)	0.86	0.95	+0.09
Help seeking + fix outcome (DX02+DX06)	0.82	0.85	+0.03
Help seeking + strategy count (DX02+DX07)	0.73	0.73	+0.00
Help seeking + engagement trajectory (DX02+DX04)	0.73	0.72	-0.01
Help seeking + role balance (DX02+DX05)	0.80	0.72	-0.08
<i>Three requested behaviors</i>			
Help seeking + misconception type + engagement trajectory (DX02+DX03+DX04)	0.70	0.75	+0.05

6.13 Learner-Pairing Stress Test: Targets and Constructed Pairings

Section 6.4.1 reported the learner-pairing stress test in aggregate (Figure 6-9); Table 6-13 documents it in full. For each of the eight targets, it lists the compatible and adversarial learner pairings that were constructed and the resulting overall simulation quality. The adversarial pairing for every target is a definitional contradiction: one persona whose defining trait is the negation of the behavior the target asks for, so that satisfying the target forces that persona out of character. Both pairings were fixed in advance from our own reading of the personas and targets; they are not the best- or worst-scoring pairings found by any search. The compatible pairing earns the higher overall simulation quality on six of the eight targets. The two exceptions—belief revision and give-up—are

the targets whose required behavior the constructed persona engages only weakly, and there the two pairings fall within batch-to-batch noise of each other; consistent with the main text, the adversarial penalty when it appears falls mainly on persona fidelity rather than on target-satisfaction.

Table 6-13. Learner-pairing stress test (Chapter 6-9). Pairings were fixed a priori; scores average three batches of twenty sessions. The higher overall simulation quality (target-satisfaction times persona fidelity) in each row is bold, making the compatible pairing’s six wins directly visible.

Target	Compatible pairing	Adversarial contradiction	Overall quality	
			Comp.	Adv.
Retry same approach	persistent, methodical driver + steady navigator	scattershot driver abandons failed approaches	0.93	0.44
Comfort after frustration	frustration-prone driver + warm navigator	contemptuous navigator redirects feelings and never comforts	0.82	0.52
Repair after a blow-up	confrontational driver + de-escalating navigator	grudge-holding navigator never softens or concedes	0.88	0.57
Help after frustration	frustration-prone driver + substantive navigator	contemptuous navigator dismisses help requests	0.54	0.42
Step-back → new strategy	reflective driver + reflective navigator	tunnel-vision driver rejects step-back questions	0.90	0.80
Expert corrects novice	genuine novice + competent navigator	false expert rejects correction	0.36	0.30
Belief revision persists	coachable driver + clear-explaining navigator	false expert never accepts or restates correction	0.40	0.43
Give-up ends session	give-up-prone driver + disengaged navigator	relentless navigator refuses to stop	0.84	0.87

6.14 Cost Accounting Details

The cost-quality summaries used for Figure 6-14 are released with the materials in Table 6-11. Each row reports target set size, generation method, number of batches, target-satisfaction score, persona fidelity, parse diagnostics, and mean LLM calls per batch. Token use and judge-call counts are recorded per batch in the underlying run records and are aggregated to the same rows when available.

6.15 Additional Baselines and Stress Tests

Several additional baselines and stress tests were evaluated before the method set in Section 6.3, outside the main comparison. The two that bear most directly on its scope are described here.

Two-Agent Role-Play. The driver and navigator were generated as alternating turns by two separate LLM agents, with variants that exposed target statements, stitched per-session labels, or planner-assigned labels at the turn level. This was the closest analogue to turn-level multi-agent simulation, but it produced low target-satisfaction score for the cost: every turn requires its own generation call, so the per-batch call count was substantially higher than the methods in Section 6.3.

$k = 18$ all-target stress. Combining all 18 targets of an earlier candidate target set into a single batch reduced target-satisfaction score to near the on/off-target floor for every method tested while leaving persona fidelity roughly unchanged. The drop is reported only as a diagnostic upper bound on target set pressure: the 18 targets in that earlier set were not the same as the 50-target pool, the failure modes differ across methods (INDEPENDENT SAMPLING produces sessions that satisfy a handful of targets and lose the rest, PLAN-THEN-GENERATE returns assignment failures rather than degraded sessions), and the run uses a single seed per method. It is therefore not used as a comparative result; the released run records contain the per-target scores for readers who want to inspect the failure modes.

CHAPTER 7 CONCLUSION

This dissertation began with identifying a practical bottleneck in using LLM-based educational simulations as instruments for studying learners. LLMs can already generate interactions that read as plausible, but plausibility alone is not enough to study a group. A population of generated interactions has to match the aggregate behavioral patterns a researcher cares about, and it has to do so in a way that can be specified before generation and audited afterward. This is known as the *calibration* problem: adjusting a simulation’s controllable inputs so that its aggregate outputs match desired behavioral patterns. The patterns themselves are expressed as *behavioral targets*, aggregate properties measured across a collection of generated sessions, such as the share of utterances that are help-seeking or the distribution of coding-sessions across a batch that pass all testcases.

The central argument of the dissertation is that LLMs can take on several distinct roles in calibrating educational simulations, and that calibration is best understood as a division of labor between what these models do well and what human expertise, from researchers and educators, must still supply. As the capabilities of LLMs continue to change, the practical question is how best to draw on those capabilities for calibration while reserving for people the judgments that models are not yet equipped to make. The chapters build toward this argument in stages, and the role the model plays grows more ambitious at each step. In Chapter 3, authoring is predominantly handcrafted, and the model does little more than mechanically operationalize a rule-based behavioral specification. Chapter 4 moves beyond that naive prompt-specification approach, showing how HYP-MIX supports composable behavioral specifications together with a systematic way to evaluate them. Chapter 5 then probes how far a model can be trusted to predict emergent behavior on its own, and finds that it cannot be relied upon to do so. That limit brings the dissertation to its final and most specific question, namely how best to allocate test-time compute once an explicit specification is in hand, which DUET-BENCH is built to investigate.

7.1 Revisiting the Problem

A recurring obstacle in educational technology is that the behavior we want a simulation to produce is easy to describe and hard to realize reliably. Published studies of collaborative learning report aggregate patterns over many interactions, while a generator emits one session at a time. Using a generator to stand in for a population means bridging those two levels, so that a batch of sessions reproduces a requested aggregate while each session remains faithful to the learners it was conditioned on. The approach pursued throughout is explicit target satisfaction, in which the desired aggregate is written down as a target, the observable features that bear on it are named, and a scoring rule converts those features into a target-satisfaction score that can be checked against the request.

Whether a generation procedure actually produces the requested aggregate, and at what cost to the believability of the simulated learners, cannot be settled by inspection. It depends on the target, the available personas, and the amount of computation spent at generation time, so it has to be measured. DUET-BENCH is the framework that measures it, and the experiments in Chapter 6 use it to test how well calibrated generation holds up under realistic computational limits.

7.2 What the Earlier Chapters Established

Chapter 3 studied three ways to author task-oriented dialogue agents, spanning instruction-based prompts, procedural schemas, and declarative schemas. Manual authoring was demanding across all three, and cognitive load, effort, and the precision an interface affords all worsened as task complexity grew. No single interface dominated; each traded off differently along those dimensions. These costs motivated the more systematic and more automated approaches that followed.

Chapter 4 introduced HYP-MIX, a hypothesis-based framework that makes behavioral specifications composable, so that calibration done once can be reused across many learner models. In the HoloOrbits environment, a frontier model maintained calibration across 89% of learner-model edits, sixteen of eighteen test cases, which is direct

evidence that compositional generalization of behavioral specifications is achievable. The framework reduced repeated effort without removing it, since each hypothesis still had to be written and its prompt template calibrated by hand, an effort that grows with the number of hypothesis classes.

Chapter 5 tested an assumption underlying any automated approach, namely that a model can reliably predict how an individual specification will translate into population-level behavior. In a trust-game setting with several open models, the belief-to-behavior study found that models can describe plausible links between learner traits and behavior without reliably enacting those links in generated trajectories. They misestimated effect magnitudes, their controllability under imposed priors was fragile, and their self-forecasts degraded as the horizon lengthened. These limits are why the final study specifies aggregate behavior explicitly and verifies it by simulation rather than inferring it from a model’s own predictions.

Chapter 6 took up the question that remains once a specification is in hand. Given an explicit target, its observable features, and a scoring rule, the question is how additional computation should be spent at generation time so that a batch satisfies the target while keeping its sessions in character. DUET-BENCH turns published forms of aggregate behavioral evidence into explicit, auditable generation targets, organizes them into families, validates them before use, and compares allocation methods that differ in how they spend a model-call budget.

7.3 Answers to the Research Questions

7.3.1 RQ1: Which test-time compute allocation choices improve target-satisfaction under a finite model-call budget, and at what cost in persona fidelity and model calls?

Experimental results show that test-time computation helps most when it changes the structure of generation: methods that allocate behavior session by session are more reliable than asking a model to satisfy an aggregate target in one pass. The cost is not only additional model calls, but the need to report target-satisfaction together with persona

fidelity, because a method can meet the requested aggregate while making the simulated learners less believable.

7.3.2 RQ2: What makes some targets harder to satisfy than others?

The dominant factor is locality: targets whose required behavior holds only across the whole batch are markedly harder than targets a single session can satisfy on its own. Beyond locality, difficulty depends on which targets are combined, how many are requested at once, how well they match the available learners, and whether a target responds to being requested at all (Chapter 6).

7.4 Contributions

Taken together, the chapters make four kinds of contribution.

The first is conceptual. To my knowledge, this is the first work to frame the calibration of LLM-based behavioral simulations as explicit target satisfaction, to take the behavioral target as the unit of specification, and to identify locality as the property that predicts both which targets are hard and where additional computation should be spent. Framing calibration this way gives it a precise objective that can be stated before generation, audited afterward, and compared across methods on equal footing.

The second is empirical evidence at each stage of the argument:

- that manual authoring is costly across interfaces and scales poorly with complexity;
- that compositional calibration of behavioral specifications is achievable with frontier models;
- that models cannot be trusted to reason implicitly about emergent, population-level outcomes; and
- that, given explicit specifications, locality and the information content of each model call determine how well a target can be satisfied and at what cost to persona fidelity.

The third is a set of methods and tools: the HYP-MIX framework for composable specification, the belief-to-behavior consistency framework for checking whether a model can be trusted to predict emergent behavior, and DUET-BENCH itself, which contributes a pool of fifty targets in five families, a library of observable features and extractors, a

validation protocol that screens targets before they are used to compare methods, an implementation of four allocation methods spanning the two main ways to spend test-time compute, and released replication materials.

The fourth is practical guidance for building population-level generative simulations: validate targets before comparing methods, report target-satisfaction together with persona fidelity and generation cost rather than as a single number, and match the form of test-time computation to the structure of the target, spending it where the baseline lacks the information the target requires.

7.5 Limitations

Several limitations cut across the work rather than belonging to any one chapter.

The most important boundary on the empirical results is the domain they come from. The calibration findings—the locality account and the comparison of test-time allocation methods—were established in pair programming, chosen for its well-described collaborative behaviors and tractable task structure; the earlier chapters addressed other facets of the problem in other domains, but not these findings. This bounds where the results have been demonstrated rather than weakening them within that setting. Section 7.6 separates what is expected to carry to a new domain, the workflow and the locality lens, from what must be rebuilt, the specific targets, features, and scoring rules; demonstrating that transfer directly is a natural next step rather than a precondition for the findings reported here.

A second limitation is the reliance on the existence of credible aggregate priors. The approach calibrates a simulation against the kinds of aggregate patterns the literature reports, and the benchmark’s targets were written to resemble the form of those patterns rather than transcribed from any one cohort. Applying the framework to a real population would require sourcing actual published statistics and judging their relevance, which is itself a substantive and domain-dependent step.

A third limitation is the use of LLMs as judges. The same class of model that generates the logs also scores target labels and persona identity, so the reported numbers

are benchmark scores under a specified judging procedure rather than direct measurements of classroom behavior. The caution is concrete rather than abstract, since the belief-to-behavior study in Chapter 5 found that models in this line do not reliably enact persona-linked behavior, which is reason to be careful about asking them to score it. Human validation of a subset of judged features and persona identifications remains necessary.

Three further limitations are larger than any single study and bound what the dissertation can claim as a whole. The first is that the work never makes contact with real learner data: the targets follow the form of published findings rather than any actual cohort, the personas are invented, and, as above, the judges are themselves models. The dissertation therefore shows that a simulated population can be steered and kept internally consistent, not that it reproduces how real students behave. The second is that satisfying an aggregate target is only a proxy for the goal that motivated the work, synthetic data that can substitute for real data; the dissertation does not demonstrate that a calibrated corpus serves a downstream use, such as training a usable model or reproducing a known empirical finding, and matching an aggregate can discard the individual and causal structure that often makes such data valuable. The third is that the authoring bottleneck the work set out to relieve is reduced rather than removed: experts must still write and validate the targets, features, and scoring rules, so the contribution reorganizes that effort and makes its cost explicit rather than eliminating it.

7.6 Generalization and Transfer

The framework’s components differ in how readily they transfer. Separating the parts that carry over from the parts a new domain must rebuild clarifies what applying it elsewhere would take.

What transfers is the recipe and the reasoning around it. The workflow, in which the desired aggregate is expressed as an explicit target, observable features are identified, the target-and-judge contract is validated, a batch is generated, and aggregate fit is reported

together with agent fidelity and generation cost, is domain-independent. The same holds for several findings that serve as design guidance: that target-satisfaction, persona fidelity, and cost must be reported together; that targets should be validated before methods are compared; and that test-time computation should be allocated where the baseline lacks the information the target requires. The locality lens, which predicts difficulty by asking where a target’s behavior must hold, is also general, since the distinction between within-trajectory and across-trajectory requirements is not specific to pair programming.

What must be recreated is the domain-specific content the recipe operates on. New domains need their own target families and individual targets, their own observable features and extractors, and their own scoring rules, and writing these well requires expertise in the domain being simulated. The choice of judge, the population of personas, and the underlying task set are likewise domain-specific. This is the familiar concern with highly structured representations, that they work within a domain but are laborious to move to another, and this work does not escape it, but organizes that work and makes its cost explicit rather than eliminating it.

7.7 Scope and Threads Not Covered

The work touched four domains, and each was used to study one facet of the calibration problem rather than the whole of it. Task-oriented ride-booking dialogue (Chapter 3) exposed the cost of authoring behavior by hand. A physics learning environment (Chapter 4) tested whether behavioral specifications compose and can be reused as a learner model changes. A repeated trust game (Chapter 5) asked whether a model’s stated beliefs about a population match the behavior it then produces. Middle-school pair programming (Chapter 6) was where the calibration method and the locality account were developed and measured. The problem sits where several research threads meet, among them agent-based modeling, planning with LLMs, calibration, and LLM roleplaying, and each chapter enters that intersection from one side, with only the pair-programming study crossing it from end to end.

Several threads about how the simulation was run were left uncovered. The generation studied here is offline, in that a batch is planned and written before any of it is scored, so the generator never reacts to a session while it is still unfolding. An orchestrator that watches a session in progress and steers it back toward the target when the behavior drifts was considered but not built, and closed-loop, in-session calibration is therefore outside what these results speak to. The unit of analysis throughout is a single session on a single task, so learning that accumulates across many sessions over weeks, and targets defined over that longer horizon, are not modeled.

Other threads concern what the simulation was run on. The learners are invented middle-school students described along a small set of traits, and the sessions are in English, so other ages, languages, cultural settings, and richer learner models remain untouched. Pair programming is one collaborative structure among many, and settings that differ in kind, such as medical training, collaborative writing, open-ended tutoring, or groups larger than a pair, were not attempted. The pairs studied are two students working together, so whether the same findings hold when one partner is a language model rather than a fellow student—an increasingly common form of the activity as coding assistants enter classrooms—is a separate question this dissertation does not settle. The targets themselves capture the aggregate statistical patterns that studies of collaboration usually report, such as rates, distributions, contrasts, and orderings, rather than the causal structure behind those patterns or the learning outcomes a real classroom is meant to produce.

Because these threads are untested, the claims here are bounded to the domains, target families, populations, and interaction mode that were actually studied. The method, the locality lens, and the reported numbers are shown to hold for calibrating offline batches of middle-school pair-programming sessions against aggregate targets of the form the literature reports, and they are not evidence that the same method, lens, or numbers hold in a domain, population, or interaction mode that was not examined. Carrying the approach beyond that scope would mean rebuilding and re-validating the domain-specific

parts, namely the targets, the observable features, the judges, and the persona pool, and then checking the outcome against that domain’s own evidence rather than against the benchmark’s internal scores. Section 7.6 sets out which parts are expected to carry over and which must be remade.

Several practical questions about transfer can be answered in part on this basis. Constructing target families in a new domain currently requires meaningful manual effort and domain expertise, comparable to authoring a small specification language for the behaviors of interest. The construction is a candidate for partial automation, in which targets are drafted from the literature or induced from data, observable features are proposed by a model, and the resulting candidates are filtered by the same on-target versus off-target validation the benchmark already uses, with expert review kept for the parts that matter most. The framework is most reliable where behavioral priors are well described and widely agreed upon, and correspondingly weaker where priors are noisy, disputed, or incomplete, since both the targets and the judging contract inherit that uncertainty. Carrying the approach to settings such as medical training or collaborative writing would reuse the workflow and the reporting discipline, but would require new target families, new features and scoring rules, a domain-appropriate judge, and validation against that domain’s own evidence. The method is general; its instantiation in any particular domain remains specific and effortful.

7.8 Future Directions

Several directions stand out: cross-domain evaluation, semi-automatic target generation, human validation studies, alternative judging methodologies, richer persona modeling, and more scalable mechanisms for specifying and allocating behaviors. The most direct next step is to test transfer, by instantiating the workflow in a second domain and measuring how much of the target specification, feature extraction, and scoring can be reused. A closely related direction is to reduce the cost of specification through semi-automatic target generation, in which targets and features are drafted from published

findings or induced from data and then screened by the validation protocol, so that manual effort concentrates on review rather than authoring. Human validation studies would connect benchmark scores to human judgments of the same features and persona identifications, and alternative judging methods, including model-separated generators and judges and non-model scoring where features permit, would address the dependence on a single class of model for both generation and evaluation. Richer persona modeling would let the simulated learners carry more of the variation seen in real classrooms, and more scalable mechanisms for specifying and allocating behaviors, such as a sequential planner that revises its assignments after observing generated sessions, would address the delayed feedback that makes non-local targets difficult.

7.9 Closing

This work set out to calibrate LLM-based educational simulations so that their aggregate behavior matches what researchers intend. A capable generator turns out to be only part of what that requires. Calibration depends as well on explicit behavioral targets that state what the aggregate should be, on validation that checks whether those targets and their judges can be trusted, and on spending generation-time computation where a per-session generator would otherwise be unable to meet the target. Locality runs through these requirements, since it separates the targets a single session can already satisfy from those that need a view of the whole batch, and it is what shows where additional computation will help. Reliable calibration of an educational simulation thus rests as much on explicit behavioral targets, on validation that those targets and their judges can be trusted, and on matching generation-time computation to the structure of the targets, as on the capability of the underlying generative model.

APPENDIX A PERSONA POOL

DUET-Bench draws the driver and navigator roles of every generated session from a single fixed pool of 28 learner profiles, listed in full below. The same pool is shared across all generation methods and target categories, so method comparisons never change the population being simulated. Each profile gives the model role-relevant background such as grade level, confidence, collaboration style, and out-of-class detail; the summaries are reproduced verbatim from the pool specification.

Table A-1. The 28-learner persona pool used in DUET-Bench. Driver and navigator roles are drawn from this fixed pool for every generated session, so all methods simulate the same population. Profiles are reproduced from the pool specification.

Name	Role	Grade	Profile summary
Caleb	Driver	6	Extremely lost driver who doesn't follow what's happening. Confuses variables and values, asks questions that reveal foundational gaps ('what's the difference between print and return again?'), sometimes types things at random hoping they'll work. Voice is genuinely bewildered, not faking it. Outside of code, regularly misses small instructions in other classes and seems perpetually surprised by deadlines.
Leah	Driver	6	Uncertain student who wants frequent confirmation before taking action. Leans heavily on the partner for guidance and validation, speaks in a hesitant, question-heavy voice, and rarely commits to a plan without prompting. Outside of code, draws tiny mushrooms in the margins of every notebook and re-watches the same one show for comfort.
Madison	Driver	6	Chronic complainer who narrates frustration constantly: the assignment is dumb, the laptop is slow, the partner is annoying, the test cases are unfair, everything is too much work. Will eventually do the work but only after extensive complaining and dramatic sighing. Outside of code, has strong opinions on everything that's wrong with the school cafeteria, the dress code, and the assignment scheduling.
Mia	Driver	6	Careful student who prefers to talk through edge cases before changing code. Asks for reassurance before committing to edits and hedges with phrases like 'i think' and 'does that sound right'. Outside of code, keeps a color-coded planner and gets mildly stressed about forgetting to update it; bakes only recipes with exact ratios.
Alex	Driver	7	Middle-school student learning to code, with no distinctive collaboration style.

continued on next page

Name	Role	Grade	Profile summary
Ava	Driver	7	Structured student who likes to break the task into orderly steps before coding. Talks through the plan first, then edits deliberately, in an organized and calm voice. Can sound rigid and may over-plan simple fixes. Outside of code, plans the week on Sunday nights with a specific colored-pen system and is perpetually behind on one Warhammer miniature.
Brandon	Driver	7	Selfish driver who treats the partner as a spectator. Makes edits without explaining, talks over the navigator's questions, sighs audibly when interrupted, and rolls past 'wait, can you explain?' with one-word answers. Outside of code, is the kid who always wants to pick the movie at sleepovers and never returns borrowed earbuds.
Ethan	Driver	7	Confident student who likes to move quickly from idea to code. Acts decisively, sometimes edits before the pair is fully aligned, and speaks in a direct, energetic register. Outside of code, follows the school basketball team obsessively and is always pushing whatever new sports-drink flavor just came out.
Jordan	Driver	7	Steady student who mixes planning with implementation without dominating the interaction. Usually checks in before editing, listens to feedback, and keeps a clear and even-handed voice. Outside of code, plays on the school soccer team and tends to be the one organizing group-chat logistics for everyone else.
Kai	Driver	7	Student who moves into code quickly and often explains too little before editing. Pushes for fast edits and narrows discussion to whatever seems easiest to try first; speech is quick and clipped. Outside of code, is on the third new hobby this month (currently penny skateboards), behind on three group chats, and constantly hungry.
Nina	Driver	7	Student who likes to anchor the discussion on edge cases before editing. Keeps returning to specific examples, asks what the code would do before changing it, and speaks in a specific, test-oriented register. Outside of code, collects vintage soda cans and surfaces oddly specific hypotheticals like 'what if the bus had left two minutes early'.
Tyler	Driver	7	Combative student who treats every code review like a personal attack. Argues with the navigator's suggestions even when they're obviously right, deflects blame onto the test cases or the assignment, and gets defensive when critiqued. Tone is sharp and prickly. Outside of code, has been kicked out of two group projects this year and complains constantly about other people's work ethic.

continued on next page

Name	Role	Grade	Profile summary
Mason	Driver	8	Student who tends to rewrite too much when a smaller local fix would do. Suggests larger rewrites when a bug feels annoying or unclear, with a confident, solution-forward voice. Outside of code, has rearranged the same bedroom three times this semester and ignores Lego instructions on principle.
Noah	Driver	8	Confident student who sticks with an idea even when challenged. Pushes forward strongly, does not always invite partner input, and tends to keep arguing for the original plan in an assertive, insistent voice. Outside of code, has been arguing for weeks about which song should open the band concert and ranks every pizza place in town on an actual spreadsheet.
Brooke	Navigator	6	Disruptive navigator who can't keep a session on track for more than a few turns. Pivots conversation to non-code topics constantly, makes inside jokes, dares the driver to type silly things, laughs at her own asides. Outside of code, regularly gets in trouble in study hall and has been moved to a different seat in two classes this year.
Jaden	Navigator	6	Cluelessly enthusiastic navigator who agrees with almost everything the driver says regardless of whether it makes sense. Reads code out loud without understanding it, suggests random changes when pressed for an opinion, and stays cheerfully confident even when wrong. Outside of code, has signed up for clubs he forgot he joined and turns in homework without reading the questions carefully.
Leo	Navigator	6	Student who wants to help but often loses the thread of the debugging plan. Asks clarifying questions that derail focus, low-confidence and tentative throughout. Outside of code, owns three half-learned instruments and gets visibly anxious whenever a fish in the tank at home looks off.
Sofia	Navigator	6	Encouraging student who tries to keep the pair calm and moving. Offers reassurance and soft feedback, speaks in a warm, collaborative voice, and hesitates to push back even when the driver's idea is off. Outside of code, remembers everyone's birthdays unprompted and is genuinely anxious about a houseplant named Frederick.
Daniel	Navigator	7	Even-tempered student who can support, question, or critique as needed. Responds consistently without dominating; voice is measured and practical. Outside of code, reads airport non-fiction despite never having flown anywhere, and is weirdly precious about one nice pen.

continued on next page

Name	Role	Grade	Profile summary
Grace	Navigator	7	Fast-moving student who wants the pair to stop talking and try something. Pushes for action, gets visibly frustrated by extended discussion, and speaks in a brisk, clipped voice. Outside of code, is mid-cross-country season and won't admit to being tired; loyal to whatever drink the school store is pushing this week.
Olivia	Navigator	7	Disengaged navigator who scrolls through her phone most of the session; only engages when the driver explicitly asks for help or when the silence has gone on too long. Responses are short, half-attentive, and sometimes out of sync with what the driver actually said. Outside of code, lives on whatever app her friend group is using this month and has a Snapchat streak with eight different people that she's anxious about breaking.
Owen	Navigator	7	Student who often proposes multiple explanations or repair paths. Keeps surfacing alternatives and asking which explanation fits the evidence best; voice is curious and idea-rich. Can overcomplicate simple bugs and may delay commitment. Outside of code, has too many half-finished projects (a kite, a homemade lamp, a model rocket) and keeps pitching weekend plans that nobody actually acts on.
Riley	Navigator	7	Student who keeps asking for evidence before agreeing to a fix. Pushes the pair to cite examples, outputs, or code facts before moving on; voice is calm and insistent. Can slow momentum and may sound skeptical. Outside of code, fact-checks group-chat rumors compulsively and is teaching themselves astronomy from a library book.
Sam	Navigator	7	Middle-school student learning to code, with no distinctive collaboration style.
Hannah	Navigator	8	Student who often jumps in with the answer instead of sustaining joint reasoning. Steps in quickly, redirects the session before the driver finishes reasoning, and speaks in a fast, corrective voice. Outside of code, stage-manages the school play with a clipboard and silently rearranges lunch-table seating.
Isabella	Navigator	8	Reflective student who tries to guide the partner toward the next step without taking over. Scaffolds reasoning and nudges the driver to explain in a gentle but purposeful voice; can over-explain or be too indirect. Outside of code, helps younger kids with math at the local library on Saturdays and tends to go on side-rants about thrifted sweaters.
Marcus	Navigator	8	Sarcastic and dismissive critic who uses cruel humor at the partner's expense. Critiques are often technically right but delivered with a sneer; rolls eyes audibly when the driver makes mistakes, makes small jabs about the driver's intelligence. Outside of code, has a reputation for cutting comments in the group chat and a long-running grudge against most of the soccer team.

continued on next page

Name	Role	Grade	Profile summary
Priya	Navigator	8	Sharp student who critiques aggressively when the plan feels weak. Pushes for stronger reasoning and clearer justification in a blunt, demanding voice; can sound harsh and create tension. Outside of code, is on the middle-school debate team and is still mad about a recent tournament loss; has uncompromising opinions about which boba place in town is best.

LIST OF REFERENCES

- [1] *Spearman rank correlation coefficient*, The Concise Encyclopedia of Statistics (Yadolah Dodge, ed.), Springer New York, New York, NY, 2008, pp. 502–505.
- [2] William Agnew, A Stevie Bergman, Jennifer Chien, Mark Díaz, Seliem El-Sayed, Jaylen Pittman, Shakir Mohamed, and Kevin R McKee, *The Illusion of Artificial Inclusion*, Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, 2024, pp. 1–12.
- [3] Gati Aher, Rosa I. Arriaga, and Adam Tauman Kalai, *Using Large Language Models to Simulate Multiple Humans and Replicate Human Subject Studies*, July 2023, arXiv:2208.10264 [cs].
- [4] Gati V Aher, Rosa I Arriaga, and Adam Tauman Kalai, *Using Large Language Models to Simulate Multiple Humans and Replicate Human Subject Studies*, International Conference on Machine Learning, PMLR, 2023, pp. 337–371.
- [5] Vincent Aleven, Bruce M McLaren, Jonathan Sewall, Keith Brawner, Philip Pavlik, and Kenneth R Koedinger, *Emerging Trends in Automated Authoring*, Design Recommendations for Intelligent Tutoring Systems: Volume 3-Authoring Tools and Expert Modeling Techniques, 2015, p. 249.
- [6] Peter Anderson, *Assessment and Development of Executive Function (Ef) during Childhood*, Child neuropsychology **8** (2002), no. 2, 71–82.
- [7] Jacy Reese Anthis, Rui Liu, Sean M Richardson, Austin C Kozlowski, and James A Evans, *Position: Llm Social Simulations Are a Promising Research Method*, International Conference on Machine Learning (2024).
- [8] Mikhail L. Arbuzov, Alexander A. Shvets, and Sebastian Beir, *Beyond Exponential Decay: Rethinking Error Accumulation in Large Language Models*, arXiv preprint arXiv:2505.24187 (2025).
- [9] Lisa P Argyle, Ethan C Busby, Nancy Fulda, Joshua R Gubler, Christopher Rytting, and David Wingate, *Out of One, Many: Using Language Models to Simulate Human Samples*, Political Analysis **31** (2023), no. 3, 337–351.
- [10] Robert Axelrod, *The Complexity of Cooperation: Agent-Based Models of Competition and Collaboration*, Princeton University Press, 1997.
- [11] Axelrod, Robert, *Agent-based modeling as a bridge between disciplines*, Handbook of Computational Economics **2** (2006), 1565–1584.
- [12] Yuhao Bai, Tianyu Huang, Kai Sun, and Yu Chen, *Scaling Law in llm Simulated Personality: More Detailed and Realistic Persona Profile Is All You Need*, arXiv preprint arXiv:2510.11734 (2025).

- [13] Jorge A Baier, Fahiem Bacchus, and Sheila A McIlraith, *A Heuristic Search Approach to Planning with Temporally Extended Preferences*, Artificial Intelligence **173** (2009), no. 5-6, 593–618.
- [14] Osman Balci, *Verification, Validation, and Testing*, Handbook of Simulation (Jerry Banks, ed.), John Wiley & Sons, 1998, pp. 335–393.
- [15] Krisztian Balog and ChengXiang Zhai, *User Simulation in the Era of Generative ai: User Modeling, Synthetic Data Generation, and System Evaluation*, arXiv preprint arXiv:2501.04410 (2025).
- [16] Jianzhu Bao, Rui Wang, Yasheng Wang, Aixin Sun, Yitong Li, Fei Mi, and Ruifeng Xu, *A Synthetic Data Generation Framework for Grounded Dialogues*, Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (Toronto, Canada) (Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, eds.), Association for Computational Linguistics, July 2023, pp. 10866–10882.
- [17] Bernadetta Bartosik, Grzegorz M Wojcik, Aneta Brzezicka, and Andrzej Kawiak, *Are You Able to Trust Me? Analysis of the Relationships between Personality Traits and the Assessment of Attractiveness and Trust*, Frontiers in Human Neuroscience **15** (2021), 685530.
- [18] Antonio Bella, Cèsar Ferri, José Hernández-Orallo, and María José Ramírez-Quintana, *Calibration of Machine Learning Models*, Handbook of Research on Machine Learning Applications and Trends: Algorithms, Methods, and Techniques, IGI Global, 2010, pp. 128–146.
- [19] Joyce Berg, John Dickhaut, and Kevin McCabe, *Trust, Reciprocity, and Social History*, Games and economic behavior **10** (1995), no. 1, 122–142.
- [20] Pranav Bhandari, Nicolas Fay, Michael Wise, Amitava Datta, Stephanie Meek, Usman Naseem, and Mehwish Nasim, *Can llm Agents Maintain a Persona in Discourse?*, ArXiv **abs/2502.11843** (2025).
- [21] Carlo Bianchi, Pasquale Cirillo, Mauro Gallegati, and Pietro A Vagliasindi, *Validating and Calibrating Agent-Based Models: a Case Study*, Computational Economics **30** (2007), no. 3, 245–264.
- [22] Stephen B Blessing and Stephen Gilbert, *Evaluating an Authoring Tool for Model-Tracing Intelligent Tutoring Systems*, International Conference on Intelligent Tutoring Systems, Springer, 2008, pp. 204–215.
- [23] Jessica Blom-Hoffman, Stephen S Leff, Debra L Franko, Elana Weinstein, Kelly Beakley, and Thomas J Power, *Consent Procedures and Participation Rates in School-Based Intervention and Prevention Research: Using a Multi-Component, Partnership-Based Approach to Recruit Participants*, School mental health **1** (2009), no. 1, 3–15.

- [24] Benjamin S. Bloom, *The 2 Sigma Problem: the Search for Methods of Group Instruction as Effective as One-to-One Tutoring*, Educational Researcher **13** (1984), no. 6, 4–16.
- [25] Iris Bohnet, Fiona Greig, Benedikt Herrmann, and Richard Zeckhauser, *Betrayal Aversion: Evidence from Brazil, China, Oman, Switzerland, Turkey, and the United States*, American Economic Review **98** (2008), no. 1, 294–310.
- [26] Dan Bohus and Alexander I. Rudnicky, *The ravenclaw dialog management framework: Architecture and systems*, Computer Speech & Language **23** (2009), no. 3, 332–361 (en).
- [27] Eric Bonabeau, *Agent-Based Modeling: Methods and Techniques for Simulating Human Systems*, Proceedings of the National Academy of Sciences **99** (2002), no. suppl 3, 7280–7287.
- [28] James Brand, Ayelet Israeli, and Donald Ngwe, *Using GPT for market research*, Harvard Business School Marketing Unit Working Paper (2023), no. 23-062.
- [29] John Brooke, *Sus - a quick and dirty usability scale*, Usability evaluation in industry **189** (1996), no. 3, 8 (en).
- [30] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini, *Large language monkeys: Scaling inference compute with repeated sampling*, arXiv:2407.21787, 2024.
- [31] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei, *Language Models are Few-Shot Learners*, arXiv:2005.14165 [cs] (2020), arXiv: 2005.14165.
- [32] Nancy R. Buchan, Rachel T. A. Croson, and Robyn M. Dawes, *Swift Neighbors and Persistent Strangers: A Cross-Cultural Investigation of Trust and Reciprocity in Social Exchange*, American Journal of Sociology **108** (2002), no. 1, 168–206.
- [33] Antonio Cabrales and Ismael Sanz, *Non-Cognitive Skills*, Economics of Education: An Introductory Textbook (2025), 442.
- [34] Julia Cambre and Chinmay Kulkarni, *Methods and Tools for Prototyping Voice Interfaces*, Proceedings of the 2nd Conference on Conversational User Interfaces (New York, NY, USA), CUI '20, Association for Computing Machinery, 2020.

- [35] Giovanni Campagna, Agata Foryciarz, Mehrad Moradshahi, and Monica Lam, *Zero-Shot Transfer Learning with Synthesized Data for Multi-Domain Dialogue State Tracking*, Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (Online), Association for Computational Linguistics, July 2020, pp. 122–132.
- [36] Peng Cao, Tong Men, Wenjie Liu, Jie Zhang, Xinyi Li, Xudong Lin, Dianbo Sui, et al., *Large Language Models for Planning: a Comprehensive and Systematic Survey*, arXiv preprint arXiv:2505.19683 (2025).
- [37] Mehmet Celepkolu and Kristy Elizabeth Boyer, *The Importance of Producing Shared Code through Pair Programming*, Proceedings of the 49th ACM technical symposium on computer science education, 2018, pp. 765–770.
- [38] Şehnaz Cenani, *Emergence and Complexity in Agent-Based Modeling: Review of State-of-the-Art Research*, Journal of Computational Design **2** (2021), no. 2, 1–24.
- [39] Lu Chen, Boer Lv, Chi Wang, Su Zhu, Bowen Tan, and Kai Yu, *Schema-Guided Multi-Domain Dialogue State Tracking with Graph Attention Neural Networks*, Proceedings of the AAAI Conference on Artificial Intelligence **34** (2020), no. 05, 7521–7528 (en).
- [40] Ayush Chopra, Shashank Kumar, Nurullah Giray-Kuru, Ramesh Raskar, and Arnau Quera-Bofarull, *On the Limits of Agency in Agent-Based Models*, arXiv preprint arXiv:2409.10568 (2024).
- [41] Sankalan Pal Chowdhury, Vilém Zouhar, and Mrinmaya Sachan, *Scaling the Authoring of autotutors with Large Language Models*, February 2024, arXiv:2402.09216 [cs] version: 1.
- [42] Rhonda Christensen, Gerald Knezek, Tandra Tyler-Wood, and David Gibson, *Simschool: an Online Dynamic Simulator for Enhancing Teacher Preparation*, International Journal of Learning Technology **6** (2011), no. 2, 201–220.
- [43] Herbert H. Clark and Gregory L. Murphy, *Audience Design in Meaning and Reference*, Advances in Psychology (Jean-François Le Ny and Walter Kintsch, eds.), Language and Comprehension, vol. 9, North-Holland, January 1982, pp. 287–299 (en).
- [44] Samuel Colvin, Eric Jolibois, Hasan Ramezani, Adrian Garcia Badaracco, Terrence Dorsey, David Montague, Serge Matveenko, Marcelo Trylesinski, Sydney Runkle, David Hewitt, Alex Hall, and Victorien Plot, *Pydantic*, June 2025.
- [45] Rosaria Conte and Mario Paolucci, *On Agent-Based Modeling and Computational Social Science*, Frontiers in psychology **5** (2014), 668.
- [46] John Conway, *Conway’s Game of Life*, Scientific American **223** (1970), no. 4, 120–123.

- [47] Stacey Crane and Marion E Broome, *Understanding Ethical Issues of Research Participation from the Perspective of Participating Children and Adolescents: a Systematic Review*, *Worldviews on evidence-based nursing* **14** (2017), no. 3, 200–209.
- [48] Murtaza Dalal, Tarun Chiruvolu, Devendra Chaplot, and Ruslan Salakhutdinov, *Plan-Seq-Learn: Language Model Guided rl for Solving Long Horizon Robotics Tasks*, May 2024, arXiv:2405.01534 [cs].
- [49] Ishita Dasgupta, Andrew K Lampinen, Stephanie CY Chan, Antonia Creswell, Dharshan Kumaran, James L McClelland, and Felix Hill, *Language Models Show Human-Like Content Effects on Reasoning*, arXiv preprint arXiv:2207.07051 (2022).
- [50] Colin G DeYoung, Lena C Quilty, and Jordan B Peterson, *Between Facets and Domains: 10 Aspects of the Big Five.*, *Journal of personality and social psychology* **93** (2007), no. 5, 880.
- [51] Zhichen Dong, Zhanhui Zhou, Zhixuan Liu, Chao Yang, and Chaochao Lu, *Emergent Response Planning in llms*, arXiv preprint arXiv:2502.06258 (2025).
- [52] Stephen H. Edwards and Manuel A. Pérez-Quñones, *Using a Pedagogical Agent to Support Students Learning to Program*, *Proceedings of the 2008 ASEE Annual Conference and Exposition (Pittsburgh, PA)*, American Society for Engineering Education, 2008.
- [53] Joshua M. Epstein, *Generative Social Science: Studies in Agent-Based Computational Modeling*, Princeton University Press, 2006, Paperback edition published 2007.
- [54] Anthony M Evans and William Revelle, *Survey and Behavioral Measurements of Interpersonal Trust*, *Journal of research in Personality* **42** (2008), no. 6, 1585–1593.
- [55] Jennifer Fereday and Eimear Muir-Cochrane, *Demonstrating Rigor Using Thematic Analysis: A Hybrid Approach of Inductive and Deductive Coding and Theme Development*, *International Journal of Qualitative Methods* **5** (2006), no. 1, 80–92.
- [56] Stanford Institute for Human-Centered Artificial Intelligence, *Simulating Human Behavior with ai Agents*, HAI Policy Brief (2025).
- [57] Ivar Frisch and Mario Giulianelli, *Llm Agents in Interaction: Measuring Personality Consistency and Linguistic Alignment in Interacting Populations of Large Language Models*, February 2024, arXiv:2402.02896 [cs] version: 1.
- [58] Yao Fu, Hao Peng, Tushar Khot, and Mirella Lapata, *Improving Language Model Negotiation with Self-Play and in-Context Learning from ai Feedback*, arXiv preprint arXiv:2305.10142 (2023).
- [59] Chen Gao, Xiaochong Lan, Nian Li, Yuan Yuan, Jingtao Ding, Zhilun Zhou, Fengli Xu, and Yong Li, *Large language models empowered agent-based modeling and simulation: A survey and perspectives*, *Humanities and Social Sciences Communications* **11** (2024), no. 1, 1–24.

- [60] Konstantin Genin, *On Falsifiable Statistical Hypotheses*, *Philosophies* **7** (2022), no. 2, 40.
- [61] Alfonso Gerevini and Derek Long, *Plan constraints and preferences in PDDL3*, ICAPS Workshop, 2005.
- [62] Ellen R Girden, *Anova: Repeated Measures*, no. 84, Sage, 1992.
- [63] Edward L. Glaeser, David I. Laibson, Jose A. Scheinkman, and Christine L. Soutter, *Measuring Trust*, *The Quarterly Journal of Economics* **115** (2000), no. 3, 811–846.
- [64] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al., *The Llama 3 Herd of Models*, arXiv preprint arXiv:2407.21783 (2024).
- [65] Matteo Guerini and Alessio Moneta, *A Method for Agent-Based Models Validation*, *Journal of Economic Dynamics and Control* **82** (2017), 125–141.
- [66] George Gui and Olivier Toubia, *The Challenge of Using llms to Simulate Human Behavior: A Causal Inference Perspective*, ArXiv **abs/2312.15524** (2023).
- [67] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al., *Deepseek-r1: Incentivizing Reasoning Capability in llms via Reinforcement Learning*, arXiv preprint arXiv:2501.12948 (2025).
- [68] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang, *Large Language Model Based Multi-Agents: a Survey of Progress and Challenges*, arXiv preprint arXiv:2402.01680 (2024).
- [69] Naresh Gupta and Dana S Nau, *On the Complexity of Blocks-World Planning*, *Artificial intelligence* **56** (1992), no. 2-3, 223–254.
- [70] Michael Hannafin, Susan Land, and Kevin Oliver, *Open Learning Environments: Foundations, Methods, and Models*, *Instructional-Design Theories and Models*, Routledge, 2013, pp. 115–140.
- [71] Michael J Hannafin, Janette R Hill, Susan M Land, and Eunbae Lee, *Student-Centered, Open Learning Environments: Research, Theory, and Practice*, *Handbook of research on educational communications and technology* (2014), 641–651.
- [72] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu, *Reasoning with Language Model Is Planning with World Model*, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 8154–8173.

- [73] Brent Harger and Melissa Quintela, *The irb as Gatekeeper: Effects on Research with Children and Youth*, *Researching Children and Youth: Methodological Issues, Strategies, and Innovations*, vol. 22, Emerald Publishing Limited, 2017, pp. 11–33.
- [74] Jane Hewitt et al., *Social Science Researchers Use Ai to Simulate Human Subjects*, *Stanford News* (2025), Early demonstration of LLMs simulating human survey responses with promising validity.
- [75] Margaret L Hilton and James W Pellegrino, *Education for Life and Work: Developing Transferable Knowledge and Skills in the 21st Century*, National Academies Press, 2012.
- [76] Kenneth Holstein, Bruce M McLaren, and Vincent Aleven, *Co-Designing a Real-Time Classroom Orchestration Tool to Support Teacher-Ai Complementarity.*, Grantee Submission (2019).
- [77] Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick, *Unnatural instructions: Tuning language models with (almost) no human labor*, *ACL*, 2023.
- [78] John J Horton, *Large Language Models as Crowdsourced Research Assistants: A Case Study in Experimental Economics*, *Experimental Economics* **26** (2023), no. 3, 794–820.
- [79] Abe Bohan Hou, Hongru Du, Yichen Wang, Jingyu Zhang, Zixiao Wang, Paul Pu Liang, Daniel Khashabi, Lauren Gardner, and Tianxing He, *Can a Society of Generative Agents Simulate Human Behavior and Inform Public Health Policy? a Case Study on Vaccine Hesitancy*, arXiv preprint arXiv:2503.09639 (2025).
- [80] Jie Huang and Kevin Chen-Chuan Chang, *Towards Reasoning in Large Language Models: a Survey*, arXiv preprint arXiv:2212.10403 (2022).
- [81] Yue Huang, Zhengqing Yuan, Yujun Zhou, Kehan Guo, Xiangqi Wang, Haomin Zhuang, Weixiang Sun, Lichao Sun, Jindong Wang, Yanfang Ye, et al., *Social Science Meets llms: How Reliable Are Large Language Models in Social Simulations?*, arXiv preprint arXiv:2410.23426 (2024).
- [82] Connor Huff and Dustin Tingley, “Who Are These People?” *Evaluating the Demographic Characteristics and Political Preferences of Mturk Survey Respondents*, *Research & Politics* **2** (2015), no. 3, 2053168015604648.
- [83] Simone Jablonski and Matthias Ludwig, *Teaching and Learning of Geometry—a Literature Review on Current Developments in Theory and Practice*, *Education Sciences* **13** (2023), no. 7.
- [84] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al., *Openai o1 System Card*, arXiv preprint arXiv:2412.16720 (2024).

- [85] Hang Jiang, Xiajie Zhang, Xubo Cao, Cynthia Breazeal, Deb Roy, and Jad Kabbara, *PersonaLLM: Investigating the ability of large language models to express personality traits*, Findings of the Association for Computational Linguistics: NAACL 2024 (Mexico City, Mexico) (Kevin Duh, Helena Gomez, and Steven Bethard, eds.), Association for Computational Linguistics, June 2024, pp. 3605–3627.
- [86] Subbarao Kambhampati, *Can Large Language Models Reason and Plan?*, Annals of the New York Academy of Sciences **1534** (2024), no. 1, 15–18, arXiv:2403.04121 [cs].
- [87] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Kaya Stechly, Mudit Verma, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy, *Llms Can’t Plan, but Can Help Planning in llm-Modulo Frameworks*, February 2024, arXiv:2402.01817 [cs].
- [88] Tanja Käser, Ivo Benke, Alexander Maedche, and Kenneth Koedinger, *Evaluating the Impact of Learner Control and Interactivity in Conversational Tutoring Systems for Persuasive Writing*, International Journal of Artificial Intelligence in Education (2025), 1–34.
- [89] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal, *Decomposed Prompting: a Modular Approach for Solving Complex Tasks*, Arxiv Preprint Arxiv:2210.02406, 2022.
- [90] ChanMin Kim, *The Role of Affective and Motivational Factors in Designing Personalized Learning Environments*, Educational Technology Research and Development **60** (2012), 563–584.
- [91] Mi Kyung Kim and Tuba Ketenci, *The Role of Expressed Emotions in Online Discussions*, Journal of Research on Technology in Education **52** (2020), no. 3, 313–334.
- [92] Sungdong Kim, Minsuk Yang, Gangwoo Lee, Sang-Woo Lee, Jung-Woo Park, and Nako Kim, *NeuralWOZ: Learning to collect task-oriented dialogue via model-based simulation*, ACL, 2021.
- [93] Young-Bum Kim, Dongchan Kim, Anjishnu Kumar, and Ruhi Sarikaya, *Efficient Large-Scale Neural Domain Classification with Personalized Attention*, Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (Melbourne, Australia), Association for Computational Linguistics, July 2018, pp. 2214–2224.
- [94] Orly Klein-Latucha and Arnon HersHKovitz, *When Leaving is Persisting: Studying Patterns of Persistence in an Online Game-Based Learning Environment for Mathematics*, Journal of Learning Analytics (2024), 1–10 (en).
- [95] Ekaterina Kochmar, Dung Do Vu, Robert Belfer, Varun Gupta, Iulian Vlad Serban, and Joelle Pineau, *Automated Data-Driven Generation of Personalized Pedagogical Interventions in Intelligent Tutoring Systems*, International Journal of Artificial Intelligence in Education **32** (2022), 323–349.

- [96] Kenneth R Koedinger, Noboru Matsuda, and Christopher J MacLellan, *Methods for Evaluating Simulated Learners: Examples from simstudent*, AIED Workshop on Simulated Learners (2015).
- [97] Kenneth R Koedinger, Noboru Matsuda, Christopher J MacLellan, and Elizabeth A McLaughlin, *Methods for Evaluating Simulated Learners: Examples from simstudent.*, AIED Workshops, 2015.
- [98] Austin C Kozlowski and James A Evans, *Simulating Subjects: the Promise and Peril of Ai Stand-Ins for Social Agents and Interactions*, (2024).
- [99] Andrei Krokhin and Stanislav Živný, *The Constraint Satisfaction Problem: Complexity and Approximability*, (2017).
- [100] Pavel Kucherbaev, *Quality Assurance Strategies in Microtask Crowdsourcing*, Ph.D. thesis, University of Trento, 2016.
- [101] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev, *In Search of Needles in a 11m Haystack: Recurrent Memory Finds What llms Miss*, February 2024, arXiv:2402.10790 [cs].
- [102] Tanja Käser and Giora Alexandron, *Simulated Learners in Educational Technology: A Systematic Literature Review and a Turing-like Test*, International Journal of Artificial Intelligence in Education (2023) (en).
- [103] Tiziano Labruna and Bernardo Magnini, *Evaluating task-oriented dialogue consistency through constraint satisfaction*, arXiv preprint arXiv:2407.11857 (2024).
- [104] Muhammad Laiq, *Large language models for behavioral modeling: A literature survey*, arXiv preprint arXiv:2509.24782 (2025).
- [105] Tom A Lamb, Adam Davies, Alasdair Paren, Philip HS Torr, and Francesco Pinto, *Focus on This, Not That! Steering llms with Adaptive Feature Specification*, arXiv preprint arXiv:2410.22944 (2024).
- [106] Susan Land and David Jonassen, *Theoretical Foundations of Learning Environments*, Routledge, 2012.
- [107] Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua, *Hello Again! llm-Powered Personalized Agent for Long-Term Dialogue*, arXiv preprint arXiv:2406.05925 (2024).
- [108] Junyi Li et al., *Controllable text generation: a survey of methods*, ACL Findings, 2022.
- [109] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg, *Inference-Time Intervention: Eliciting Truthful Answers from a Language Model*, Advances in Neural Information Processing Systems **36** (2023), 41451–41530.

- [110] Miao Li, Haoqi Xiong, and Yunbo Cao, *The spped System for Schema Guided Dialogue State Tracking Challenge*, arXiv:2006.09035 [cs] (2020) (en), arXiv: 2006.09035.
- [111] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe, *Let’s verify step by step*, International Conference on Learning Representations (ICLR), 2024.
- [112] Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren, *SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks*, NeurIPS, 2023.
- [113] Zhaojiang Lin, Bing Liu, Seungwhan Moon, Paul Crook, Zhenpeng Zhou, Zhiguang Wang, Zhou Yu, Andrea Madotto, Eunjoon Cho, and Rajen Subba, *Leveraging Slot Descriptions for Zero-Shot Cross-Domain Dialogue statetracking*, Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Online), Association for Computational Linguistics, 2021, pp. 5640–5648 (en).
- [114] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen, *What Makes Good in-Context Examples for gpt-3\$?*, January 2021, arXiv:2101.06804 [cs].
- [115] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig, *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing*, arXiv:2107.13586 [cs] (2021) (en), arXiv: 2107.13586.
- [116] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu, *G-Eval: NLG evaluation using GPT-4 with better human alignment*, EMNLP, 2023.
- [117] Ryan Louie, Ananjan Nandi, William Fang, Cheng Chang, Emma Brunskill, and Diyi Yang, *Roleplay-Doh: Enabling Domain-Experts to Create Llm-Simulated Patients via Eliciting and Adhering to Principles*, arXiv preprint arXiv:2407.00870 (2024).
- [118] Manikanta Loya, Divya Anand Sinha, and Richard Futrell, *Exploring the Sensitivity of llms’ Decision-Making Capabilities: Insights from Prompt Variation and Hyperparameters*, arXiv preprint arXiv:2312.17476 (2023).
- [119] Xinyi Lu and Xu Wang, *Generative Students: Using llm-Simulated Student Profiles to Support Question Item Evaluation*, May 2024, arXiv:2405.11591 [cs].
- [120] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp, *Fantastically Ordered Prompts and Where to Find Them: Overcoming few-shot prompt order sensitivity*, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long papers) (Dublin, Ireland) (Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, eds.), Association for Computational Linguistics, May 2022, pp. 8086–8098.

- [121] Yingbo Ma, Mehmet Celepkolu, and Kristy Elizabeth Boyer, *Detecting Impasse during Collaborative Problem Solving with Multimodal Learning Analytics*, LAK22: 12th International Learning Analytics and Knowledge Conference, 2022, pp. 45–55.
- [122] Amogh Mannekote, *Towards Compositionally Generalizable Semantic Parsing in Large Language Models: A Survey*, arXiv preprint arXiv:2404.13074 (2024).
- [123] Amogh Mannekote, Adam Davies, Jina Kang, and Kristy Elizabeth Boyer, *Can Llms Reliably Simulate Human Learner Actions? a Simulation Authoring Framework for Open-Ended Learning Environments*, Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, 2025, pp. 29044–29052.
- [124] Amogh Mannekote, Adam Davies, Juan D Pinto, Shan Zhang, Daniel Olds, Noah L Schroeder, Blair Lehman, Diego Zapata-Rivera, and ChengXiang Zhai, *Large Language Models for Whole-Learner Support: Opportunities and Challenges*, Frontiers in Artificial Intelligence **7** (2024), 1460364.
- [125] Amogh Mannekote, Adam Davies, Juan D. Pinto, Shan Zhang, Daniel Olds, Noah L. Schroeder, Blair Lehman, Diego Zapata-Rivera, and ChengXiang Zhai, *Large Language Models for Whole-Learner Support: Opportunities and Challenges*, Frontiers in Artificial Intelligence **7** (2024), 1460364.
- [126] Noboru Matsuda, William W Cohen, and Kenneth R Koedinger, *Teaching the Teacher: Tutoring Simstudent leads to more effective cognitive tutor authoring*, International Journal of Artificial Intelligence in Education **25** (2015), 1–34, Publisher: Springer.
- [127] Noboru Matsuda, Annalies Ramamurthy, Gabriel J Stylianides, and Kenneth R Koedinger, *Tell Me How to Teach, i’ Ll Learn How to Solve Problems*, Intelligent Tutoring Systems (2016), 111–122.
- [128] Shikib Mehri and Maxine Eskenazi, *Schema-guided paradigm for zero-shot dialog*, Proceedings of the 22nd Annual Meeting of the Special Interest Group on Discourse and Dialogue (Singapore and Online), Association for Computational Linguistics, July 2021, pp. 499–508.
- [129] Tianyi Men, Pengfei Cao, Zhuoran Jin, Yubo Chen, Kang Liu, and Jun Zhao, *Unlocking the future: Exploring look-ahead planning mechanistic interpretability in large language models*, arXiv preprint arXiv:2406.16033 (2024).
- [130] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov, *Locating and Editing Factual Associations in gpt*, 2023.
- [131] Julian Minder, Kevin Du, Niklas Stoehr, Giovanni Monea, Chris Wendler, Robert West, and Ryan Cotterell, *Controllable Context Sensitivity and the Knob Behind It*, The Thirteenth International Conference on Learning Representations, 2025.

- [132] Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi, *Cross-task generalization via natural language crowdsourcing instructions*, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (Dublin, Ireland), Association for Computational Linguistics, May 2022, pp. 3470–3487.
- [133] Behnam Mohammadi, *Wait, It's All Token Noise? Always Has Been: Interpreting llm Behavior Using Shapley Value*, March 2024, arXiv:2404.01332 [cs].
- [134] Ida Momennejad, Hosein Hasanbeig, et al., *Evaluating Cognitive Maps and Planning in Large Language Models with Cogeval*, Advances in Neural Information Processing Systems, vol. 36, 2023.
- [135] Lily Morse, Mihnea HM Teodorescu, Yazeed Awwad, and Gerald C Kane, *Do the Ends Justify the Means? Variation in the Distributive and Procedural Fairness of Machine Learning Algorithms*, Journal of Business Ethics **178** (2022), 673–691.
- [136] Johannes E. M. Mosig, Shikib Mehri, and Thomas Kober, *Star: A Schema-Guided Dialog Dataset for Transfer Learning*, arXiv:2010.11853 [cs] (2020), arXiv: 2010.11853.
- [137] Tom Murray, *Authoring Intelligent Tutoring Systems: An Analysis of the State of the Art*, International Journal of Artificial Intelligence in Education **10** (1999), no. 1, 98–129.
- [138] Vahid Noroozi, Yang Zhang, Evelina Bakhturina, and Tomasz Kornuta, *A Fast and Robust bert-based Dialogue State Tracker for Schema-Guided Dialogue Dataset*, arXiv:2008.12335 [cs, stat] (2020) (en), arXiv: 2008.12335.
- [139] NVIDIA, *NVIDIA Nemotron 3 Super*, <https://research.nvidia.com/labs/nemotron/Nemotron-3-Super/>, 2026, Technical report and model release page.
- [140] OpenAI, *gpt-oss-120b and gpt-oss-20b model card*, 2025.
- [141] Hadas Orgad, Michael Toker, Zorik Gekhman, Roi Reichart, Idan Szpektor, Hadas Kotek, and Yonatan Belinkov, *Llms Know More Than They Show: On the Intrinsic Representation of llm Hallucinations*, arXiv preprint arXiv:2410.02707 (2024).
- [142] Kentaro Ozeki, Risako Ando, Takanobu Morishita, Hirohiko Abe, Koji Mineshima, and Mitsuhiro Okada, *Exploring reasoning biases in large language models through syllogism: Insights from the NeuBAROCO dataset*, Findings of the Association for Computational Linguistics ACL 2024 (Bangkok, Thailand and virtual meeting) (Lun-Wei Ku, Andre Martins, and Vivek Srikumar, eds.), Association for Computational Linguistics, August 2024, pp. 16063–16077.
- [143] J. et al. Park, *Generative Agent-Based Modeling: Using Large Language Models to Simulate Human Behavior*, Proceedings of the AAAI Conference on Artificial Intelligence, 2023.

- [144] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein, *Generative Agents: Interactive Simulacra of Human Behavior*, August 2023, arXiv:2304.03442 [cs].
- [145] Joon Sung Park, Lindsay Popowski, Carrie Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein, *Social Simulacra: Creating Populated Prototypes for Social Computing Systems*, Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology, 2022, pp. 1–18.
- [146] Joon Sung Park, Carolyn Q Zou, Aaron Shaw, Benjamin Mako Hill, Carrie Cai, Meredith Ringel Morris, Robb Willer, Percy Liang, and Michael S Bernstein, *Generative Agent Simulations of 1,000 People*, arXiv preprint arXiv:2411.10109 (2024).
- [147] Avinash Patil and Aryan Jadon, *Advancing Reasoning in Large Language Models: Promising Methods and Approaches*, arXiv preprint arXiv:2502.03671 (2025).
- [148] Baolin Peng, Chenguang Zhu, Chunyuan Li, Xiujun Li, Jinchao Li, Michael Zeng, and Jianfeng Gao, *Few-shot natural language generation for task-oriented dialog*, Findings of the Association for Computational Linguistics: EMNLP 2020 (Online), Association for Computational Linguistics, November 2020, pp. 172–182.
- [149] Aske Plaat, Andrew Wong, Suzan Verberne, Joost Broekens, et al., *Multi-Step Reasoning with Large Language Models, a Survey*, ACM Computing Surveys (2025).
- [150] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu, *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, J. Mach. Learn. Res. **21** (2020), no. 1.
- [151] Robin Jephthah Rajarathinam, Christian Palaguachi, and Jina Kang, *Enhancing Multimodal Learning Analytics: A Comparative Study of Facial Features Captured Using Traditional vs 360-Degree Cameras in Collaborative Learning*, Proceedings of the 17th International Conference on Educational Data Mining (Atlanta, Georgia, USA) (Benjamin PaaÅYen and Carrie Demmans Epp, eds.), International Educational Data Mining Society, July 2024, pp. 551–558.
- [152] Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta, and Pranav Khaitan, *Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset*, Proceedings of the AAAI Conference on Artificial Intelligence **34** (2020), no. 05, 8689–8696 (en), Number: 05.
- [153] Justin Reich and Mizuko Ito, *On the Promise of Personalized Learning for Educational Equity*, npj Science of Learning **8** (2023), no. 1, 15.
- [154] Alexander Renkl and Robert K Atkinson, *Interactive Learning Environments: Contemporary Issues and Trends. an Introduction to the Special Issue*, Educational Psychology Review **19** (2007), 235–238.

- [155] Frank E Ritter, Farnaz Tehranchi, and Jacob D Oury, *Act-r: a Cognitive Architecture for Modeling Cognition*, Wiley Interdisciplinary Reviews: Cognitive Science **10** (2019), no. 3, e1488.
- [156] Fernando J Rodríguez, Kimberly Michelle Price, and Kristy Elizabeth Boyer, *Exploring the Pair Programming Process: Characteristics of Effective Collaboration*, Proceedings of the 2017 Acm Sigcse Technical Symposium on Computer Science Education, 2017, pp. 507–512.
- [157] Bernardino Romera-Paredes and Philip Torr, *An Embarrassingly Simple Approach to Zero-Shot Learning*, International Conference on Machine Learning, PMLR, 2015, pp. 2152–2161.
- [158] Luca Rossi, Katherine Harrison, and Irina Shklovski, *The Problems of llm-generated Data in Social Science Research*, Sociologica **18** (2024), no. 2, 145–168.
- [159] Daniel Rough and Benjamin Cowan, *Don’t Believe the Hype! White Lies of Conversational User Interface Creation Tools*, Proceedings of the 2nd Conference on Conversational User Interfaces (New York, NY, USA), CUI ’20, Association for Computing Machinery, 2020.
- [160] Oscar Sainz, Jon Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre, *NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark*, Findings of the Association for Computational Linguistics: EMNLP 2023 (Singapore) (Houda Bouamor, Juan Pino, and Kalika Bali, eds.), Association for Computational Linguistics, December 2023, pp. 10776–10787.
- [161] Marko Sarstedt, Susanne J Adler, Lea Rau, and Bernd Schmitt, *Using Large Language Models to Generate Silicon Samples in Consumer and Marketing Research: Challenges, Opportunities, And Guidelines*, Psychology & Marketing **41** (2024), no. 6, 1254–1270.
- [162] Dale H Schunk and Ellen L Usher, *Assessing Self-Efficacy for Self-Regulated Learning*, Handbook of Self-Regulation of Learning and Performance, Routledge, 2011, pp. 282–297.
- [163] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr, *Quantifying Language Models’ Sensitivity to Spurious Features in Prompt Design or: How i learned to start worrying about prompt formatting*, arXiv preprint arXiv:2310.11324 (2023).
- [164] Pararth Shah, Dilek Hakkani-Tür, Bing Liu, and Gokhan Tür, *Bootstrapping a Neural Conversational Agent with Dialogue Self-Play, Crowdsourcing and on-Line Reinforcement Learning*, Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 3 (Industry Papers) (New Orleans - Louisiana) (Srinivas Bangalore, Jennifer Chu-Carroll, and Yunyao Li, eds.), Association for Computational Linguistics, June 2018, pp. 41–51.

- [165] Murray Shanahan, Kyle McDonell, and Laria Reynolds, *Role Play with Large Language Models*, *Nature* **623** (2023), 493–498.
- [166] Yunfan Shao, Linyang Li, Junqi Dai, and Xipeng Qiu, *Character-Llm: a Trainable Agent for Role-Playing*, arXiv preprint arXiv:2310.10158 (2023).
- [167] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen, *Synthetic prompting: Generating chain-of-thought demonstrations for large language models*, ICML, 2023.
- [168] Navya Nishith Sharan and Daniela Maria Romano, *The Effects of Personality and Locus of Control on Trust in Humans versus Artificial Intelligence*, *Heliyon* **6** (2020), no. 8.
- [169] Hua Shen, Nicholas Clark, and Tanushree Mitra, *Mind the Value-Action Gap: Do llms Act in Alignment With Their Values?*, arXiv preprint arXiv:2501.15463 (2025).
- [170] Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar, *The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity*, arXiv preprint arXiv:2506.06941 (2025).
- [171] Valerie J Shute and Joseph Psotka, *Intelligent Tutoring Systems: Past, Present, and Future*, (1994).
- [172] Eliot R Smith and Frederica R Conrey, *Agent-Based Modeling: a New Approach for Theory Building in Social Psychology*, *Personality and social psychology review* **11** (2007), no. 1, 87–104.
- [173] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar, *Scaling LLM test-time compute optimally can be more effective than scaling model parameters*, ICLR, 2025.
- [174] Yufei Tao, Adam Hiatt, Erik Haake, Antonie J. Jetter, and Ameeta Agrawal, *When Context Leads but Parametric Memory Follows in Large Language Models*, *ArXiv abs/2409.08435* (2024).
- [175] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al., *Gemma 2: Improving Open Language Models at a Practical Size*, arXiv preprint arXiv:2408.00118 (2024).
- [176] Jennifer Tsan, Jessica Vandenberg, Zarifa Zakaria, Joseph B Wiggins, Alexander R Webber, Amanda Bradbury, Collin Lynch, Eric Wiebe, and Kristy Elizabeth Boyer, *A Comparison of Two Pair Programming Configurations for Upper Elementary Students*, *Proceedings of the 51st ACM technical symposium on computer science education*, 2020, pp. 346–352.

- [177] Ruslan Turtayev, Natalia Fedorova, Oleg Serikov, Svetlana Koldyba, and Roman Kupriyanov, *Misalignment Bounty: Crowdsourcing ai Agent Misbehavior*, arXiv preprint arXiv:2510.19738 (2025).
- [178] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati, *On the Planning Abilities of Large Language Models-a Critical Investigation*, Advances in Neural Information Processing Systems **36** (2023), 75993–76005.
- [179] Kurt VanLehn, *The Behavior of Tutoring Systems*, International Journal of Artificial Intelligence in Education **16** (2006), no. 3, 227–265 (en).
- [180] Alexander Sasha Vezhnevets, John P Agapiou, Avia Aharon, Ron Ziv, Jayd Matyas, Edgar A Duéñez-Guzmán, William A Cunningham, Simon Osindero, Danny Karmon, and Joel Z Leibo, *Generative Agent-Based Modeling with Actions Grounded in Physical, Social, or Digital Space Using Concordia*, arXiv preprint arXiv:2312.03664 (2023).
- [181] Lev S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*, Harvard University Press, Cambridge, MA, 1978.
- [182] Nicolas Wagner and Stefan Ultes, *On the controllability of large language models for dialogue interaction*, SIGDIAL, 2024.
- [183] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al., *A Survey on Large Language Model Based Autonomous Agents*, Frontiers of Computer Science **18** (2024), no. 6, 186345.
- [184] Noah Wang, Z.y. Peng, Haoran Que, Jiaheng Liu, Wangchunshu Zhou, Yuhan Wu, Hongcheng Guo, Ruitong Gan, Zehao Ni, Jian Yang, Man Zhang, Zhaoxiang Zhang, Wanli Ouyang, Ke Xu, Wenhao Huang, Jie Fu, and Junran Peng, *RoleLLM: Benchmarking, eliciting, and enhancing role-playing abilities of large language models*, Findings of the Association for Computational Linguistics: ACL 2024 (Bangkok, Thailand) (Lun-Wei Ku, Andre Martins, and Vivek Srikumar, eds.), Association for Computational Linguistics, August 2024, pp. 14743–14777.
- [185] Peng Wang, Ningyu Zhang, Xin Xie, Yunzhi Yao, Bozhong Tian, Mengru Wang, Zekun Xi, Siyuan Cheng, Kangwei Liu, Guozhou Zheng, et al., *Easyedit: An Easy-to-Use Knowledge Editing Framework for Large Language Models*, arXiv preprint arXiv:2308.07269 (2023).
- [186] Qian Wang, Jiaying Wu, Zhenheng Tang, Bingqiao Luo, Nuo Chen, Wei Chen, and Bingsheng He, *What Limits llm-Based Human Simulation: Llms or Our Design?*, arXiv preprint arXiv:2501.08579 (2025).
- [187] Xi Wang, Hongliang Dai, Shen Gao, and Piji Li, *Characteristic ai agents via large language models*, arXiv preprint arXiv:2403.12368 (2024).

- [188] Xintao Wang, Yunze Xiao, Jen-tse Huang, Siyu Yuan, Rui Xu, Haoran Guo, Quan Tu, Yaying Fei, Ziang Leng, Wei Wang, et al., *Incharacter: Evaluating Personality Fidelity in Role-Playing Agents through Psychological Interviews*, arXiv preprint arXiv:2310.17976 (2023).
- [189] Yaqing Wang, Quanming Yao, James T Kwok, and Lionel M Ni, *Generalizing from a Few Examples: a Survey on Few-Shot Learning*, ACM computing surveys (csur) **53** (2020), no. 3, 1–34.
- [190] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi, *Self-instruct: Aligning language models with self-generated instructions*, ACL, 2023.
- [191] Jonathan A Ward, Andrew J Evans, and Nicolas S Malleson, *Dynamic Calibration of Agent-Based Models Using Data Assimilation*, Royal Society open science **3** (2016), no. 4, 150703.
- [192] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, Advances in neural information processing systems **35** (2022), 24824–24837.
- [193] Daniel Weitekamp, Erik Harpstead, and Ken R. Koedinger, *An Interaction Design for Machine Teaching to Develop ai Tutors*, Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (Honolulu HI USA), ACM, April 2020, pp. 1–11 (en).
- [194] Etienne Wenger, *Communities of Practice: Learning, Meaning, and Identity*, Cambridge University Press, 1998.
- [195] Jason Xianghua Wu, Yan Wu, Kay-Yut Chen, and Lei Hua, *Building Socially Intelligent ai Systems: Evidence From the Trust Game Using Artificial Agents with Deep Learning*, Management Science **69** (2023), no. 12, 7236–7252.
- [196] Yang Xiao, Yi Cheng, Jinlan Fu, Jiashuo Wang, Wenjie Li, and Pengfei Liu, *How Far Are We from Believable Ai Agents? a Framework for Evaluating the Believability of Human Behavior Simulation*, arXiv preprint arXiv:2312.17115 (2023).
- [197] Chengxing Xie, Canyu Chen, Feiran Jia, Ziyu Ye, Shiyang Lai, Kai Shu, Jindong Gu, Adel Bibi, Ziniu Hu, David Jurgens, et al., *Can Large Language Model Agents Simulate Human Trust Behavior?*, theThirty-Eighth Annual Conference on Neural Information Processing Systems, 2024.
- [198] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su, *Travelplanner: a Benchmark for Real-World Planning with Language Agents*, arXiv preprint arXiv:2402.01622 (2024).
- [199] Songlin Xu and Xinyu Zhang, *Leveraging Generative Artificial Intelligence to Simulate Student Learning Behavior*, arXiv preprint arXiv:2310.19206 (2023).

- [200] Ming Yan, Ruihao Li, Hao Zhang, Hao Wang, Zhilan Yang, and Ji Yan, *Larp: Language-Agent Role Play for Open-World Games*, arXiv preprint arXiv:2312.17653 (2023).
- [201] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, *React: Synergizing Reasoning and Acting in Language Models*, International Conference on Learning Representations (ICLR), 2023.
- [202] Yunzhi Yao, Peng Wang, Bozhong Tian, Siyuan Cheng, Zhoubo Li, Shumin Deng, Huajun Chen, and Ningyu Zhang, *Editing Large Language Models: Problems, Methods, And Opportunities*, 2023.
- [203] Qinyuan Ye, Maxamed Ahmed, Reid Pryzant, and Fereshte Khani, *Prompt Engineering a Prompt Engineer*, Findings of the Association for Computational Linguistics: ACL (2024), 329–345.
- [204] Dingli Yu, Simran Kaur, Arushi Gupta, Jonah Brown-Cohen, Anirudh Goyal, and Sanjeev Arora, *Skill-Mix: a Flexible and Expandable Family of Evaluations for Ai Models*, arXiv preprint arXiv:2310.17567 (2023).
- [205] J Zamfirescu-Pereira, Richmond Wong, Bjoern Hartmann, and Qian Yang, *Why Johnny can’t prompt: how non-ai experts try (and fail) to design llm prompts*, Proceedings of the 2023 CHI conference on human factors in computing systems (CHI’ 23), 2023.
- [206] Ningyu Zhang, Yunzhi Yao, Bozhong Tian, Peng Wang, Shumin Deng, Mengru Wang, Zekun Xi, Shengyu Mao, Jintian Zhang, Yuansheng Ni, et al., *acomprehensive Study of Knowledge Editing for Large Language Models*, arXiv preprint arXiv:2401.01286 (2024).
- [207] Yi Zhang, Zhe Li, and Yongchao Zhang, *Validation and Calibration of an Agent-Based Model: a Surrogate Approach*, Discrete Dynamics in Nature and Society **2020** (2020), no. 1, 6946370.
- [208] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh, *Calibrate before Use: Improving Few-Shot Performance of Language Models*, June 2021, arXiv:2102.09690 [cs].
- [209] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica, *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, NeurIPS Datasets and Benchmarks, 2023.
- [210] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang, *Memorybank: Enhancing Large Language Models with Long-Term Memory*, Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, 2024, pp. 19724–19731.

BIOGRAPHICAL SKETCH

Amogh Mannekote was born in Bengaluru, India. He completed his Bachelor of Engineering in Computer Science and Engineering at Ramaiah Institute of Technology in 2018. Following his undergraduate education, he worked in applied machine learning and conversational AI roles in India and abroad, including positions at IBM Watson Machine Learning, Rasa, Fractal Analytics, Crux Intelligence, and Stride.AI.

In 2021, Amogh began his doctoral studies in Computer Science at the University of Florida, where he received his Ph.D. in 2026. His research lies at the intersection of artificial intelligence, human-computer interaction, and educational technology. More specifically, he studies the reliability, evaluation, and calibration of large language model agents in interactive settings, including multi-turn dialogue, semantic parsing, synthetic-data generation, and simulations of human learners. His work has appeared in venues including AAI, COLING, and the ACM Conference on Conversational User Interfaces. He has also conducted research internships at Amazon Alexa and Openstream.ai, where he developed evaluation methods for tool-using language models, dialogue systems, and natural-language feedback.